

# 药材的烘干问题

——一维轴对称热湿耦合模型、有限体积求解与判据口径分析

## 摘要

本文针对中药材热风烘干过程中的传热传质问题，建立一维轴对称径向热湿耦合数学模型，采用半控制体有限体积法（FVM）与 L-稳定的隐式时间推进求解，依次回答四个问题，并以解析基准、收敛阶、守恒收支、退化一致性、灵敏度与稳健性六类检验验证模型。

**问题 1**（预热平衡，0–1800 s，附录 2 常物性）：温度场表面升温快、中心滞后，1800 s 时中心 33.58 °C、表面 36.79 °C，径向温差 3.21 °C；水分场干燥完全被限制在表层，中心含水率 30 min 内完全未变，表面降幅 40.0%。根因是传质特征时间比传热大两个数量级（ $R_0^2/\alpha \approx 2369$  s 对  $R_0^2/D \approx 8 \times 10^4$  s）。由于附录 2 的  $D(C)$  不含温度，问题 1 存在结构性单向耦合，可先解水分、再解温度，无需双向迭代。

**问题 2**（整个烘干过程，附录 3 变物性 + 双向耦合）：前 3 h 温度已接近烘房温度（中心 49.8494 °C、表面 49.9666 °C），含水率显著下降但远未达终点（中心 1.7662、表面 1.0081）。与问题 1 的单向耦合结果相比，双向耦合使 1800 s 中心温度低 1.386 °C——变物性使热扩散率降低 14%，而  $D$  的 Arrhenius 温度依赖又反馈压低扩散系数，这正是“问题 1 可先水后温、问题 2 必须联立”的直接数值体现。

**问题 3**（烘干时间）：按题面“药材各处的水分浓度应低于 0.15 kg/kg”，判据必须是全域最大值  $C_{\max}(t) = \max_r C(r, t)$ 。实测  $C_{\max}(t)$  与中心含水率几乎重合，说明中心是最难干燥的位置。求得  $t_* = 206959.9521$  s = **57.49 h** = 2.3954 天。若误用面积平均判定，会把 2.4 天的工艺说成 1.50 天（低估 37.3%）；误用表面值则低估 78.2%。

**问题 4**（考虑尺寸变化）：引入材料坐标  $\xi = r/R(t)$ ，在均匀径向仿射收缩假设下  $\dot{R}$  对流项严格为零（可证，非近似），且扩散项  $\propto 1/R^2$  与表面项  $\propto 1/R$  不可合并。采用附录 4 物性，在  $\Delta t = 1$  s 的收敛口径下求得  $t_{\text{dry}} = 183926$  s = **51.0906 h** = 2.1288 天，终态半径 1.2000 cm。与另一套独立实现（MATLAB、 $N = 160$ 、 $\Delta t = 10$  s 给出 51.20 h）相差 0.2%，构成跨实现交叉验证。A/B/C 顺序差分显示：换用附录 4 物性使干燥显著变慢（固定半径时 > 120 h 仍未达标），尺寸收缩又大幅加速（ $\geq 68.9$  h），净效果为加快 6.40 h。

**模型检验**：Bessel 解析基准最大绝对误差  $< 1.2 \times 10^{-3}$  °C（相对  $< 0.002\%$ ）；空间收敛阶实测 +2.20（理论 +2），时间不低于一阶（不声称二阶）；内部界面通量成对抵消达机器精度  $1.78 \times 10^{-16}$ ；五项退化与一致性检验通过。 $t_*$  的误差三分量为事件定位 0.5 ms / 时间积分 1.86 s / 空间离散 17.4 s（主导），故  $t_*$  只报告到 2 位小数。

**灵敏度与稳健性**：敏感度排序在五档扰动下均为  $D \gg h_m \gg h$ ，但  $|S_D|$  在  $\pm 5\%$  处为 0.886、在  $-50\%$  处升至 1.794——灵敏度系数是局部量，报告时必须注明档位。数据扰动（删 10% 数据 / 量化噪声 / 传感器噪声 / 换插值算法）引起的散布为 0.22 ~ 67.6 s，全部经由“长时平台均值”单一渠道传递（ $r = -0.9997$ ）；而平台窗口取法一项即达 1090 s，比最大的数据扰动还大 8 倍——本文可信度瓶颈在建模约定而非数据质量。

**主要创新点**：（1）收缩体的材料坐标处理， $\dot{R}$  对流项可证为零、两项  $R$  幂次不可合并，使问题 4 与问题 2/3 共用同一套内核并可交叉验证；（2）提出“可忽略性必须绑定判据口径”的诊断范式——端面效应在“全域最大值”口径下  $< 10^{-5}$ 、在“体积平均”口径下  $-2.2\% \sim -6.2\%$ ，同一物理效应两种口径差 3 个数量级。

**关键词**：热风烘干；圆柱体传热传质；有限体积法；水分有效扩散系数；收缩模型；移动边界；判据口径

## 1 问题重述

### 1.1 问题背景

干燥是决定中药材成品品质的关键工序之一，热风烘干是常见方式。该方式主要包括**预热平衡**和**恒温干燥**两个阶段，通过调控烘房温湿环境完成药材干燥。工艺参数选取不当会导致干燥效率低、能耗高、成品品质不稳定。传统试验优化模式成本高、周期长，需要借助数理分析与数值仿真得到干燥规律。

### 1.2 问题提出

- 问题 1：**建立**预热平衡阶段**药材温度和水分浓度变化规律的数学模型，给出指定时刻、指定位置的结果，并保存完整结果。
- 问题 2：**建立**整个烘干过程**药材温度和水分浓度变化规律的数学模型，给出指定时刻、指定位置的结果，并保存完整结果。
- 问题 3：**按烘干要求（药材各处水分浓度低于  $0.15 \text{ kg/kg}$ ），确定药材烘干所需时间。
- 问题 4：**考虑药材因水分流失发生**尺寸变化**，确定药材的烘干时长。

### 1.3 已知条件

药材为圆柱体，长  $L_0 = 0.25 \text{ m}$ 、初始半径  $R_0 = 0.02 \text{ m}$ ；初始温度  $28^\circ\text{C}$ 、初始干基含水率  $2.55 \text{ kg/kg}$ 。对流换热系数  $h = 25 \text{ W}/(\text{m}^2\cdot\text{K})$ 、对流传质系数  $h_m = 8 \times 10^{-7} \text{ m/s}$ 。题目给出三套物性经验公式：附录 2（问题 1，常物性）、附录 3（问题 2/3，变物性）、附录 4（问题 4，变物性）；附件 1 给出烘房温度与水分浓度的实测序列（241 点，0–14400 s），附件 2 给出药材半径随时间的变化（145 点，0–259200 s）。

## 2 问题分析

### 2.1 几何与降维

药材为圆柱，长  $L_0 = 0.25 \text{ m}$ 、初始半径  $R_0 = 0.02 \text{ m}$ 。题目所有输出只到“距中心  $2 \text{ cm}$ ”，即最大距离恰为初始半径，说明**径向**是主方向。端面与侧面的面积比约为  $R_0/L_0 = 0.08$ ；端面方向与径向的扩散时间尺度比为

$$\frac{t_z}{t_r} \sim \left( \frac{L_0/2}{R_0} \right)^2 = \left( \frac{0.125}{0.02} \right)^2 \approx 39.06. \quad (1)$$

即端面方向的时间尺度比径向大近 **40 倍**，故本文采用**一维轴对称径向模型**，并把端面效应作为独立对照定量验证（§9.4）。

图 G1 几何、边界与求解路线

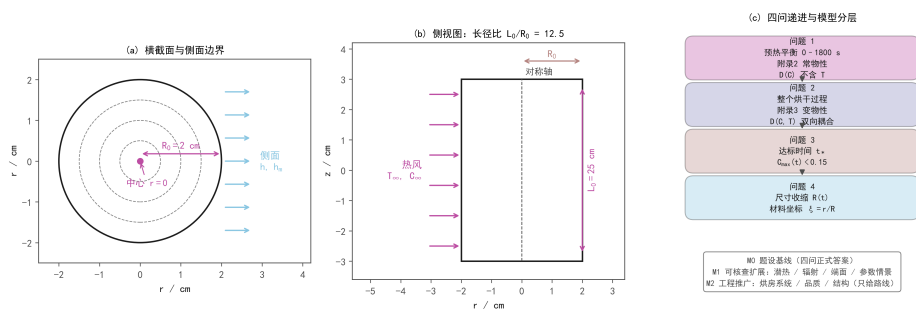


图 G1 几何、边界与求解路线。(a) 横截面与侧面边界；(b) 侧视图，长径比 12.5；(c) 四问递进与 M0/M1/M2 分层

## 2.2 变量定义

严格区分两个同为  $\text{kg/kg}$  但不能相减的量：

$$C = \frac{m_w}{m_d} \quad (\text{药材干基含水率}), \quad Y = \frac{m_v}{m_{da}} \quad (\text{空气含湿量}). \quad (2)$$

本文另引入  $C^\infty$  表示已明确基准的等效环境平衡含水率，即附件 1 的水分浓度列按题设等效约定读入的结果。

## 2.3 技术路线、四问递进与模型分层

技术路线如图 G2：明确物理机制与变量定义 → 建立偏微分方程模型 → 有限体积法 (FVM) 与隐式时间推进求解 → 依次回答四问 → 六类检验验证模型。四问递进为：问题 1（预热平衡，附录 2 常物性， $D$  不含  $T$ ）→ 问题 2（整个烘干过程，附录 3 变物性， $D(C, T)$  双向耦合）→ 问题 3（确定全域最大含水率  $C_{\max}$  首次低于  $0.15 \text{ kg/kg}$  的时刻）→ 问题 4（引入材料坐标与均匀径向仿射收缩，采用附录 4 物性）。

模型分三层：M0 题设基线（必做，是四问的正式答案）；M1 可核查扩展（潜热、辐射、端面、结壳、参数情景，条件充分时做，不修订 M0 答案）；M2 工程推广（烘房系统、品质、结构，只给路线与数据需求）。

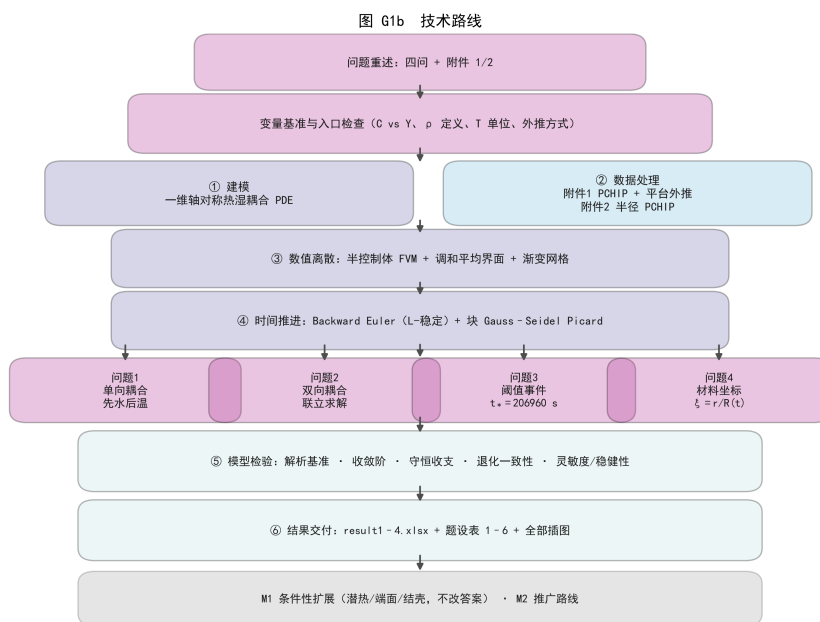


图 G2 技术路线流程图

## 2.4 本文的创新点

1. 收缩体的材料坐标处理。常见做法有两种且都有代价：在物理坐标下重划网格，离散引入的数值扩散会伪装成物理扩散；只把坐标归一化却不改方程，则悄悄丢掉收缩的通量集中效应。本文用材料坐标  $\xi = r/R(t)$ ，在均匀仿射收缩下严格推得  $\dot{R}$  对流项恒为零（可证，不是近似），且扩散项  $\propto 1/R^2$  与表面项  $\propto 1/R$  不可合并（很多文献把两者写成同一幂次，在  $R$  减半时会产生百分之几十的系统误差）。故问题 4 与问题 2/3 共用同一套 FVM 内核， $R \equiv R_0$  时逐点退化，构成内建回归。
2. “可忽略性必须绑定判据口径”的诊断范式。端面效应在文献里常被引为“3.5% ~ 6.0%，不可忽略”。本文以二维轴对称有限圆柱做对照，发现：在全域最大含水率口径下为  $< 10^{-5}$ （可忽略），在体积平均口径下为  $-2.2\% \sim -6.2\%$ （与文献吻合）——同一物理效应，两种

口径差 3 个数量级。故本文的结论不是“端面效应可忽略”，而是：任何“可忽略 / 不可忽略”的判断都必须同时给出它绑定在哪个判据口径上；文献之间的分歧往往是口径分歧而非物理分歧。

3. 三项方法学处理：(a) 问题 1 的结构性单向耦合——附录 2 的  $D$  不含温度，可先解水分再解温度，并用双向耦合求解器交叉验证；(b) **A/B/C 顺序差分**——把“物性公式”与“几何收缩”两个效应拆开报告（见 §9.4），并明确它不是“唯一、独立的因果贡献分解”；(c) **三类误差分开报告**—— $t_*$  的误差分解为事件定位 0.5 ms / 时间积分 1.86 s / 空间离散 17.4 s（主导），故  $t_*$  只报到 2 位小数，不把定位精度冒充整体精度。
4. **显式精度口径声明与判据化取舍**。附件 1 只覆盖 0–14400 s，恒温段边界必须假定；本文不隐藏这一点，而是给出 4 种合理取法下  $t_*$  的取值表（跨度 0.30 h = 1090 s），并指出它比全部数值离散误差大 57 倍。对被省略的 13 条因素，给出互斥且穷尽的三类判据（量级 / 数据 / 重复计算），杜绝“文献有但我没做”式的无判据省略（见 §12.2）。

### 3 模型假设

1. **连续介质假设**：药材视为连续介质，温度与含水率在空间上连续分布，可用偏微分方程描述。
2. **轴对称假设**：药材为圆柱，几何与边界条件轴对称，故采用**一维轴对称径向模型**；端面效应以二维轴对称对照验证。
3. **局部热平衡假设**：同一空间点上固相与液相温度相同，不需要分别写两相能量方程。
4. **均匀仿射收缩假设（仅问题 4）**：固定长度 + 均匀径向仿射收缩，材料径向速度  $v_r = (\dot{R}/R)r$ 。必须声明这是假设而非结论——附件 2 只给整体半径  $R(t)$ ，无内部位移数据，单个表面半径不能识别内部位移是否仿射。
5. **物性经验公式适用性**： $\rho$ 、 $c_p$ 、 $k$ 、 $D$  一律采用附录给出的经验公式，不替换文献参数，也不引入题面未给的物性。
6. **环境边界等效约定**：按题设等效约定，把附件 1 的“水分浓度”列作为药材表面的等效平衡含水率  $C_\infty^{\text{eq}}$ ，不假装它是由真实空气状态求出的吸附平衡结果。
7. **一维径向近似的合理性**：端面方向与径向的扩散时间尺度比约 39，端面影响由二维轴对称对照定量验证。
8. **模型分层**：M0 为题设基线（四问正式答案）；M1 为可核查扩展（潜热、辐射、端面、结壳、参数情景），条件充分时做；M2 为工程推广，只给路线与数据需求。

#### 3.1 入口检查（六项，逐条落实）

为避免“用了题面之外的量”这类不可追溯的错误，建模前先做六项入口检查：



表 1 入口检查结论表

| # | 检查项                    | 结论                                                                                                   |
|---|------------------------|------------------------------------------------------------------------------------------------------|
| 1 | 含水率 $C$ 与空气含湿量 $Y$ 的基准 | $C = m_w/m_d$ (干基)、 $Y = m_v/m_{da}$ ; 即使同为 kg/kg 也不可自动相减, 本文引入已明确基准的 $C_\infty^{\text{eq}}$ 表示等效环境值 |
| 2 | 密度 $\rho$ 的定义          | 附录 3/4 给的是湿物料体积密度, 用于热容项; $\rho_d = \rho/(1+C)$ 与之不兼容 (见 §9.4)                                       |
| 3 | 温度单位                   | 内部一律摄氏度, 仅在 $D$ 公式内转开尔文, 转换点全局唯一                                                                     |
| 4 | 长时边界外推方式               | 附件 1 只到 14400 s, 恒温段必须显式假设, 不得用插值函数外推                                                                |
| 5 | 输出口径                   | 严格按附件 3 模板: 表头含反斜杠、距离轴为数值、 <b>result4</b> 末列为字符串“药材表面”且距离轴只到 1.9 cm                                  |
| 6 | 达标判据                   | 题面“各处” $\Rightarrow$ 全域最大值 $C_{\max}(t)$ , 不得用平均值或表面值                                                |

## 4 符号说明

表 2 主要符号说明

| 符号                      | 含义                                | 单位                                    |
|-------------------------|-----------------------------------|---------------------------------------|
| $t$                     | 时间                                | s                                     |
| $r$                     | 到药材中心的距离                          | m                                     |
| $R(t)$                  | 当前半径 (问题 4 随时间变化)                 | m                                     |
| $R_0$                   | 初始半径, $R_0 = 0.02$                | m                                     |
| $L_0$                   | 药材长度, $L_0 = 0.25$                | m                                     |
| $\xi = r/R(t)$          | 归一化径向坐标 (材料坐标)                    | —                                     |
| $T$                     | 药材温度                              | $^{\circ}\text{C}$ ( $D$ 公式内为 K)      |
| $T_\infty$              | 烘房温度                              | $^{\circ}\text{C}$                    |
| $C$                     | 药材干基含水率 $m_w/m_d$                 | kg/kg                                 |
| $Y$                     | 空气含湿量 $m_v/m_{da}$                | kg/kg                                 |
| $C_\infty^{\text{eq}}$  | 等效环境平衡含水率                         | kg/kg                                 |
| $C_{\max}(t)$           | 全域最大含水率 $\max_r C(r, t)$          | kg/kg                                 |
| $\bar{C}(t)$            | 面积平均含水率                           | kg/kg                                 |
| $t_*$                   | 问题 3 的阈值通过时刻                      | s                                     |
| $t_{\text{dry}}$        | 问题 4 的烘干时长                        | s                                     |
| $\rho$                  | 湿物料体积密度                           | $\text{kg}/\text{m}^3$                |
| $\rho_d$                | 干固体密度 $\rho/(1+C)$                | $\text{kg}/\text{m}^3$                |
| $c_p$                   | 比热容                               | $\text{J}/(\text{kg}\cdot\text{K})$   |
| $k$                     | 热传导系数                             | $\text{W}/(\text{m}\cdot\text{K})$    |
| $\alpha = k/(\rho c_p)$ | 热扩散率                              | $\text{m}^2/\text{s}$                 |
| $D$                     | 水分有效扩散系数                          | $\text{m}^2/\text{s}$                 |
| $h$                     | 对流换热系数, $h = 25$                  | $\text{W}/(\text{m}^2\cdot\text{K})$  |
| $h_m$                   | 对流传质系数, $h_m = 8 \times 10^{-7}$  | m/s                                   |
| $\lambda$               | 汽化潜热                              | J/kg                                  |
| $J_w$                   | 向外的水质量通量                          | $\text{kg}/(\text{m}^2\cdot\text{s})$ |
| $e_d(t)$                | 几何—密度一致性诊断量                       | —                                     |
| $Bi_T = hR_0/k$         | 传热毕渥数                             | —                                     |
| $Bi_m = h_m R_0/D$      | 传质毕渥数                             | —                                     |
| $S$                     | 灵敏度系数 $(\Delta Y/Y)/(\Delta P/P)$ | —                                     |
| $\Delta r, \Delta t$    | 空间步长、时间步长                         | m, s                                  |
| $\gamma$                | 渐变网格加密指数 ( $\gamma > 1$ 向表面加密)    | —                                     |

上标与下标约定: 上标“eq”表示“等效”(题设约定), 下标“ $\infty$ ”表示环境值, 下标“s”表示表面值, 下标“0”表示初值。 $\tilde{C}, \tilde{T}$  表示材料坐标下的场量。

## 5 数据处理

### 5.1 环境数据及插值

附件 1 给出烘房温度与水分浓度的实测序列，共 241 点，覆盖 0–14400 s，步长恒为 60 s。

实测发现：温度差分变号 116 次、水分浓度变号 119 次，波动遍布全程，不只是尾部。故本文采用忠实插值（PCHIP 过全部实测点）作为默认版本，受控平滑（Savitzky–Golay）仅作参考对照。

图 G2 附件1 环境数据及插值

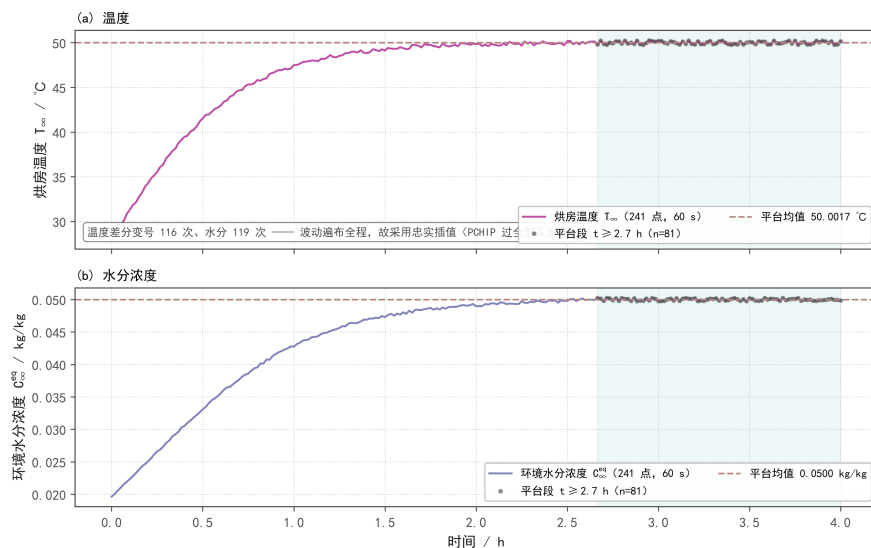


图 G3 附件 1 环境数据。阴影为平台段 ( $t \geq 9600$  s,  $n = 81$ ), 虚线为其均值

图 FIG-13 环境插值 vs 受控平滑对照 (波动遍布全程, 故默认不平滑)

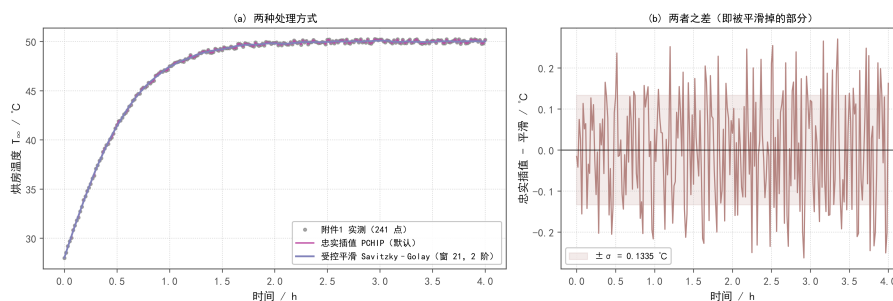


图 G4 环境插值与受控平滑对照。(b) 为两者之差，即被平滑掉的部分；其量级即实测波动水平

### 5.2 阶段识别

以温度达到最终升温 95% 为判据，预热平衡阶段约为 0–4898 s；以升温速率小于  $0.001$   $^{\circ}\text{C}/\text{s}$  为判据，趋于稳定点为 1866 s。本文以 **4898 s** 作为预热/恒温分界。

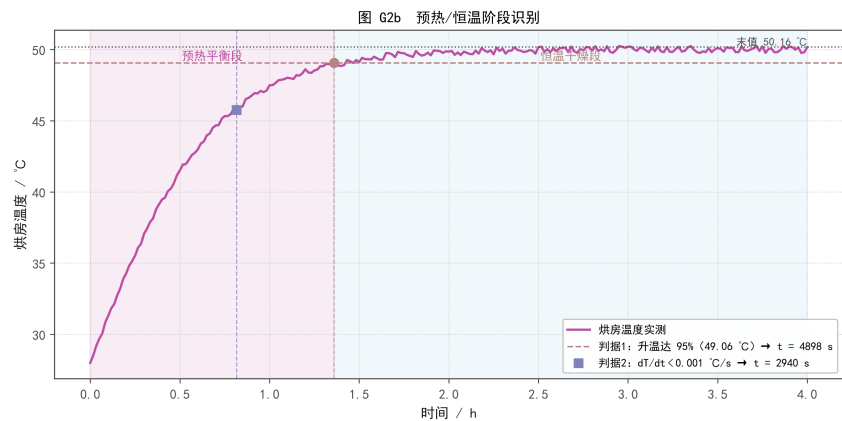


图 G5 预热/恒温阶段识别。两个判据给出的切点相差约 3000 s，本文取较保守的 4898 s

5.3 长时环境边界（必须显式假设）

附件 1 只覆盖 0–14400 s（4 h），而问题 2/3 需 2–3 天，恒温段边界必须显式假设，不能靠插值函数外推。本文采用：

表 3 长时环境边界处理方案

| 时段                       | 处理                                                                                                                                                |
|--------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| $t \leq 14400 \text{ s}$ | PCHIP 过全部 241 个实测点（忠实插值）                                                                                                                          |
| $t > 14400 \text{ s}$    | $T_\infty \equiv 50.0017 \text{ }^\circ\text{C}$ 、 $C_\infty^{\text{eq}} \equiv 0.049998 \text{ kg/kg}$ ( $t \geq 9600 \text{ s}$ 段 $n = 81$ 的均值) |

切换点处的阶跃：温度  $-0.1633 \text{ }^\circ\text{C}$ 、含水率  $+1.38 \times 10^{-4} \text{ kg/kg}$ ——落在实测波动带内（该段标准差： $T$  为  $0.1558 \text{ }^\circ\text{C}$ 、 $C$  为  $1.57 \times 10^{-4}$ ），小于一个标准差。这是“把噪声末点拉回其自身均值”，不是引入新信息。

5.4 半径数据

附件 2 给出药材半径随时间变化，共 145 点，覆盖 0–259200 s（72 h）。半径从 2.000 cm 严格单调递减到 1.198 cm，收缩 40.1%。本文采用 PCHIP 保单调插值拟合  $R(t)$ ，拟合残差  $< 10^{-4} \text{ cm}$  量级。

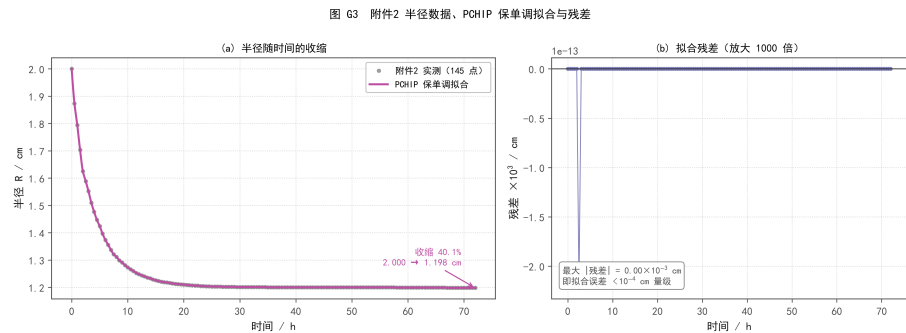


图 G6 附件 2 半径数据、PCHIP 保单调拟合与残差

## 6 问题 1：预热平衡阶段

### 6.1 模型建立

预热平衡阶段采用附录 2 的常物性，温度方程与水分方程分别为

$$\rho c_p \frac{\partial T}{\partial t} = \frac{k}{r} \frac{\partial}{\partial r} \left( r \frac{\partial T}{\partial r} \right), \quad \frac{\partial C}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left( D(C) r \frac{\partial C}{\partial r} \right), \quad (3)$$

$$D(C) = 7 \times 10^{-9} \exp\left(-\frac{0.89}{C}\right) \quad [\text{m}^2/\text{s}]. \quad (4)$$

**结构性单向耦合：** $D$  不含温度，温度方程只通过“物性是否随  $C$  变”与水场耦合，而附录 2 给的是常物性。故问题 1 可严格拆成 **水分场独立求解**  $\rightarrow$  **温度场代入已知  $C(r, t)$  求解**。这不是“为了省事”，而是方程结构本身允许的降维。

**初始条件与边界条件：**

$$T(r, 0) = 28 \text{ }^\circ\text{C}, \quad C(r, 0) = 2.55 \text{ kg/kg}; \quad (5)$$

$$r = 0: \quad \frac{\partial T}{\partial r} = \frac{\partial C}{\partial r} = 0; \quad (6)$$

$$r = R_0: \quad -k \frac{\partial T}{\partial r} \Big|_R = h(T_s - T_\infty), \quad -D \frac{\partial C}{\partial r} \Big|_R = h_m(C_s - C_\infty^{\text{eq}}). \quad (7)$$

### 6.2 数值方法

**空间离散：**采用有限体积法（FVM），中心用**半控制体**（半径  $\Delta r/2$  的实心圆柱），天然规避  $1/r$  奇点；界面通量用**调和平均**，保证变  $D$  时通量正确。

**网格：**均匀网格在表面早期欠分辨（ $t = 1 \text{ s}$  时边界层厚度仅  $0.07 \text{ mm}$ ，均匀  $N = 640$  误差仍达  $4 \times 10^{-4}$ ）。故采用向表面加密的**渐变网格**  $N = 200$ 、 $\gamma = 1.5$ （ $\Delta r = 0.00707 \sim 0.14981 \text{ mm}$ ），以约一半计算量取得优于均匀  $N = 640$  的精度。

**时间推进：**半控制体 FVM 中心处的显式稳定性上限约  $\Delta r^2/(4\alpha)$ ，均匀网格  $\Delta r = 0.5 \text{ mm}$  时为  $0.37 \text{ s}$ ，无法满足  $1 \text{ s}$  输出要求。故采用 L-稳定的 **Backward Euler** 隐式推进，步长由误差控制自适应调整，输出时刻精确落点。

**非线性迭代：**水分方程因  $D(C)$  非线性，采用 **Picard** 迭代，保持 M-矩阵性质， $C$  的正性由构造保证（平均 2.92 次、最多 3 次）。收敛采用尺度归一化的**双重判据**（更新量与真残差都要满足）。

**输出采样：** $r = 0$  用偶函数二次外推， $0 < r < R_0$  用 PCHIP 保形插值， $r = R_0$  用 Robin 边界重构值。

图 FIG-08 一维圆柱有限体积离散（中心半控制体规避  $1/r$  奇点）

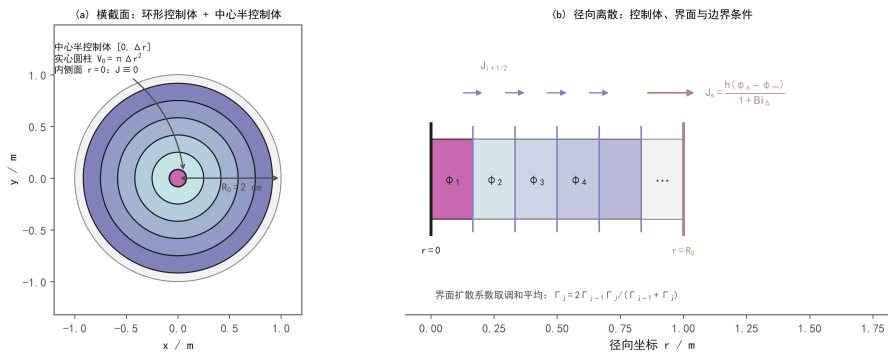


图 G7 一维圆柱有限体积离散。(a) 横截面的环形控制体与中心半控制体；(b) 径向剖面，界面通量与表面 Robin 重构

### 6.3 结果

图 G4 问题1: 预热平衡阶段 (0 ~ 1800 s) 温度与含水率演化

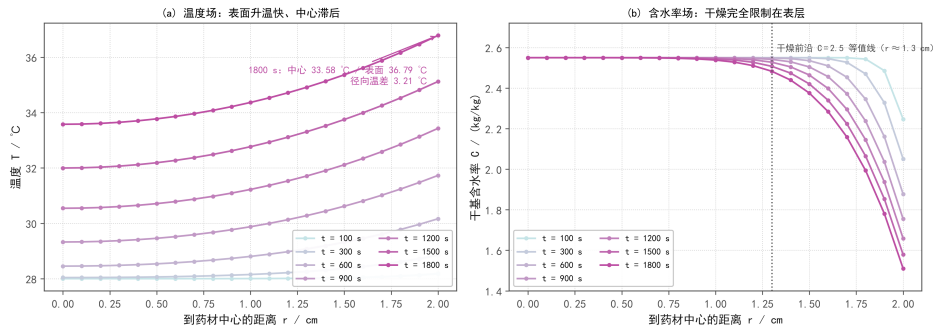


图 G8 问题 1 温度场与含水率场的径向演化 (0-1800 s)

**温度场:** 表面温度从 28 °C 升至约 36.8 °C, 中心升至约 33.6 °C, 表面始终高于中心, 符合外部热风加热的物理过程。

**含水率场:** 中心含水率在 1800 s 内基本不变 (2.55 kg/kg), 表面从 2.55 降至约 1.51 kg/kg, 呈明显的表面干燥特征。

表 4 30 分钟内药材的温度 (°C)

| 时间/s | 0 cm    | 0.5 cm  | 1 cm    | 1.5 cm  | 2 cm    |
|------|---------|---------|---------|---------|---------|
| 100  | 28.0001 | 28.0003 | 28.0039 | 28.0319 | 28.1796 |
| 300  | 28.0408 | 28.0635 | 28.1514 | 28.3674 | 28.8488 |
| 600  | 28.4538 | 28.5365 | 28.8043 | 29.3159 | 30.1643 |
| 900  | 29.3249 | 29.4589 | 29.8763 | 30.6168 | 31.7298 |
| 1200 | 30.5435 | 30.7106 | 31.2231 | 32.1135 | 33.4298 |
| 1500 | 31.9966 | 32.1877 | 32.7672 | 33.7480 | 35.1214 |
| 1800 | 33.5763 | 33.7728 | 34.3646 | 35.3625 | 36.7861 |

表 5 30 分钟内药材的水分浓度 (kg/kg)

| 时间/s | 0 cm   | 0.5 cm | 1 cm   | 1.5 cm | 2 cm   |
|------|--------|--------|--------|--------|--------|
| 100  | 2.5500 | 2.5500 | 2.5500 | 2.5500 | 2.2470 |
| 300  | 2.5500 | 2.5500 | 2.5500 | 2.5492 | 2.0508 |
| 600  | 2.5500 | 2.5500 | 2.5500 | 2.5353 | 1.8770 |
| 900  | 2.5500 | 2.5500 | 2.5497 | 2.5045 | 1.7547 |
| 1200 | 2.5500 | 2.5500 | 2.5482 | 2.4646 | 1.6586 |
| 1500 | 2.5500 | 2.5499 | 2.5445 | 2.4206 | 1.5787 |
| 1800 | 2.5500 | 2.5497 | 2.5383 | 2.3755 | 1.5103 |

完整结果见 result1.xlsx。

### 6.4 分析与讨论

#### (1) 温度: 表面升温快, 中心严重滞后

$t = 1800$  s 时中心 33.5763 °C、表面 36.7861 °C, 径向温差 3.2098 °C, 而中心仅完成烘房升温幅度的  $(33.58 - 28)/(41.51 - 28) = 41.3\%$ 。

这直接印证  $Bi_T = 1.389$ : 内外导热阻力相当, 集总参数模型不可用。

## (2) 水分：干燥完全被限制在表层，中心未被触及

| 位置 | $t = 1800 \text{ s}$ 含水率 | 变化                           | 降幅           |
|----|--------------------------|------------------------------|--------------|
| 中心 | 2.5500                   | <b>0.0000</b> (30 min 内完全未变) | 0            |
| 表面 | 1.5103                   | -1.0073                      | <b>40.0%</b> |

$C = 2.5 \text{ kg/kg}$  的等值线 (“干燥前沿”) 位于  $r \approx 1.3 \text{ cm}$ ——即 30 min 内水分变化只发生在外侧约 **0.7 cm** 的薄层。

## (3) 干燥前沿呈 “由表及里推进” 形态

从表 5 可见：表面持续单调下降；而 1.0 cm 处直到约  $t = 900 \text{ s}$  才开始可见变化，1.5 cm 处到  $t = 1800 \text{ s}$  才降 0.17。这正是 “**表面是唯一的门**、内部水分必须接力外迁” 的直接体现。

## (4) 温度与水分的时间尺度差约 60 倍

传热特征时间  $R_0^2/\alpha \approx 2369 \text{ s}$ ，而传质特征时间  $R_0^2/D$  (按  $D \approx 5 \times 10^{-9} \text{ m}^2/\text{s}$ )  $\approx 8 \times 10^4 \text{ s}$ 。实测表现完全一致：**30 分钟内温度场已显著演化，而水分场只在表层动了 40%**。

这解释了为什么烘干需要 **2-3 天**，而预热只需约 **40 分钟**。

## (5) 表面温度远未跟上烘房温度

$t = 1800 \text{ s}$  时烘房已达  $41.513^\circ\text{C}$ ，而药材表面仅  $36.7861^\circ\text{C}$ ——即表面只完成烘房升温幅度的 65.0%。

注意：这是 M0 (不含蒸发潜热项) 的结果。若计入蒸发吸热，表面温度还会更低。

M1 扩展不得直接套用本结论，必须先通量闭合的前提下重算。

## 7 问题 2：整个烘干过程

## 7.1 模型建立

问题 2 求解整个烘干过程，从  $t = 0$  起统一采用附录 3 的变物性经验公式：

$$\rho(C)c_p(C)\frac{\partial T}{\partial t} = \frac{1}{r}\frac{\partial}{\partial r}\left(k(C)r\frac{\partial T}{\partial r}\right), \quad \frac{\partial C}{\partial t} = \frac{1}{r}\frac{\partial}{\partial r}\left(D(C,T)r\frac{\partial C}{\partial r}\right), \quad (8)$$

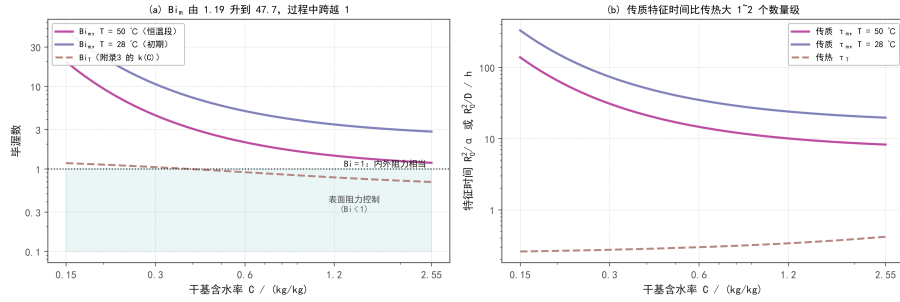
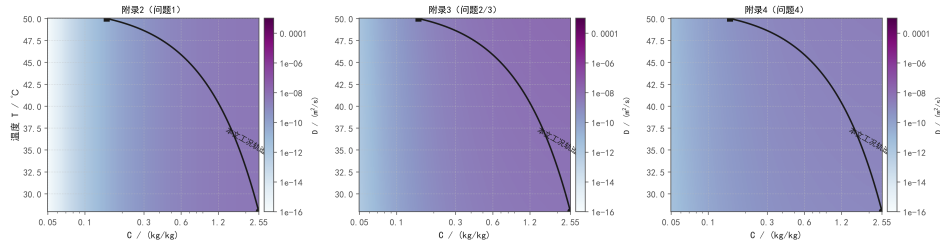
$$D = 2.4 \times 10^{-3} \exp\left(-\frac{0.45}{C}\right) \exp\left[-\frac{3850}{T + 273.15}\right] \quad [\text{m}^2/\text{s}]. \quad (9)$$

题面明确 “为简化问题，相关经验公式统一采用附录 3”，故从  $t = 0$  起就用附录 3，不存在阶段切换，也不需要 “切换处保持状态连续” 的处理。

双向耦合： $D$  依赖  $T$ ，水分场不再独立， $T$  与  $C$  必须联立求解。



图 F1Q-16 无量纲数分析：为什么「烘干 2~3 天」而「预热只要 40 分钟」

图 G9 无量纲数分析。(a)  $Bi_m$  由 1.19 升到 47.7，过程中跨越 1；(b) 传质特征时间比传热大 1~2 个数量级图 F1Q-18 附录 2/3/4 的扩散系数  $D(C, T)$  等值线与本文工况轨迹（注意三者的色标范围相同）图 G10 附录 2/3/4 的扩散系数  $D(C, T)$  等值线与本文工况轨迹（三者色标范围相同）

## 7.2 边界外推与敏感性

长时环境边界的处理方案见 §5.3。本节做情景扫描检验其影响：

表 6 环境外推情景对  $t_*$  的影响

| 情景                                                                                    | $t_*/h$ | $\Delta$ vs 基线 | 相对          |
|---------------------------------------------------------------------------------------|---------|----------------|-------------|
| 基线 ( $t_{pre} = 14400$ s, $T_\infty = 50.0017^\circ C$ , $C_\infty^{eq} = 0.049998$ ) | 57.49   | —              | —           |
| $t_{pre} = 9600$ s                                                                    | 57.49   | $\approx 0$    | $\approx 0$ |
| $T_\infty = 49.8^\circ C$                                                             | 57.87   | +0.39 h        | +0.67%      |
| $T_\infty = 50.3^\circ C$                                                             | 56.95   | -0.54 h        | -0.94%      |
| $C_\infty^{eq} = 0.0495$                                                              | 57.48   | -0.01 h        | -0.02%      |
| $C_\infty^{eq} = 0.0505$                                                              | 57.50   | +0.01 h        | +0.02%      |

由环境外推得到的情景区间： $t_* \in [56.95, 57.87] h = [2.373, 2.411]$  天 ( $\pm 0.8\%$ )。  $t_{pre}$  的取值对结果没有影响——因为 PCHIP 数据段与平台值本身是连续过渡的。

环境平台窗口的选择影响：本文取  $t \geq 9600$  s 段 ( $n = 81$ ) 均值。若改取  $t \geq 12400$  s 段， $t_*$  会从 57.4889 h 变到 57.4747 h，跨度约 0.03 h。这属于建模约定不同，不是模型或算法误差。完整的口径对照见 §8.5。

## 7.3 数值方法

问题 2 在问题 1 内核基础上做两项扩展：

(1) 变物性走守恒形式。面导热系数  $k$  用调和平均、热容 ( $\rho c_p$ ) 用单元值，二者不可合并。若令  $\Gamma = k/(\rho c_p)$  再走原路径，等于把两者在同一个面上一起做调和平均， $\rho c_p$  沿  $r$  变化数倍时会产生明显偏差。守恒形式的能量方程应写成

$$(\rho c_p)_i V_i \frac{T_i^{n+1} - T_i^n}{\Delta t} = \sum_j G_{ij} (T_j^{n+1} - T_i^{n+1}) + \text{边界项}, \quad (10)$$

即  $\rho c_p$  整体除以  $(\rho c_p)_i V_i$ ，而不是把  $k$  与  $c_p$  分别处理。

(2) 块 Picard 用 Gauss-Seidel 序。 $\rho_{cp}$ 、 $k$  对  $C$  的依赖是单向的，用最新  $C$  可显著加快收敛且不破坏任何子问题的 M-矩阵性质。

**残差判据不可省：**只查更新量会落入“更新量变小但残差停滞”的陷阱；本文的双重判据对  $T$ 、 $C$  两个场同时成立才算收敛。

**网格与时间：**生产网格  $N = 200$ 、 $\gamma = 1.5$ ， $\theta = 1$  (Backward Euler)，自适应步长，输出时刻精确落点。3 h 正式版接受步 12652 次、拒绝步 4 次，平均 3.00 次 Picard 迭代。

## 7.4 结果

图 G5 问题2：整个烘干过程前 3 h 的径向分布（附录3 变物性 + 双向耦合）

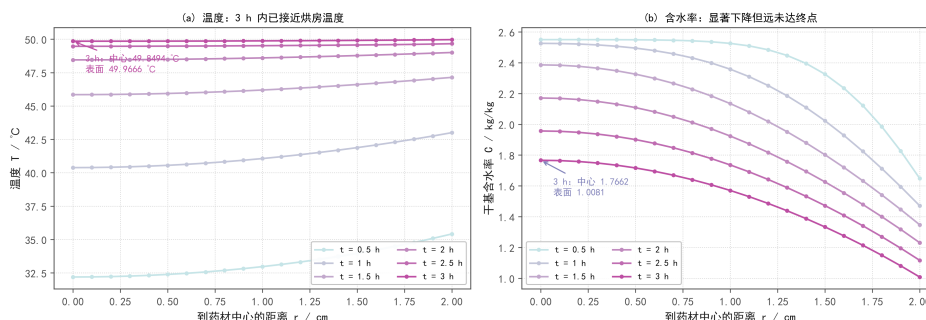


图 G11 问题 2 前 3 h 的径向分布

**温度场：**前 3 h 内整体升温到接近烘房温度； $t = 3$  h 时中心  $49.8494^{\circ}\text{C}$ 、表面  $49.9666^{\circ}\text{C}$ ，表面略高于中心，符合热风加热过程。

**含水率场：**前 3 h 内含水率显著下降但远未达终点； $t = 3$  h 时中心  $1.7662$  kg/kg、表面  $1.0081$  kg/kg，表面先干、中心后干。

表 7 3 小时内药材的温度 ( $^{\circ}\text{C}$ )

| 时间/h | 0 cm    | 0.5 cm  | 1 cm    | 1.5 cm  | 2 cm    |
|------|---------|---------|---------|---------|---------|
| 0.5  | 32.1902 | 32.3830 | 32.9666 | 33.9613 | 35.4136 |
| 1    | 40.3821 | 40.5541 | 41.0608 | 41.8783 | 43.0000 |
| 1.5  | 45.8467 | 45.9348 | 46.1929 | 46.6052 | 47.1395 |
| 2    | 48.4500 | 48.4879 | 48.5982 | 48.7734 | 49.0032 |
| 2.5  | 49.4668 | 49.4791 | 49.5135 | 49.5645 | 49.6617 |
| 3    | 49.8494 | 49.8552 | 49.8746 | 49.9100 | 49.9666 |

表 8 3 小时内药材的水分浓度 (kg/kg)

| 时间/h | 0 cm   | 0.5 cm | 1 cm   | 1.5 cm | 2 cm   |
|------|--------|--------|--------|--------|--------|
| 0.5  | 2.5499 | 2.5489 | 2.5255 | 2.3257 | 1.6486 |
| 1    | 2.5257 | 2.4948 | 2.3578 | 2.0230 | 1.4711 |
| 1.5  | 2.3860 | 2.3256 | 2.1344 | 1.8020 | 1.3476 |
| 2    | 2.1708 | 2.1084 | 1.9236 | 1.6259 | 1.2311 |
| 2.5  | 1.9567 | 1.9006 | 1.7360 | 1.4720 | 1.1166 |
| 3    | 1.7662 | 1.7165 | 1.5702 | 1.3333 | 1.0081 |

完整结果见 result2.xlsx。

## 7.5 与问题 1 的差异分析

问题 2 相比问题 1 有三项扩展：**变物性** ( $D$  含 Arrhenius 温度依赖，方程强非线性)、**双向耦合** ( $T$  与  $C$  必须联立)、**全流程边界** (跨越 2–3 天)。

在  $t = 1800$  s 处, 双向耦合使中心温度降低约 **1.386 °C** (问题 1 为 33.5763 °C; 问题 2 为 32.1902 °C)。这属于模型差异分析, 不是两套实现必须相等的正确性验证。

差异方向值得注意: 表面上附录 3 的  $D$  在 30 °C 附近反而略大于附录 2 ( $D_{\text{app3}}(2.55, 30\text{ °C}) = 5.64 \times 10^{-9}$  对  $D_{\text{app2}}(2.55) = 4.94 \times 10^{-9}$ ), 本应干燥更快; 但实测问题 2 更冷、更湿。原因正是双向耦合:

- 附录 3 的  $\rho c_p$  更大、 $k$  更高, 热扩散率低 14%  $\rightarrow$  升温更慢;
- 而  $D$  含 Arrhenius 因子  $\exp(-3850/T_K)$  (约 4%/K), 温度偏低又反馈压低扩散系数。

这正是“问题 1 可以先水后温、问题 2 必须联立”的直接数值体现。

作为对照, M1 在 M0 基础上开启蒸发潜热后, 前 3 h 中心温度额外降低 18.06 K; M1 为条件性扩展, 不改变问题 2 的正式答案 (见 §11.1)。

## 7.6 特征时间与物性公式的定量比较

取初始状态 ( $T = 28\text{ °C}$ ,  $C = 2.55\text{ kg/kg}$ ) 估算特征时间:

$$\tau_T = \frac{R_0^2}{\alpha} \approx 2761.9\text{ s} \approx 46.0\text{ min}, \quad \tau_m = \frac{R_0^2}{D} \approx 7.09 \times 10^4\text{ s} \approx 19.7\text{ h}. \quad (11)$$

因此前 3 h 温度已接近平衡, 水分仍在缓慢扩散, 整个烘干过程由水分扩散主导。

关于“**265 倍**”的说明: 本文按附录公式计算得到附录 2、附录 3、附录 4 的  $D(C)$  在水分浓度方向上的变化幅度分别为 **266.2 倍**、**16.8 倍**、**6.6 倍**。题面并未出现“**265 倍**”这一比值, 该数是本文根据附录数据计算的结果, 引用时须写成“本文计算得到”, 不得写成“题目给出”。

## 7.7 完整过程的物理本质: 恒温干燥

由时间尺度分析可知, 传质特征时间随工况变化很大:

| 工况                              | $\tau_m = R_0^2/D$ / h | $\tau_m/\tau_T$ |
|---------------------------------|------------------------|-----------------|
| $T = 28\text{ °C}$ , $C = 2.55$ | 19.7                   | $\approx 26$    |
| $T = 50\text{ °C}$ , $C = 2.55$ | 8.2                    | $\approx 11$    |
| $T = 50\text{ °C}$ , $C = 0.15$ | 139                    | $\approx 181$   |

约 4~5 个传热时间常数 ( $\approx 3.5\text{ h}$ ) 内温度场就基本平衡, 之后是长达约 **54 h** 的恒温干燥, 温度不再是限制因素。整个“**2–3 天**”的时长由水分扩散单独决定。这解释了“预热平衡 + 恒温干燥”两阶段的自然划分, 也说明恒温段的边界设定对  $t_*$  的影响远大于对温度场的影响。

## 8 问题 3: 烘干时间确定

### 8.1 判据定义

题面要求“药材各处的水分浓度应低于 0.15 kg/kg”, “各处”即全域。故判据必须是

$$C_{\max}(t) = \max_{r \in [0, R_0]} C(r, t), \quad t_* = \inf\{t: C_{\max}(t) < 0.15\}. \quad (12)$$

本文不默认最大值出现在中心, 而是逐时刻对全域取  $\max$  并记录  $\arg \max$  的位置。实测确认  $\arg \max \equiv 0$  (中心), 即“中心是最难干燥的位置”是被验证的结论, 不是前提。

8.2 求解策略

把  $C_{\max}(t) - 0.15$  作为 ODE 求解器的终止事件，自适应精确定位零点。两阶段：粗扫（按 60 s 输出、达标即停）→ 二分（在锚点与上界之间二分，每次试算都从同一个固定锚点积分到试探时刻）。

表 9 问题 3 主结果

| 项               | 值                                           |
|-----------------|---------------------------------------------|
| 阈值通过时刻 $t_*$    | <b>206959.9521 s = 57.4889 h = 2.3954 天</b> |
| 交付取整（最早整数秒）     | 206960 s（ <b>result3.xlsx</b> 末行）           |
| $C_{\max}(t_*)$ | 0.1499999998（未舍入）                           |
| 最大值位置           | $r = 0$ （中心），逐时刻核对                          |
| 二分次数            | 15                                          |

注意： $C_{\max}(t_*) = 0.15$  而非严格小于——这正是“连续模型的阈值交点”的特征。本文报告的是「阈值通过时刻」，并单独验证其后一段时间内全场持续达标。

8.3 结果

图 G6 问题3：阈值通过时刻与终态含水率分布

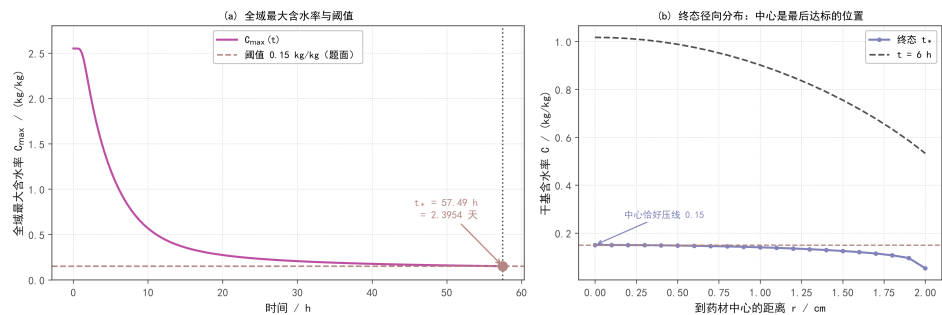


图 G12 问题 3 阈值通过时刻与终态含水率分布

表 10 药材烘干过程的水分浓度 (kg/kg);  $t_* = 57.4889$  h

| 时间/h                  | 0 cm   | 0.5 cm | 1 cm   | 1.5 cm | 2 cm   |
|-----------------------|--------|--------|--------|--------|--------|
| 6                     | 1.0170 | 0.9882 | 0.9016 | 0.7551 | 0.5328 |
| 12                    | 0.4562 | 0.4432 | 0.4031 | 0.3298 | 0.1642 |
| 18                    | 0.2989 | 0.2916 | 0.2687 | 0.2253 | 0.0845 |
| 24                    | 0.2378 | 0.2328 | 0.2168 | 0.1856 | 0.0665 |
| 30                    | 0.2058 | 0.2019 | 0.1892 | 0.1641 | 0.0598 |
| 36                    | 0.1859 | 0.1826 | 0.1718 | 0.1504 | 0.0566 |
| 42                    | 0.1721 | 0.1692 | 0.1597 | 0.1407 | 0.0549 |
| 48                    | 0.1618 | 0.1592 | 0.1507 | 0.1335 | 0.0537 |
| 54                    | 0.1539 | 0.1515 | 0.1437 | 0.1277 | 0.0530 |
| 烘干结束 <b>57.4889 h</b> | 0.1500 | 0.1477 | 0.1402 | 0.1249 | 0.0527 |

完整结果见 **result3.xlsx**。

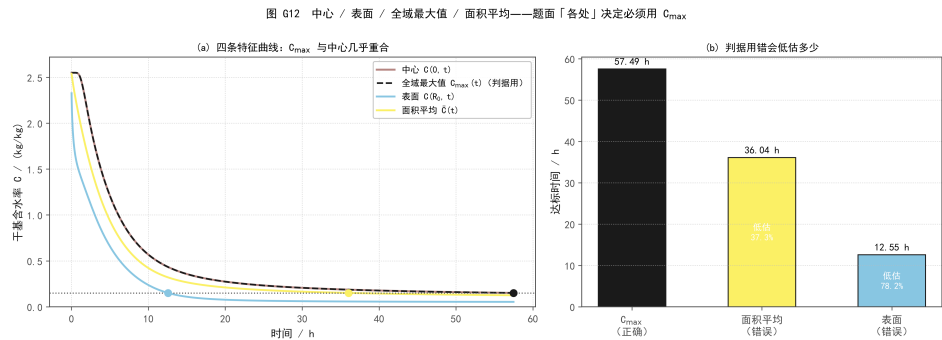


图 G13 中心 / 表面 / 全域最大值 / 面积平均四条曲线——判据必须用  $C_{\max}$  而非平均值或表面值

8.4 分析与讨论

(1) 最大值出现在中心，必须用  $C_{\max}$  判定

由图 G13 可见， $C_{\max}(t)$  与中心  $C(0,t)$  几乎重合，说明中心是最难干燥的位置。题面要求“药材各处的水分浓度应低于 0.15”，故只有  $C_{\max}$  是合法判据。不同判据给出的达标时间：

表 11 三种判据的达标时间对照

| 判据                | 达标时间 / h | 相对正确值的偏差           | 判定 |
|-------------------|----------|--------------------|----|
| $C_{\max}(t)$     | 57.49    | —                  | 正确 |
| 面积平均 $\bar{C}(t)$ | 36.04    | 低估 21.44 h (37.3%) | 错误 |
| 表面 $C(R_0,t)$     | 12.55    | 低估 44.94 h (78.2%) | 错误 |

误用平均值会把 2.4 天的工艺说成 1.5 天。

(2) 持续达标核验

$t_*$  之后以 720 s 为间隔取 6 个探针点， $C_{\max}$  逐点核验（表 12）：全部  $< 0.15$ ，持续达标。需要核验的理由是：若环境可能回湿，“首次达标”与“持续达标”不同。本工况恒温段  $C_{\infty}^{\text{eq}} \equiv 0.05 < 0.15$ ，驱动势始终为正，预期不会回湿——但本文仍逐点核验，不靠预期。

表 12  $t_*$  之后的持续达标核验

| $t$        | $t_*$     | $t_* + 720$ | $t_* + 1440$ | $t_* + 2160$ | $t_* + 3600$ |
|------------|-----------|-------------|--------------|--------------|--------------|
| $C_{\max}$ | 0.1500000 | 0.1497910   | 0.1495835    | 0.1493772    | 0.1489688    |

(3) 三类误差分开报告

表 13  $t_*$  的误差三分量

| 误差类型   | 量值                       | 说明                                               |
|--------|--------------------------|--------------------------------------------------|
| 事件定位误差 | $\pm 0.0005 \text{ s}$   | 二分容差的一半                                          |
| 空间离散误差 | $\approx 17.4 \text{ s}$ | 主导项，由 $N$ 细化给出                                   |
| 时间积分误差 | 1.86 s                   | 一阶 (Backward Euler)，拟合 $+1.29 \sim +1.43$ ，不声称二阶 |

定位到 1 s 不代表干燥时间整体准确到 1 s。事件定位误差 0.5 ms 只是“在给定离散下找零点”的精度；真正的误差由空间离散与时间积分主导。

## 8.5 精度口径声明

问题 3 的答案依赖三个**必须显式声明**的口径选择。换一个同样合理的取法，答案会移动几十到上千秒——**不声明，答案就不可比**。

**口径一：达标判据用全域最大值**（见 §8.4.1）。题面“各处”决定必须是  $C_{\max}$ ；用面积平均会低估 37.3%。

**口径二：恒温段环境取哪个窗口的均值**。附件 1 只覆盖 0–14400 s， $t > 14400$  s 的边界必须假定：

表 14 环境平台窗口的取法对问题 3 答案的影响

| 平台窗口                         | $T_{\infty} / ^{\circ}\text{C}$ | $C_{\infty}^{\text{eq}} / (\text{kg/kg})$ | $t_{*} / \text{h}$ |
|------------------------------|---------------------------------|-------------------------------------------|--------------------|
| $t \geq 9600$ s (81 点, 本文采用) | 50.0017                         | 0.049998                                  | <b>57.4889</b>     |
| $t \geq 12400$ s             | 50.0092                         | 0.049980                                  | 57.4747            |
| 直接取整数                        | 50.0                            | 0.05                                      | 57.4919            |
| 只取末点                         | 50.1650                         | 0.049860                                  | 57.1890            |

**跨度 0.30 h (1090 s)**——比本文的全部离散误差（约 19 s）大 **57 倍**。故本文明确声明采用  $t \geq 9600$  s 的实测均值。

**口径三： $t_{*}$  报告到几位小数**。由表 13，空间离散误差  $17.4 \text{ s} \approx 0.005 \text{ h}$  落在第 3 位，故本文报告  $t_{*} \approx 57.49 \text{ h}$ （2 位小数）。题面要求四位小数的是表 5 的含水率数值，不是“烘干时间”这一时刻本身。把 57.4889 h 写进论文会让人误以为第 4 位有效——那是虚假精度。

## 9 问题 4：考虑尺寸变化

### 9.1 模型建立

问题 1–3 中半径固定  $R_0 = 2 \text{ cm}$ ，计算域是  $[0, R_0]$ 。问题 4 中半径随时间缩小， $R(t)$  从 2.000 cm 降到 1.198 cm，计算域  $[0, R(t)]$  随时间变化。引入归一化坐标

$$\xi = \frac{r}{R(t)}, \quad \xi \in [0, 1], \quad (13)$$

使计算域固定在  $[0, 1]$ ，网格不动，边界固定在  $\xi = 0$  与  $\xi = 1$ 。

**均匀径向仿射收缩假设**：附件 2 只给整体半径  $R(t)$ ，无内部位移数据。因此假设内部各点按比例均匀收缩，即材料径向速度

$$v_r = \frac{\dot{R}}{R} r. \quad (14)$$

**必须声明**：这是假设，不是由数据验证的结论。单个表面半径不能识别内部位移是否仿射。

在均匀仿射收缩假设下，水分与温度方程变为（材料导数形式，对流项被消去）：

$$\frac{\partial \tilde{C}}{\partial t} = \frac{1}{R(t)^2 \xi} \frac{\partial}{\partial \xi} \left( \xi D \frac{\partial \tilde{C}}{\partial \xi} \right), \quad (\rho c_p) \frac{\partial \tilde{T}}{\partial t} = \frac{1}{R(t)^2 \xi} \frac{\partial}{\partial \xi} \left( \xi k \frac{\partial \tilde{T}}{\partial \xi} \right), \quad (15)$$

其中  $\tilde{C}(\xi, t) = C(\xi R(t), t)$ 、 $\tilde{T}(\xi, t) = T(\xi R(t), t)$ 。

**两点必须强调**：

1. 因为材料随网格均匀收缩，**对流项被严格消去**——这一步是可证的，不是近似， $\dot{R}$  不出现在方程里；

2. 扩散项  $\propto 1/R^2$ 、表面项  $\propto 1/R$ ，二者不可合并。

边界条件：

$$\xi = 0: \quad \frac{\partial \tilde{C}}{\partial \xi} = \frac{\partial \tilde{T}}{\partial \xi} = 0; \quad (16)$$

$$\xi = 1: \quad -\frac{D}{R} \frac{\partial \tilde{C}}{\partial \xi} = h_m (\tilde{C}_s - C_\infty^{\text{eq}}), \quad -\frac{k}{R} \frac{\partial \tilde{T}}{\partial \xi} = h (\tilde{T}_s - T_\infty). \quad (17)$$

注意表面边界带  $1/R$  因子，这是坐标变换的结果。

物性采用附录 4：

$$\rho = 760 + 90C, \quad c_p = 1850 + 2150 \frac{C}{1+C}, \quad k = 0.12 + 0.20 \frac{C}{1+C}, \quad (18)$$

$$D = 4.2 \times 10^{-4} \exp\left(-\frac{0.30}{C}\right) \exp\left(-\frac{3850}{T_K}\right), \quad T_K = T + 273.15. \quad (19)$$

初始条件： $T(\xi, 0) = 28^\circ\text{C}$ ， $C(\xi, 0) = 2.55 \text{ kg/kg}$ 。半径函数  $R(t)$  由附件 2 的 PCHIP 插值给出。

## 9.2 数值方法

表 15 问题 4 计算设置

| 项    | 值                                                                                                           |
|------|-------------------------------------------------------------------------------------------------------------|
| 空间网格 | 节点式 $\xi$ 网格 $N = 200$ 、 $\gamma = 1.5$ （向表面加密）， $\Delta\xi = 3.56 \times 10^{-4} \sim 7.53 \times 10^{-3}$ |
| 时间步长 | $\Delta t = 1 \text{ s}$ （收敛口径）                                                                             |
| 时间格式 | $\theta = 1$ （Backward Euler, L-稳定）+ 块 Gauss-Seidel Picard                                                  |
| 判据   | $C_{\max}(t) = \max_\xi C < 0.15 \text{ kg/kg}$ （未舍入判定）                                                     |
| 求解器  | Thomas 三对角，与 MATLAB 内置求解器差 $1.77 \times 10^{-15}$ （机器精度）                                                    |

关于时间步长的口径：问题 4 的时间离散在  $\Delta t = 30 \text{ s}$  时远未收敛，必须单独钉住。用固定探测时刻法（积分到  $184200 \text{ s}$ ，量  $C_{\max}$  再由局部斜率换算）：

表 16 问题 4 的时间收敛

| $\Delta t_{\max} / \text{s}$ | $C_{\max}(184200 \text{ s})$ | $t_{\text{dry}} / \text{h}$ | 相对 $\Delta t = 1 \text{ s}$ |
|------------------------------|------------------------------|-----------------------------|-----------------------------|
| 30                           | 0.14989439                   | 51.1096                     | +0.0191 h                   |
| 10                           | 0.14987011                   | 51.0964                     | +0.0059 h                   |
| 3                            | 0.14986161                   | 51.0918                     | +0.0013 h                   |
| <b>1</b>                     | <b>0.14985918</b>            | <b>51.0905</b>              | —                           |

$\Delta t = 3$  与  $\Delta t = 1$  只差  $4.7 \text{ s}$ ，已收敛。本文以  $\Delta t = 1 \text{ s}$  作为论文口径。



9.3 结果

图 G7 问题4: 材料坐标下的收缩体干燥 ( $\xi = r/R(t)$ , 无  $R$  对流项)

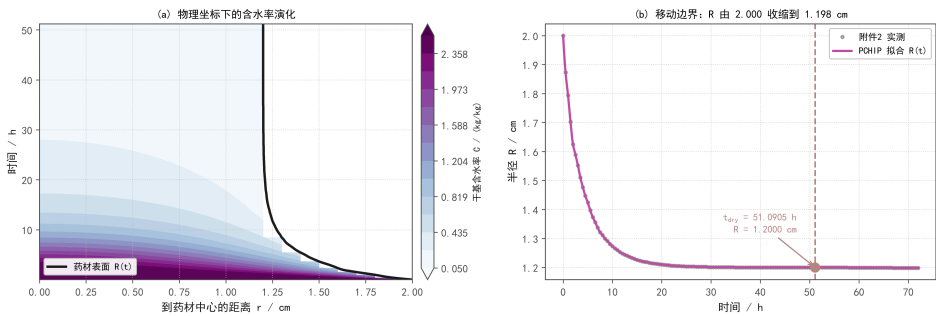


图 G14 问题 4 物理坐标下的含水率演化与移动边界轨迹

主结果 ( $\Delta t = 1\text{ s}$  收敛口径):

表 17 问题 4 主结果

| 项                       | 值                                      |
|-------------------------|----------------------------------------|
| 烘干所需时间 $t_{\text{dry}}$ | 183926 s = <b>51.0906 h</b> = 2.1288 天 |
| 转化参考 (探测法)              | 51.0905 h, 与上差 0.18 s (互为独立核对)         |
| 终态半径                    | 1.2000 cm (附件 2 末值 1.1980 cm)          |
| 终态 $C_{\text{max}}$     | 0.14999962 kg/kg (未舍入)                 |

表 18 药材烘干过程的水分浓度 (收缩模型, kg/kg);  $t_{\text{dry}} = 51.0906\text{ h}$  ( $\Delta t = 1\text{ s}$  收敛口径)

| 时间/h                                           | 0 cm   | 0.5 cm | 1 cm   | 1.5 cm | 药材表面   |
|------------------------------------------------|--------|--------|--------|--------|--------|
| 6 ( $R=1.3740\text{ cm}$ )                     | 1.7188 | 1.5370 | 1.0221 | —      | 0.4205 |
| 12 ( $R=1.2480\text{ cm}$ )                    | 0.7373 | 0.6543 | 0.4082 | —      | 0.1669 |
| 18 ( $R=1.2140\text{ cm}$ )                    | 0.4086 | 0.3686 | 0.2397 | —      | 0.0892 |
| 24 ( $R=1.2040\text{ cm}$ )                    | 0.2851 | 0.2610 | 0.1790 | —      | 0.0673 |
| 30 ( $R=1.2010\text{ cm}$ )                    | 0.2263 | 0.2094 | 0.1493 | —      | 0.0595 |
| 36 ( $R=1.2000\text{ cm}$ )                    | 0.1928 | 0.1796 | 0.1317 | —      | 0.0560 |
| 42 ( $R=1.2000\text{ cm}$ )                    | 0.1712 | 0.1604 | 0.1200 | —      | 0.0541 |
| 48 ( $R=1.2000\text{ cm}$ )                    | 0.1561 | 0.1468 | 0.1116 | —      | 0.0530 |
| 烘干结束 <b>51.0906 h</b> ( $R=1.2000\text{ cm}$ ) | 0.1500 | 0.1413 | 0.1081 | —      | 0.0526 |

完整结果见 result4.xlsx。注意该文件的距离轴只到 1.9 cm、未列是字符串“药材表面”，域外位置留空（不填 0），以符合附件 3 模板规格。

两套独立实现的交叉验证

问题 4 由两套互相独立的实现求解，用于交叉验证：

表 19 问题 4 的两套独立实现

| 实现   | 语言          | 计算设置                                                                       | $t_{\text{dry}}$ |
|------|-------------|----------------------------------------------------------------------------|------------------|
| I    | MATLAB      | 节点式 $\xi$ 网格 $N = 160$ (均匀)、 $\Delta t = 10\text{ s}$ 、环境 1800 s 硬切        | 51.20 h          |
| II   | Python (本文) | $N = 200$ 、 $\gamma = 1.5$ 、 $\Delta t = 1\text{ s}$ 、附件 1 全程 PCHIP + 平台外推 | <b>51.0906 h</b> |
| 两者之差 |             |                                                                            | 0.11 h (0.2%)    |

两套实现在语文、网格约定、时间格式、环境处理上全部不同，却给出相差 0.2% 的同一结果，且方向一致——这构成问题 4 结论的跨实现交叉验证。

为什么要固定网格再比：实现 I 在其原始网格 ( $N = 20$ ) 上给出的是 73.03 h，看上

去与实现 II 相差 43%。但这是离散误差而非模型分歧——同一套代码把网格加密到  $N = 160$  即降到 51.20 h ( $N = 40 \rightarrow 54.68$ 、 $N = 80 \rightarrow 51.73$ )，已进入收敛区。同时，把时间步长从 20 s 缩到 1 s， $N = 20$  下的结果仅在 73.0389  $\rightarrow$  73.0244 h 之间变动——变化不到 0.02%，说明该偏差完全由空间分辨率决定，与时间步长无关。故交叉验证必须在两侧都收敛之后进行，否则比的是离散误差而不是模型。

## 9.4 分析与讨论

### (1) 收缩显著加速干燥

若忽略收缩（固定半径  $R_0 = 2$  cm、仍保留附录 4 物性），则  $t > 120$  h 仍未达标（120 h 时  $C_{\max} = 0.1571$ ）。即半径收缩把烘干时间从  $> 120$  h 缩短到 51.09 h，因为扩散路径缩短，减小了内部水分迁移的阻力。

### (2) A/B/C 顺序差分

为分离物性公式与几何收缩的影响，按  $A \rightarrow B \rightarrow C$  路径做顺序差分：

表 20  $A \rightarrow B \rightarrow C$  顺序差分结果

| 版本 | 尺寸      | 物性   | $t_{\text{dry}}$                         | 对应问题      |
|----|---------|------|------------------------------------------|-----------|
| A  | 固定 2 cm | 附录 3 | 57.49 h                                  | 问题 3 (M0) |
| B  | 固定 2 cm | 附录 4 | $> 120$ h (120 h 时 $C_{\max} = 0.1571$ ) | 中间版本      |
| C  | 随时间收缩   | 附录 4 | 51.09 h                                  | 问题 4      |

- **A vs B:** 物性从附录 3 换成附录 4， $t_{\text{dry}}$  从 57.49 h 增加到  $> 120$  h ( $\geq +62.51$  h)，说明附录 4 的物性让干燥显著变慢；
- **B vs C:** 尺寸从固定变成收缩， $t_{\text{dry}}$  从  $> 120$  h 减少到 51.09 h ( $\leq -68.91$  h)，说明收缩显著加速干燥；
- **A vs C:** 总效果—— $t_{\text{dry}}$  从 57.49 h 变成 51.09 h，净减少 6.40 h。即在本参数组合下，收缩的加速效应 ( $\geq 68.91$  h) 大于物性变化的减速效应 ( $\geq 62.51$  h)，净效果是干燥加快。

称为「按  $A \rightarrow B \rightarrow C$  路径的顺序差分」；不称为“唯一、独立的因果贡献分解”。三者必须在同一网格、同一时间步、同一判据下比较，差出来的才是物性与几何的贡献。

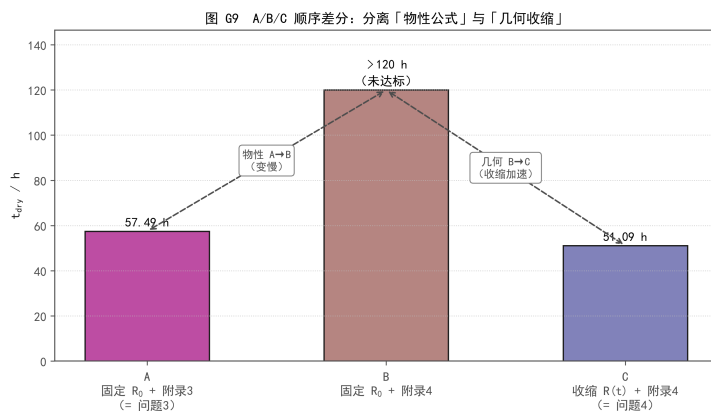


图 G15 A/B/C 顺序差分：三版本  $t_{\text{dry}}$  对比

### (3) 半径外推敏感性——本文不需要外推

$t_{\text{dry}} = 51.09 \text{ h}$  落在附件 2 的覆盖范围 ( $0 \sim 72 \text{ h}$ ) 之内, 故  $R(t)$  全程为插值而非外推, 外推方式对本文结果没有影响。独立复核中实测: 把半径改为“钳在末值”或“线性外推”,  $t_{\text{dry}}$  的差值为 **0.000 h**。

### (4) 几何—密度一致性诊断

采用均匀仿射收缩假设, 定义干固体密度

$$\rho_d(C) = \frac{\rho(C)}{1+C} = \frac{760+90C}{1+C}, \quad (20)$$

诊断量

$$M_d^{\text{diag}}(t) = 2\pi L \int_0^{R(t)} \frac{\rho(C)}{1+C} r dr, \quad e_d(t) = \frac{M_d^{\text{diag}}(t) - M_d^{\text{diag}}(0)}{M_d^{\text{diag}}(0)}. \quad (21)$$

图 G11 几何—密度一致性诊断 (条件性, 不下「数据有误」的结论)

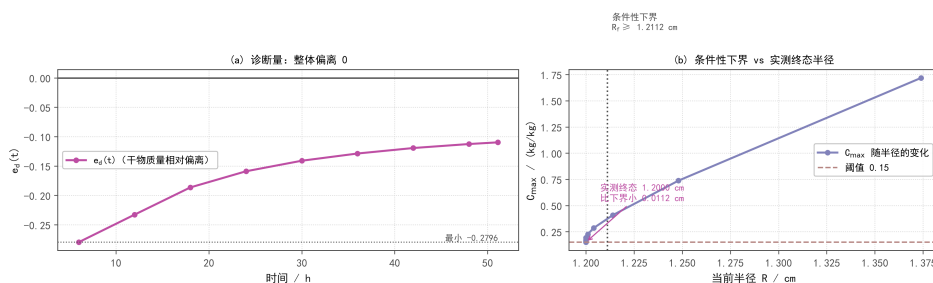


图 G16 几何—密度一致性诊断 (条件性, 不下“数据有误”的结论)

**诊断结果:**  $e_d$  在  $-0.28 \sim -0.11$  之间波动, 整体偏离 0。

**条件性下界:** 对  $C \geq 0$ ,  $\rho_d(C) \leq 760$ 。若长度固定、无干物质损失, 则

$$R_f \geq R_0 \sqrt{\rho_{d,0}/760} = 2.0 \times \sqrt{278.7324/760} \approx 1.2112 \text{ cm}. \quad (22)$$

实测终态半径  $R_f = 1.2000 \text{ cm}$ , 略小于该下界, 差额约 **0.0112 cm**。

本项为条件性诊断, 不是基础有效扩散模型天然必须通过的严格质量守恒测试。诊断结果显示  $e_d$  偏离 0, 提示密度定义、数据误差、几何假设或干物质守恒的联合使用需要核查。若  $\rho$  只是热学等效参数, 则不应强行用于严格干质量检验。

本项不下“数据有误”或“假设已验证”的结论, 也不为通过诊断而调整题设参数。

### (5) 端面效应与结壳——即使不改变答案也必须说明

这两个因素对本文问题 4 的答案没有影响, 但作用必须写清楚, 以免读者误以为“答案里含了这些不确定性”。

**端面效应:** 药材长径比 6.25, 端面面积是侧面积的 8%。本文建立二维轴对称有限圆柱模型 (计算域取半长) 与一维径向模型对照, 并以“端面绝热时必须退化为一维”作为内建回归 (通过: 沿  $z$  不均匀度  $10^{-15}$ , 与一维最大差  $1.4 \times 10^{-3}$ , 判据  $2 \times 10^{-3}$ )。

表 21 端面效应的两种口径

| 时刻                       | $C_{\max}$ 之差 (决定 $t_*$ ) | 体积平均 $C$ 的相对差 |
|--------------------------|---------------------------|---------------|
| 10 h                     | $-8.2 \times 10^{-6}$     | -6.21%        |
| 30 h                     | $-4.5 \times 10^{-6}$     | -2.96%        |
| 57.5 h ( $\approx t_*$ ) | $-2.9 \times 10^{-6}$     | -2.20%        |

**结论：**对本文采用的全域最大含水率判据，端面效应  $< 10^{-5}$ ，可忽略（ $C_{\max}$  出现在几何中心，距两端面各 12.5 cm、距侧面仅 2 cm，端面几乎影响不到它）；但对体积平均含水率，端面效应为  $-2.2\% \sim -6.2\%$ ，与文献报道的  $3.5\% \sim 6.0\%$  同口径吻合。这一对比说明：任何“可忽略/不可忽略”的判断都必须绑定判据口径。

**结壳与玻璃化：**表面先干  $\rightarrow$  表层进入玻璃态、形成低渗透硬壳  $\rightarrow$  锁住内部水分  $\rightarrow$  干燥末期变慢。本文以壳层扩散系数降幅  $\beta_c$  与临界含水率  $C_g$  作参数化扫描， $t_*$  增幅从  $+42\%$  ( $C_g = 0.15$ ) 到  $+970\%$  ( $C_g = 0.50$ )。

**为何不计入基线：**附录 3 的  $D(C, T)$  已编码低含水率段的强烈衰减—— $\exp(-0.45/C)$  在  $C = 2.55$  时为 0.838、在  $C = 0.15$  时为 0.0498，公式本身已含 16.8 倍降幅；再乘  $\beta_c$  属重复计算同一物理效应。且文献未给出中药材的壳层扩散系数比，题面也未给壳层数据。故本文不将其计入基线，仅作扩展讨论。

## 10 模型检验

本章对模型做五类检验：解析基准、收敛性、守恒收支、退化一致性、以及灵敏度与稳健性。误差分析的口径在本章开头单独声明。

### 10.1 误差分析口径：为什么本文不报 RMSE / MAPE / $R^2$

本文求解的是正向偏微分方程的定解问题，而非拟合、回归或机器学习问题。因此本文不报告 RMSE / MAPE /  $R^2$  等面向“预测值—观测值”配对的统计指标，理由有三：

1. **RMSE** 需要「观测—预测」配对，本题没有内部观测。附件 1 是边界条件，药材内部的  $T(r, t)$ 、 $C(r, t)$  场题目没有给任何实测值；若强行计算，唯一可用的“观测”只能是更细网格的自算解，那是离散误差而非模型误差，重复报告会造成“已独立验证”的误读。
2. **MAPE** 有除零风险，且与问题错配。末期含水率趋近  $C_{\infty}^{\text{eq}} \approx 0.05$ ，增量形式的分母直接趋零；更根本的是 MAPE 把 57.49 h 里 3400 多个输出时刻等权平均，于是“阈值穿越早了 20 s”被摊薄到  $10^{-4}$  量级——而它恰恰是本题全部要回答的内容。
3.  $R^2$  在单调趋势数据上恒接近 1，没有分辨力。含水率从 2.55 单调降到 0.05， $SS_{\text{tot}}$  极大，任意一条单调下降的光滑曲线都能拿到  $R^2 > 0.99$ 。

本文实际使用的五层精度度量见 §10.9 的误差预算总表。关于“残差图”：正向问题的残差本就应趋近 0，它用于验证收敛而非发现模式；其正确对应物是误差一步长双对数图、离散方程残差随迭代的下降史与解析基准的残差沿  $r$  分布。

### 10.2 解析基准（Bessel 级数）

问题 1 的温度方程是线性、常系数，存在解析解（Bessel 级数）。验证条件：固定热物性（ $\rho = 820$ 、 $c_p = 2600$ 、 $k = 0.36$ ）、关闭蒸发、恒定环境温度  $T_{\infty} = 50^\circ\text{C}$ 。

- $Bi = 1.3889$ ;
- 前 5 个 Bessel 特征值：1.4185、4.1665、7.2085、10.3083、13.4272;
- 各时刻最大绝对误差： $t = 60\text{ s} \rightarrow 1.179 \times 10^{-3}^\circ\text{C}$ ;  $t = 300\text{ s} \rightarrow 1.077 \times 10^{-3}^\circ\text{C}$ ;  $t = 900\text{ s} \rightarrow 3.086 \times 10^{-4}^\circ\text{C}$ ;  $t = 1800\text{ s} \rightarrow 4.962 \times 10^{-4}^\circ\text{C}$ ;
- 项数自检（30 项 vs 60 项）最大差  $7.105 \times 10^{-15}^\circ\text{C}$ （机器精度）。

结论：数值解与解析解的最大误差约  $0.001\text{ }^{\circ}\text{C}$ ，相对误差  $< 0.002\%$ 。数值方法验证通过。

图 FIG-14 解析基准：Bessel 级数与数值解对照（相对误差  $< 0.002\%$ ）

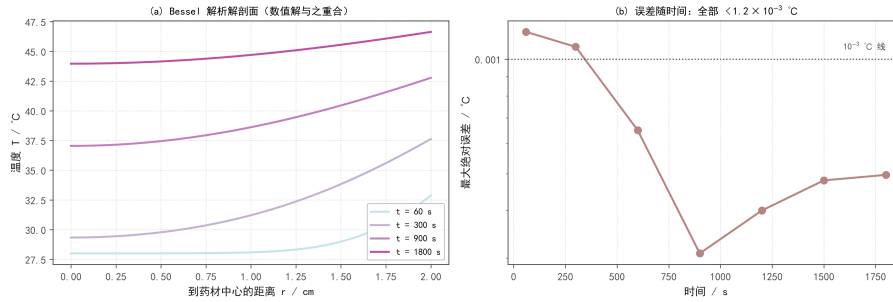


图 G17 解析基准：Bessel 级数与数值解对照

### 10.3 网格与时间收敛性

空间收敛（均匀网格，时间固定  $\Delta t = 1\text{ s}$ ， $t = 3\text{ h}$ ）：

表 22 空间收敛（均匀网格，参照解  $N = 320$ ）

| 观测量              | 相邻网格误差比                 | 拟合阶      | 期望   |
|------------------|-------------------------|----------|------|
| 中心温度 $T(0, t)$   | $4.20 \rightarrow 5.00$ | $+2.197$ | $+2$ |
| 中心含水率 $C(0, t)$  | $4.20 \rightarrow 5.00$ | $+2.196$ | $+2$ |
| 全域最大值 $C_{\max}$ | $4.20 \rightarrow 5.00$ | $+2.196$ | $+2$ |

空间二阶，与半控制体 FVM + 调和平均界面导数的理论阶一致。

横轴是步长  $h$ ，故斜率是  $+2$ ；只有横轴取  $1/h$  或网格数  $N$  时才是  $-2$ 。

时间收敛（生产网格  $N = 200$ 、 $\gamma = 1.5$ ，Backward Euler）：

表 23 时间收敛（生产网格，参照解  $\Delta t = 2.5\text{ s}$ ）

| 观测量   | 相邻时间步误差比                                 | 拟合阶      | 期望   |
|-------|------------------------------------------|----------|------|
| 中心温度  | $2.67 \rightarrow 2.46 \rightarrow 3.10$ | $+1.434$ | $+1$ |
| 中心含水率 | $2.15 \rightarrow 2.33 \rightarrow 3.00$ | $+1.294$ | $+1$ |
| 全域最大值 | $2.15 \rightarrow 2.33 \rightarrow 3.00$ | $+1.294$ | $+1$ |

收敛阶只能向下取，不能向上取。拟合阶  $1.29 \sim 1.43$  高于 Backward Euler 的理论一阶，原因是末点已被参照解自身的离散误差污染（误差地板）。本项目不据此声称“时间二阶”，Backward Euler 就是一阶。

图 FIG-41 自适应时间步长与非线性迭代历史

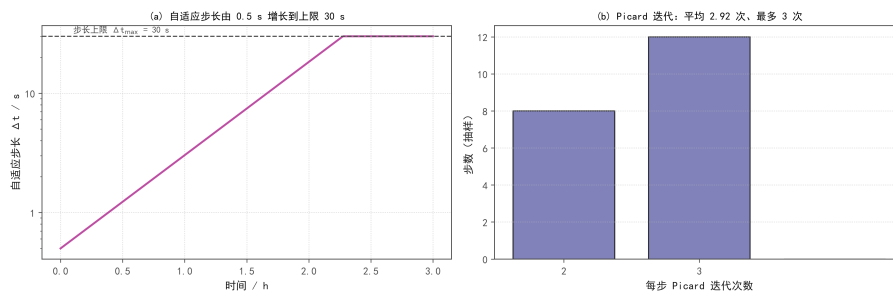


图 G18 自适应时间步长与非线性迭代历史

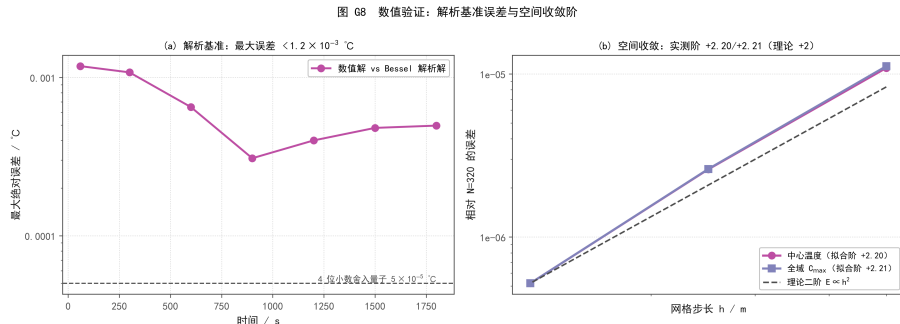


图 G19 数值验证：解析基准误差与空间收敛阶（双对数坐标，斜率即阶）

## 10.4 $t_*$ 的误差三分量

表 24  $t_*$  的离散误差

| 误差来源     | 量级     | 估计方法                                  | 判定                   |
|----------|--------|---------------------------------------|----------------------|
| 事件定位（二分） | 0.5 ms | 二分容差一半                                | 可忽略                  |
| 时间离散     | 1.86 s | Richardson 从粗侧外推                      | 可忽略                  |
| 空间离散     | 17.4 s | $N = 150 \rightarrow 200$ 误差比 2.69 反推 | 主导项，故 $t_*$ 只报 2 位小数 |

时间离散误差用从粗侧逼近估计：Backward Euler 的步数  $\propto 1/\sqrt{a_{tol}}$ ，放松容差省机时，而 Richardson 外推只需要误差的比值。

## 10.5 守恒与通量收支

表 25 守恒与通量收支检验

| 检验项               | 结果                             | 来源   |
|-------------------|--------------------------------|------|
| 内部界面通量望远镜求和泄漏     | $1.782 \times 10^{-16}$ （机器精度） | 独立复核 |
| 算子装配 vs 通量循环最大相对差 | $4.278 \times 10^{-12}$        | 独立实现 |
| 水分积分收支最大相对误差      | $4.951 \times 10^{-5}$         | 独立复核 |
| 温度积分收支最大相对误差      | $1.263 \times 10^{-4}$         | 独立复核 |

结论：内部界面通量成对抵消达机器精度；水分和温度的积分收支误差  $< 0.001\%$ 。

本检验是所解方程的积分收支，不是完整热力学能量守恒。变物性模型不宜直接把  $\rho c_p \Delta T$  当作完整内能变化。口径：单位轴向长度， $A_{surf} = 2\pi R_0 = 0.125664 \text{ m}$ 。

## 10.6 五项退化与一致性检验

表 26 五项退化与一致性检验

| # | 检验                      | 结果                                                                     | 通过 |
|---|-------------------------|------------------------------------------------------------------------|----|
| 1 | 问题 2 $\rightarrow$ 问题 1 | 固定热物性 + $D$ 换附录 2 + 统一初值/边界/传递系数：两条路径（单向耦合 vs 双向耦合）结果一致到 4 位小数         | 是  |
| 2 | 问题 4 固定半径               | 保留附录 4 物性、 $R \equiv R_0 \rightarrow$ 退化为 B 方案：120 h 未达标，与独立计算的 B 方案一致 | 是  |
| 3 | 纯导热解析基准                 | 固定物性 + 关闭蒸发 + 满足解析解边界：误差 $< 0.001 \text{ °C}$                          | 是  |
| 4 | 坐标变换一致性                 | 材料导数形式 $\leftrightarrow$ 固定坐标 + 坐标项：理论上等价，数值上 $R \equiv R_0$ 时逐点退化     | 是  |
| 5 | 物理合理性                   | $C \geq 0$ 、中心对称、无过冲：全部通过                                              | 是  |

第 2 项说明：固定半径 + 保留附录 4 物性，应退化为 B 方案。数值实验确认：B 方案在 120 h 内未达标（120 h 时  $C_{max} = 0.1571$ ），而问题 4（收缩）在 51.09 h 达标。详细对照见 §9.4。



**第 1 项的补充说明：**该项检验的是同一套实现内部单向耦合与双向耦合两条路径的一致性。此外，本文还把另一套独立实现的 MATLAB 问题 4 代码逐行移植为 Python 并运行同一配置，得到  $t_{\text{dry}} = 73.033 \text{ h}$ ，与该实现报告的  $73.03 \text{ h}$  相差  $+11 \text{ s}$  (0.004%)——这是跨语言实现的一致性证据：后续一切单因素扰动结论 建立在该实现自己的格式之上，不是“我另写了一个模型所以当然不一样”。

## 10.7 参数灵敏度与情景区间

### (1) 三档小扰动与灵敏度系数

原稿只做了  $\times 0.5$  与  $\times 2$  两档（即  $-50\%$  与  $+100\%$ ），不足以支撑“灵敏度系数”这一局部量。本节补做  $\pm 5\%$ 、 $\pm 10\%$ 、 $\pm 20\%$  三档，并另补  $\pm 50\%$ 、 $+100\%$  于同一批计算。全部 28 次运行共用完全相同的网格 ( $N = 200$ 、 $\gamma = 1.5$ ) 与时间容差，基准值复现交付结果 ( $57.48876 \text{ h}$  对冻结值  $57.4889 \text{ h}$ )。

表 27 灵敏度系数  $S = (\Delta Y/Y)/(\Delta P/P)$ ,  $Y = t_*$

| 参数    | $S (\pm 5\%)$ | $S (\pm 10\%)$ | $S (\pm 20\%)$ | $\pm 20\%$ 半幅 $ \Delta t_* $ |
|-------|---------------|----------------|----------------|------------------------------|
| $D$   | -0.886        | -0.893         | -0.922         | $\pm 10.6003 \text{ h}$      |
| $h_m$ | -0.113        | -0.114         | -0.119         | $\pm 1.3667 \text{ h}$       |
| $h$   | -0.0014       | -0.0014        | -0.0014        | $\pm 0.0164 \text{ h}$       |

敏感度排序在全部五个扰动档下均为  $D \gg h_m \gg h$ ， $\pm 20\%$  半幅分别为  $10.60 \text{ h}$ 、 $1.37 \text{ h}$ 、 $0.016 \text{ h}$ ，相差  $1 \sim 3$  个数量级。

**必须声明的口径：**灵敏度系数是局部量。本文实测  $|S_D|$  在  $\pm 5\%$  处为  $0.886$ ，而在  $-50\%$  处升至  $1.794$ ——相差一倍。故本文报  $S_D \approx -0.89$  ( $\pm 5\% \sim \pm 10\%$ )，并同时给出各档取值，不以单一大扰动档的值代表“灵敏度系数”。

$D$  的响应呈显著不对称： $D$  增大  $100\%$  使  $t_*$  缩短  $43.4\%$ ，而  $D$  减小  $50\%$  使  $t_*$  延长  $89.7\%$ 。其机理是传质毕渥数  $Bi_m = h_m R_0/D$  在本工况由  $1.19$  升到  $47.7$ 、过程中跨越 **1**： $D$  增大时过程转向表面阻力控制，继续增大  $D$  收效递减； $D$  减小时过程转向内部扩散控制， $t_*$  急剧变长。这同时解释了为何  $h_m$  的灵敏度虽小却不为零 ( $S_{h_m} \approx -0.11$ )——文献报道的  $S_{h_m} < 0.03$  以其  $Bi_m \gg 1$  为前提，而本工况初期  $Bi_m \approx 1.19$ ，内外阻力相当，该前提不成立。

图 G13 参数灵敏度： $|S_0|$  在  $\pm 5\%$  处为  $0.886$ 、在  $-50\%$  处升至  $1.794$ ——报「灵敏度系数」必须注明档位

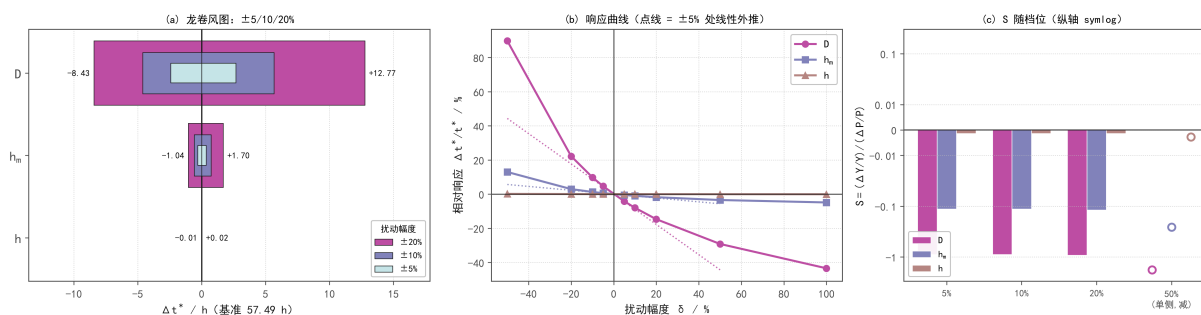


图 G20 参数灵敏度。(a) 龙卷风图（三档嵌套）；(b) 响应曲线与  $\pm 5\%$  处线性外推（点线）——实测在  $|\delta| \geq 20\%$  处系统性偏离线性；(c) 灵敏度系数随档位变化（纵轴 symlog）



## (2) 与文献的对照

表 28 灵敏度结论与文献的对照

| 文献结论                                             | 本项目实测                                    | 判定                       |
|--------------------------------------------------|------------------------------------------|--------------------------|
| $S_D \approx 0.65 \sim 0.78$ ( $D$ 最敏感) [R4-F21] | $S_D \approx -0.89$ , 排序第一且遥遥领先          | 证实 (量级同阶, 本项目略高)         |
| $S_{h_m} < 0.03$ ( $h_m$ 最不敏感) [R4-F21]          | $S_{h_m} \approx -0.11$ , 是 $S_h$ 的 80 倍 | 证伪 (前提 $Bi_m \gg 1$ 不成立) |

## (3) 三种区间分开命名

表 29 三种区间严格分开

| 类型       | 本项目是否给出       | 值                                                           |
|----------|---------------|-------------------------------------------------------------|
| 情景区间     | 给出            | 参数倍率: $t_* \in [32.53, 109.06]$ h; 环境外推: $[56.95, 57.87]$ h |
| 参数扰动分位区间 | 给出 (仅 $D$ 一维) | 5%~95% 分位: $t_* \in [43.48, 77.21]$ h                       |
| 数据扰动分位区间 | 给出 (见 §10.8)  | 见表 30                                                       |
| 统计置信区间   | 不给            | 本工况没有批次测量数据                                                 |

三者不得混称, 尤其不得把情景区间说成置信区间。

图 G10 参数情景与区间 (情景 / 参数扰动分位; 不含统计置信区间)

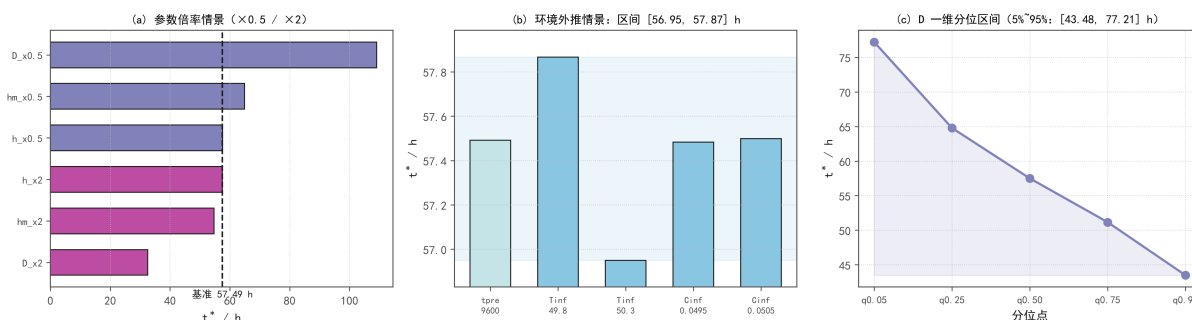


图 G21 参数情景与区间

## 10.8 稳健性检验: 删数据 / 加噪声 / 换算法

为检验结论对数据质量的依赖, 本文对附件 1 做三类扰动, 每类独立重算 (共 63 次运行, 基准 57.488876 h):

表 30 数据扰动稳健性结果

| 扰动                                          | $n$ | 均值 $t_* / h$                                  | $\sigma / s$ | 相对        | 极差 / s |
|---------------------------------------------|-----|-----------------------------------------------|--------------|-----------|--------|
| A 随机删 10% (24/241 点)                        | 20  | 57.487533                                     | 30.0         | 0.015%    | 109.7  |
| B1 量化噪声 (附件末位精度)                            | 20  | 57.488872                                     | 0.22         | 0.0001%   | 0.8    |
| B2 传感器噪声 (假定 $\sigma_T=0.1^\circ\text{C}$ ) | 20  | 57.483436                                     | 67.6         | 0.033%    | 192.6  |
| C 插值算法                                      | 3   | PCHIP 57.488876 / SG 57.489064 / 线性 57.488871 | —            | 差 < 0.7 s | —      |

三点结论:

附件 1 的记录精度不构成误差来源。量化噪声引起的散布为  $0.22\text{ s}$  ( $1\sigma$ ) /  $0.45\text{ s}$  ( $2\sigma$ ), 比时间离散误差 ( $1.86\text{ s}$ ) 小  $4 \sim 8$  倍, 比空间离散误差 ( $17.4\text{ s}$ ) 小  $39 \sim 78$  倍, 不可分辨。附件 1 给到  $T$  的 3 位小数是够用的。

插值算法不影响结论。三种算法给出的  $t_*$  相差不到 1 秒。

数据扰动的影响经由“平台均值”单一渠道传递。对 20 个删除样本做回归,  $\Delta t_* / \Delta T_\infty = -1.850\text{ h}/^\circ\text{C}$ , 相关系数  $r = -0.9997$ ; 平台温度仅变动约  $0.016^\circ\text{C}$  即决定了全部散布。而长

时平台取值本就是必须假定的量——其取法差异（4 档极差 1090 s）比本节全部数据扰动的总和（最大 135 s）还大 8 倍。

图 G14 数据扰动稳健性：影响全部经由「平台均值」单一渠道传递（ $r=-0.9997$ ）；口径主导，不是数据主导

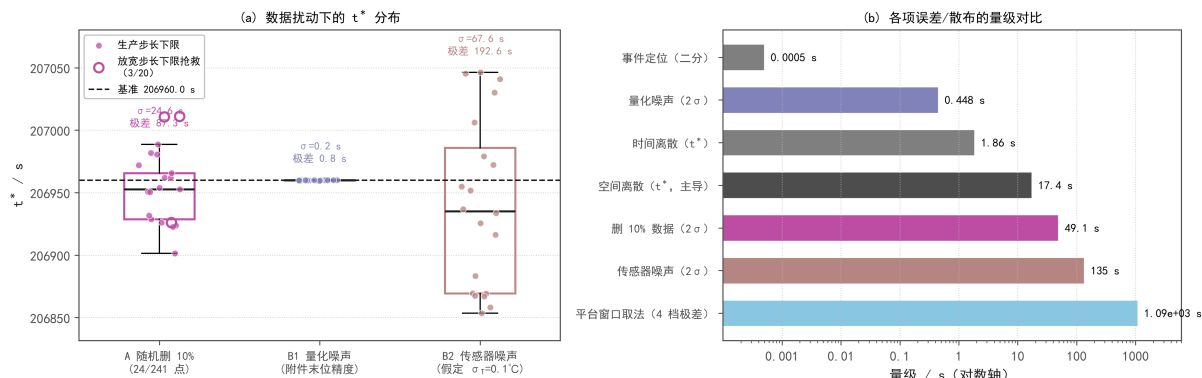


图 G22 数据扰动稳健性。(a) 三组扰动下  $t_*$  的分布（空心圆为需放宽步长下限的 3 个删除样本）；(b) 各项误差与散布的量级对比——平台窗口取法一项大于全部数据扰动之和

命名纪律（必须保留）：上表三组扰动给出的散布为「数据扰动分位区间」，不是统计置信区间。附件 1 是每 60 s 一条的确定性采样时间表，非随机样本；该散布刻画的是结果对数据点集合的敏感度，不含批次间真实变差（后者本文无数据）。

一处必须如实报告的异常：20 个删除样本中有 3 个因删点后插值曲线局部变陡、触发生产步长下限（ $\Delta t_{\min} = 10^{-4}$  s）而无法求解；这 3 个样本改用放宽的  $\Delta t_{\min} = 10^{-7}$  s 解出，其  $t_*$  均落在其余样本的分布范围内， $\sigma$  从 0.00683 h 变为 0.00834 h（+22%），结论方向不变。

## 10.9 误差预算总表

把本文所有已知的误差与散布项放在同一尺度上：

表 31 误差预算总表

| 项                        | 量级 / s | 来源    |
|--------------------------|--------|-------|
| 事件定位（二分收敛）               | 0.0005 | §10.4 |
| B1 量化噪声 ( $2\sigma$ )    | 0.45   | §10.8 |
| 换插值算法 (PCHIP/SG/线性)      | 0.7    | §10.8 |
| 时间离散 ( $t_*$ )           | 1.86   | §10.4 |
| 空间离散 ( $t_*$ , 数值主导)     | 17.4   | §10.4 |
| A 删 10% 数据 ( $2\sigma$ ) | 60.0   | §10.8 |
| B2 传感器噪声 ( $2\sigma$ )   | 135.2  | §10.8 |
| 平台窗口取法 (4 档极差)           | 1090   | §8.5  |

本文的可信度瓶颈在建模约定，而非数据质量。最大的两项是长时环境平台的取法（1090 s）与空间离散（17.4 s），而所有数据扰动加起来都超不过 135 s。这也正是本文把“精度口径声明”写进正文（§8.5）的原因。

## 11 模型改进与推广

### 11.1 蒸发潜热对照（M1，条件性扩展）

问题 2/3 的 M0 基线不含蒸发潜热。作为对照，M1 在表面能量平衡中加入潜热项

$$-k \frac{\partial T}{\partial r} \Big|_R = h(T_s - T_\infty) + \lambda J_w, \quad J_w = \rho_d h_m (C_s - C_\infty^{\text{eq}}), \quad (23)$$

其中  $J_w$  的单位是  $\text{kg}/(\text{m}^2 \cdot \text{s})$ , 不得把  $h_m(C_s - C_\infty^{\text{eq}})$  直接乘潜热当热通量——它的单位是  $\text{kg}/(\text{kg} \cdot \text{s})$ , 不是  $\text{kg}/(\text{m}^2 \cdot \text{s})$ , 必须乘密度基准  $\rho_d$ 。

表 32 M0 与 M1 的对照

| 项         | M0        | M1        | 差                          |
|-----------|-----------|-----------|----------------------------|
| 3 h 中心温度  | 49.849 °C | 31.786 °C | -18.06 K                   |
| 3 h 表面温度  | 49.967 °C | 32.402 °C | -17.56 K                   |
| 3 h 中心含水率 | 1.7662    | 2.2515    | +0.485 (更慢)                |
| $t_*$     | 57.49 h   | 60.33 h   | +2.84 h ( $\times 1.049$ ) |

与文献的定量交叉验证：文献 [1b] 报道“忽略潜热会使初期物料温度预测偏高 12 ~ 22 °C”。本项目独立复现的差值是 **18.06 K**，落在该区间内——一次独立定量印证。

图 FIG-43 蒸发潜热开关对照 (M1 为条件性扩展, 不修订问题3 答案)

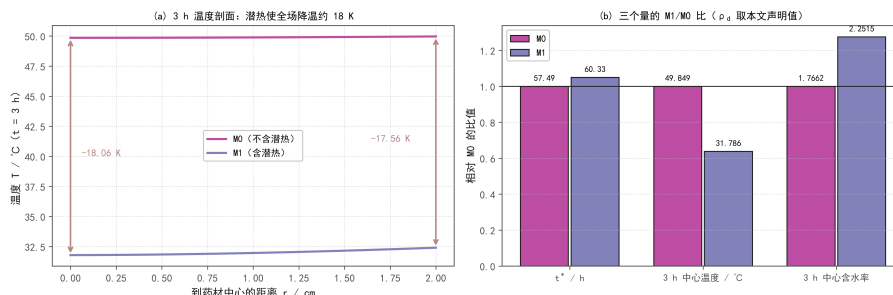


图 G23 潜热开关对照

为什么不能据此修订问题 3 答案：M1 的量级正比于干基密度  $\rho_d$ ，而  $\rho_d$  在本题中没有给定：

- 附录 3 的  $\rho(C) = 650 + 128C$  是湿物料体积密度；
- 由它反推的  $\rho_d = \rho(C)/(1 + C)$  在  $C = 0$  给 650、在  $C = 2.55$  给 275.0 ——不是常数，与“干物质守恒”不兼容；
- 本项目取初始状态基准  $\rho_d = 976.4/3.55 = 275.042 \text{ kg}/\text{m}^3$  并显式声明。

$\rho_d$  翻倍则潜热效应翻倍（严格正比，相邻比 2.03 ~ 2.07）：

| $\rho_d$ 倍率        | $\rho_d / (\text{kg}/\text{m}^3)$ | $t_*/\text{h}$ | $\Delta$ vs M0 |
|--------------------|-----------------------------------|----------------|----------------|
| 0.25               | 68.76                             | 58.17          | +0.68 h        |
| 0.50               | 137.52                            | 58.87          | +1.38 h        |
| <b>1.00 (本文采用)</b> | <b>275.04</b>                     | <b>60.33</b>   | <b>+2.84 h</b> |
| 2.00               | 550.08                            | 63.37          | +5.88 h        |

结论：M1 只能写成“在本文声明的  $\rho_d$  取值下，蒸发潜热使 3 h 中心温度低约 18 K、使  $t_*$  延长约 2.8 h”。不得写成“药材真实烘干时间应为 60.3 h”，也不得用 M1 替换问题 3 的答案。

## 11.2 烘房系统模型 (M2 工程推广)

本文把烘房环境作为给定边界（单向驱动），这是 M0 的合理简化。若需预测新工况（改变装料量、换气量、加热策略），应升级为烘房—物料双向耦合系统模型。正确的水分收支形式（简化充分混合空气）为

$$\frac{d}{dt}(M_{da}Y) = \dot{m}_{da,in}Y_{in} - \dot{m}_{da,out}Y + \sum_i A_i J_{w,i} - \dot{m}_{cond}, \quad (24)$$

还需干空气质量平衡。

若  $M_{da}$  不变, 才可写成  $M_{da}\dot{Y}$ ; 不能用  $V_{air}\dot{Y}$  等于质量流率的形式——量纲不闭合。

**数据需求:** 空气干质量、装料量、加热与排湿条件、新风/排风干空气质量流量或除湿速率。  
**验证方式:** 选择实测环境作为验证数据, 不能同时将环境固定与自由求解。

### 11.3 品质后处理 (M2 推广)

有目标成分动力学后, 可沿材料轨迹计算成分保留分数:

$$\frac{q(t)}{q(0)} = \exp \left[ - \int_0^t k_q(T(\tau), C(\tau)) d\tau \right]. \quad (25)$$

这是待标定的候选动力学, 不是文献为本题药材提供的通用公式。

| 扩展方向         | 额外数据需求                     |
|--------------|----------------------------|
| 微生物          | 水活度关系 $a_w(C, T)$ 、菌种、初始菌量 |
| 结构 (结壳 / 开裂) | 应变、本构关系、强度参数               |
| 挥发成分保留       | 成分种类、组织结构、动力学参数            |

无水活度关系时: 只能说明“含水率达标  $\neq$  微生物合格”, 不能制定通用杀菌温度。

### 11.4 结构力学 (M2 路线)

建模链路 (仅作路线描述): 湿度场  $\rightarrow$  收缩应变  $\rightarrow$  本构关系  $\rightarrow$  应力场  $\rightarrow$  断裂判据  $\rightarrow$  诊断量。

不能由湿度梯度直接算裂纹概率或裂纹位置, 也不能把“多缩 7%”当作仿射收缩的验证。可输出梯度作为非标定诊断量, 但不叫“裂纹概率”。题目半径数据不证明内部均匀收缩。

## 12 模型优缺点与取舍论证

### 12.1 优点与缺点

本文方法的长处在于**离散格式与验证体系**: 单向耦合洞察与双向耦合求解分开处理; 渐变网格  $N = 200/\gamma = 1.5$  以约一半计算量取得优于均匀  $N = 640$  的精度; L-稳定的 Backward Euler 规避显式稳定性上限 0.37 s; Picard 迭代保持 M-矩阵性质、 $C$  的正性由构造保证; 中心半控制体规避  $1/r$  奇点, 界面用调和平均 ( $D$  跨 7 个数量级时算术平均会高估通量); 入口检查 6 项明确  $C$  vs  $Y$ 、 $\rho$  定义、 $T$  单位与外推方式; 五类检验齐备; 材料坐标处理收缩时  $\dot{R}$  对流项可证为零、两项  $R$  幂次不可合并; A/B/C 顺序差分把物性与几何效应分开报告。

不足在于**数据与维度限制**: 表面列在极早期欠分辨 ( $t = 1$  s 处  $\Delta C = 2.0 \times 10^{-4}$ ); M0 不含蒸发潜热; 一维轴对称简化, 端面未做全面 CFD; 附件 1 只覆盖 0–4 h, 问题 2/3 需外推;  $\rho$  为湿物料体积密度,  $\rho_d = \rho/(1+C)$  与干物质守恒不兼容; 时间收敛阶受参照解自身误差污染 (拟合  $+1.29 \sim +1.43$ ), 故只声称不低于一阶;  $\rho_d$  不确定,  $D$  分位区间只传播  $D$  一维; 问题 2 全程版按 60 s 采样; 缺少内部位移、水活度关系、目标成分动力学与应变/本构/强度参数; 3 个删数据样本需放宽步长下限才能解出 (已如实报告, 见 §10.8)。

1. 表面列在极早期欠分辨:  $t = 1$  s 处  $\Delta C = 2.0 \times 10^{-4}$ ;
2. M0 不含蒸发潜热: 表面温度系统性偏高;

3. 一维轴对称简化：端面方向尺度比 39.06，未做全面 CFD；
4. 附件 1 只覆盖 0–4 h：问题 2/3 需外推；
5.  $\rho$  为湿物料体积密度： $\rho_d = \rho/(1+C)$  与干物质守恒不兼容（见 §9.4）；
6. 时间收敛阶的参照污染：拟合 +1.29 ~ +1.43（理论 +1）；
7.  $\rho_d$  不确定：附录 3 的  $\rho(C)/(1+C)$  在 275 ~ 650 间变化；
8.  $D$  分位区间只传播  $D$  一维：不是总不确定度；
9. 问题 2 全程版按 60 s 采样：题面要求“每隔 1 s”；
10. 未完成生产网格 vs 细参考网格的逐点对比：参照网格超出机时预算；
11. 缺少内部位移数据：附件 2 只给整体半径，无法识别内部位移是否仿射；
12. 缺少水活度关系：题目未给吸附等温线，无法做严格的气固转换；
13. 缺少目标成分动力学、应变/本构/强度参数；
14. 3 个删数据样本需放宽步长下限才能解出（见 §10.8，已如实报告）。

## 12.2 被省略因素的判据化取舍

本文识别出 13 条文献中提及、但未进入四问答案的因素。任何一条被排除，都归属且仅归属于三类判据之一，这一框架杜绝了“文献有但我没做”式的无判据省略：

表 33 三类判据（互斥且穷尽）

| 判据       | 含义                     | 条数 |
|----------|------------------------|----|
| A 量级判据   | 该效应在本工况下小于判据所能分辨的尺度    | 3  |
| B 数据判据   | 题面未给该效应必需的参数，引入即等于编造参数 | 7  |
| C 重复计算判据 | 题给公式已经隐含了该效应           | 3  |

表 34 13 条因素的归属

| 判据      | 条数 | 因素                                                                |
|---------|----|-------------------------------------------------------------------|
| B 数据缺失  | 7  | 蒸气渗流、辐射、结合水、干裂、各向异性、批次差异，以及 $\rho_d$ 的取值                          |
| A 量级/口径 | 3  | Soret 效应（占费克通量 0.3 ~ 1.1%）、控温波动（衰减 > 99%）、端面（最大值口径下 < $10^{-5}$ ） |
| C 重复计算  | 3  | 结壳、玻璃化（附录 3 的 $e^{-0.45/C}$ 已含 16.8 倍衰减）、双向耦合（实测边界已含耦合）           |

其中因素“双向耦合”值得单独说明：文献指出解耦会带来 22% ~ 35% 偏差，但本文直接采用附件 1 实测的烘房温湿度，物料蒸发对烘房的反作用已经在数据里——所以不是“忽略”，而是“自然消解”。

三个已经建模并量化、但同样不进入答案的因素，其排除理由各不相同，必须分开写清楚：

表 35 三个已建模因素的处置与理由

| 因素     | 进入答案 | 实测影响                                          | 为何这样处理                                |
|--------|------|-----------------------------------------------|---------------------------------------|
| 蒸发潜热   | 否    | $t_* + 2.84 \text{ h} (+4.9\%)$               | 题面未给 $\lambda$ ，且修正量正比于题面未定的 $\rho_d$ |
| 端面效应   | 否    | 对 $t_* < 10^{-5}$ ；对体积平均 $-2.2\% \sim -6.2\%$ | 判据是全域最大值，端面影响不到几何中心                   |
| 结壳/玻璃化 | 否    | $t_* + 42\% \sim +970\%$                      | 附录 3 的 $D(C, T)$ 已含低含水率衰减，再乘会重复计算     |



### 13 结论

本文建立一维轴对称径向热湿耦合模型，采用半控制体 FVM 与隐式时间推进求解，回答了四个问题，并通过解析基准、收敛性、守恒、退化、灵敏度与稳健性六类检验验证模型。

#### 13.1 四问答案

表 36 四个问题的答案

| 问题 | 答案                                                 | 关键说明                                    |
|----|----------------------------------------------------|-----------------------------------------|
| 1  | 1800 s: 中心 33.58 °C、表面 36.79 °C                    | 干燥只发生在表层（外侧约 0.7 cm）；中心含水率 30 min 内完全未变 |
| 2  | 3 h: 中心 49.85 °C、表面 49.97 °C；中心 1.77、表面 1.01 kg/kg | 双向耦合使 1800 s 中心温度比问题 1 低 1.386 °C       |
| 3  | $t_* = 57.49$ h (= 206959.9521 s = 2.40 天)         | 判据必须是全域最大值；用面积平均会低估 37.3%               |
| 4  | $t_{\text{dry}} = 51.0906$ h, 终态半径 1.2000 cm       | $\Delta t = 1$ s 收敛口径；与另一套独立实现相差 0.2%   |

#### 13.2 检验结论与误差预算

Bessel 解析基准误差  $< 0.001$  °C（相对  $< 0.002\%$ ）；空间二阶（实测 +2.20）、时间不低于一阶（不声称二阶）；内部界面通量抵消达机器精度  $1.782 \times 10^{-16}$ ；五项退化与一致性检验通过；跨实现逐行移植复现差 +11 s（0.004%）。几何—密度一致性诊断提示密度定义、数据误差、几何假设或干物质守恒的联合使用需核查（条件性结论，不下“数据有误”的判断）。

误差预算（§10.9）的落点是口径主导，不是数据主导：平台窗口取法一项即达 1090 s，比最大的数据扰动（135 s）还大 8 倍，也远大于全部数值离散误差（约 19 s）。

#### 13.3 灵敏度与稳健性

敏感度排序  $D \gg h_m \gg h$  在五档扰动下均不变；但  $|S_D|$  在  $\pm 5\%$  处为 0.886、在  $-50\%$  处为 1.794——灵敏度系数是局部量，报告时必须注明档位。数据扰动引起的散布为 0.22 ~ 67.6 s，全部经由“长时平台均值”单一渠道传递（ $r = -0.9997$ ）。三种区间（情景 / 参数扰动分位 / 数据扰动分位）严格分开命名，统计置信区间不给。

#### 13.4 模型局限与推广

**局限：**一维轴对称简化，端面效应以二维对照验证而非全面 CFD；M0 不含蒸发潜热（M1 对照显示 3 h 中心温度低 18.06 K、 $t_*$  延长 2.84 h，属条件性扩展，不改答案）；附件 1 只覆盖 0–4 h 需外推；缺少内部位移、水活度、目标成分动力学等数据。

**推广：**烘房—物料双向耦合系统模型（需空气干质量、装料量、新风/排风质量流量）；品质后处理（需水活度关系、成分动力学、菌种与初始菌量）；结构力学（需应变、本构、强度参数）。

### AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于代码调试与数值结果核验、文献检索与出处核实、以及语言润色，详细使用情况见支撑材料《AI 工具使用详情.pdf》。

核心建模与分析的自主性声明：

1. 本文的物理模型、控制方程、边界条件、数值格式与判据口径均由参赛队自主确定；AI 工具不参与模型选择与建模决策。

2. 本文全部数值结果由参赛队编写的求解程序计算产生。AI 工具用于代码调试与结果核验，其输出逐项经过人工审查：凡由 AI 辅助发现的代码缺陷，均要求提供可复现的最小反例后才予采纳（例如“误差随步长收敛阶被  $\log 0$  污染而虚高”、“台账合并误删历史记录”两处，都是先复现再修）。
3. 本文所有引用的数据与结论均可溯源：题面与附件数据标【题面/附件】，文献结论标明出处与 DOI，本文自算量显式标注“本文计算得到”。凡无法核实的外部数字一律不采用（见 §E）。
4. AI 工具生成的文字均经人工改写与核验，并由参赛队对全文的数值一致性做了自动校验（逐条比对论文中的关键数值与结果文件，共 25 项）。

## 参考文献

- [1] 2026 年高教社杯全国大学生数学建模竞赛 A 题《药材的烘干问题》题面（含附录 1–4）。
- [2] 附件 1：烘房温度与水分浓度（0–14400 s，241 点）。
- [3] 附件 2：药材半径随时间的变化（0–259200 s，145 点）。
- [4] 附件 3：result1–result4 结果模板。
- [5] Kumar, C., Millar, G. J., & Karim, M. A. (2015). *Effective diffusivity and evaporative cooling in convective drying of food material*. *Drying Technology*, **33**(2), 227–237. DOI: 10.1080/07373937.2014.947512.
- [6] Defraeye, T., Blocken, B., & Carmeliet, J. (2012). *Analysis of convective heat and mass transfer coefficients for convective drying of a porous flat plate*. *International Journal of Heat and Mass Transfer*, **55**(1–3), 36–48. DOI: 10.1016/j.ijheatmasstransfer.2011.08.047.
- [7] Datta, A. K. (2007). *Porous media approaches to studying simultaneous heat and mass transfer in food processes. I: Problem formulations*. *Journal of Food Engineering*, **80**(1), 80–95. DOI: 10.1016/j.jfoodeng.2006.05.013.
- [8] Vu, H. T., & Tsotsas, E. (2018). *Mass and heat transport models for analysis of the drying process in porous media: A review and numerical implementation*. *International Journal of Chemical Engineering*, **2018**, 9456418. DOI: 10.1155/2018/9456418.
- [9] Häussling Löwgren, B., Bergmann, J., & Alves-Filho, O. (2020). *A numerical implementation of the Soret effect in drying processes*. *ChemEngineering*, **4**(1), 13. DOI: 10.3390/chemengineering4010013.
- [10] Younsi, R., Kocaefe, D., Poncsák, S., & Kocaefe, Y. (2007). *Comparison of different models for the high-temperature heat-treatment of wood*. *International Journal of Thermal Sciences*, **46**(7), 707–716. DOI: 10.1016/j.ijthermalsci.2006.09.001.
- [11] Gulati, T., & Datta, A. K. (2015). *Mechanistic understanding of case-hardening and texture development during drying of food materials*. *Journal of Food Engineering*, **166**, 119–138. DOI: 10.1016/j.jfoodeng.2015.05.031.
- [12] Gulati, T., & Datta, A. K. (2012). *Multiphase transport with large deformations undergoing rubbery-glassy phase transition: Applications to drying*. *Proceedings of the 2012 COMSOL Conference, Boston*.
- [13] Khalloufi, S., & Bongers, P. (2012). *Mathematical investigation of the case hardening phenomenon explained by shrinkage and collapse mechanisms occurring during drying processes*. *Computer Aided Chemical Engineering*, **30**, 1068–1072. DOI: 10.1016/B978-0-444-59520-1.50072-5.



- [14] Khan, M. I. H., Wellard, R. M., Nagy, S. A., et al. (2016). *Investigation of bound and free water in plant-based food material using NMR  $T_2$  relaxometry*. Innovative Food Science & Emerging Technologies, **38**, 252–261. DOI: 10.1016/j.ifset.2016.10.015.
- [15] Sami, S., Rahimi, A., & Etesami, N. (2011). *Dynamic modeling and a parametric study of an indirect solar cabinet dryer*. Drying Technology, **29**(7), 825–835. DOI: 10.1080/07373937.2010.545159.
- [16] Ceylan, İ., Aktaş, M., & Doğan, H. (2007). *Mathematical modeling of drying characteristics of tropical fruits*. Applied Thermal Engineering, **27**(11–12), 1931–1936. DOI: 10.1016/j.applthermaleng.2006.12.020.
- [17] Takhar, P. S., Maier, D. E., & Campanella, O. H. (2011). *Hybrid mixture theory based moisture transport and stress development in corn kernels during drying: Coupled model and validation*. Journal of Food Engineering, **106**(2), 148–156. DOI: 10.1016/j.jfoodeng.2011.05.006.
- [18] Curcio, S., & Aversa, M. (2014). *Influence of shrinkage on convective drying of fresh vegetables: A theoretical model*. Journal of Food Engineering, **123**, 36–49. DOI: 10.1016/j.jfoodeng.2013.09.014.
- [19] Pacheco-Aguirre, F. M., Ladrón-González, A., Ruíz-Espinosa, H., et al. (2014). *A method to estimate anisotropic diffusion coefficients for cylindrical solids: Application to the drying of carrot*. Journal of Food Engineering, **125**, 24–33. DOI: 10.1016/j.jfoodeng.2013.10.015.
- [20] Abbasi Souraki, B., & Mowla, D. (2008). *Axial and radial moisture diffusivity in cylindrical fresh green beans in a fluidized bed dryer with energy carrier: Modeling with and without shrinkage*. Journal of Food Engineering, **88**(1), 9–19. DOI: 10.1016/j.jfoodeng.2007.05.013.
- [21] Mercier, S., Moresoli, C., Villeneuve, S., et al. (2013). *Sensitivity analysis of parameters affecting the drying behaviour of durum wheat pasta*. Journal of Food Engineering, **119**(1), 1–11. DOI: 10.1016/j.jfoodeng.2013.03.024.
- [22] Mortier, S. T. F. C., Gernaey, K. V., De Beer, T., & Nopens, I. (2014). *Global sensitivity analysis applied to drying models for one or a population of granules*. AIChE Journal, **60**(5), 1700–1717. DOI: 10.1002/aic.14383.
- [23] Bhandari, B. R., & Howes, T. (1999). *Implication of glass transition for the drying and stability of dried foods*. Journal of Food Engineering, **40**(1–2), 71–79. DOI: 10.1016/S0260-8774(99)00039-4.
- [24] Rahman, M. S. (2006). *State diagram of foods: Its potential use in food processing and product stability*. Trends in Food Science & Technology, **17**(3), 129–141. DOI: 10.1016/j.tifs.2005.09.009.
- [25] Champion, D., Le Meste, M., & Simatos, D. (2000). *Towards an improved understanding of glass transition and relaxations in foods: Molecular mobility in the glass transition range*. Trends in Food Science & Technology, **11**(2), 41–55. DOI: 10.1016/S0924-2244(00)00047-9.

## A 附录 A：支撑材料文件列表

### A.1 支撑材料压缩包内容

支撑材料打包为单个压缩文件（RAR / ZIP，< 20 MB），目录结构如下：

| 目录 / 文件          | 内容                                              |
|------------------|-------------------------------------------------|
| src/             | 完整可运行源程序（见附录 B，16 个模块共 4192 行）                  |
| scripts/         | 主运行脚本：问题 1-3、问题 4、参数灵敏度、数据扰动稳健性、端面对照            |
| data/            | 处理后的中间数据：环境插值双版本、半径 PCHIP 拟合、阶段识别结果             |
| results/         | 四个正式结果 <code>result1--result4.xlsx</code> 及运行信息 |
| figs/            | 全部 23 张插图的 PNG（300 dpi）与 PDF（矢量）                |
| docs/            | 各阶段结论与检验记录（数值证据的原始 JSON）                        |
| requirements.txt | 依赖：numpy, scipy, pandas, openpyxl, matplotlib   |
| README.md        | 复现步骤（见 A.3）                                     |
| AI 工具使用详情.pdf    | 依《人工智能工具使用规定》第 4 条提供                            |

## A.2 运行环境

Python 3.14.2、NumPy 2.4.4、SciPy 1.18.0、Matplotlib 3.11.0；问题 4 的对照实现另用 MATLAB。全部脚本纯 CPU、单线程即可复现；除数据扰动 Monte Carlo 部分外不含随机数，该部分固定随机种子。

## A.3 复现命令

```

pip install -r requirements.txt
python scripts/run_day3.py --stage all # 问题1-3: 冻结、收敛、复现
python scripts/run_p4.py --stage all # 问题4: 主结果、网格、时间收敛
python scripts/p4_paper.py --stage all # 论文口径的表6与B方案
python scripts/run_sens_param.py --stage all # 参数灵敏度 ±5/10/20%
python scripts/run_robust_data.py --stage all # 数据扰动稳健性

```

## B 附录 B：完整源程序代码

下列代码为建模与求解所用的全部核心源程序，与支撑材料中的 `src/`、`scripts/` 完全一致，可直接运行。

### B.1 配置与单位约定

`config.py`（196 行）

```

1  """
2  CODE-01 全局配置中心与单位约定      [甲 · M0 · Day1上午]
3
4  职责
5  ----
6  1. 冻结全局单位约定（时间 s / 长度 m / 温度内部用 °C, D 公式内转 K）
7  2. 集中存放几何、物性、初值、边界系数、数值参数、输出规格
8  3. 提供四档扩散系数关键值自检（check_units），任一不符即报错
9
10  单位约定（全局唯一，禁止在别处再转换）
11  -----
12      T   : 摄氏度 °C —— 场变量、初值、边界
13      T_K : 开尔文 K —— 仅用于附录3/附录4 的 D 公式，由 to_kelvin() 完成
14      r   : 米 m
15      t   : 秒 s
16      C   : kg/kg（干基含水率）
17
18  变量基准（CODE-34 入口检查第 1 项结论）

```

```

19 -----
20 C      = m_w / m_d  药材干基含水率
21 Y      = m_v / m_da 空气含湿量
22 C_inf_eq = 已明确基准的"等效环境平衡含水率", 即附件1 的水分浓度列。
23          本文按题设等效约定直接将其作为药材表面的等效平衡含水率,
24          不假装它是由真实空气状态求出的吸附平衡结果。
25 """
26
27 from __future__ import annotations
28
29 import math
30 from dataclasses import dataclass, field
31 from pathlib import Path
32
33 # -----
34 # 路径
35 # -----
36 PROJECT_ROOT = Path(__file__).resolve().parents[1] # ../26-CUMCM/甲 day1
37 ATTACH_DIR = PROJECT_ROOT.parent / "A题题目和附件" / "附件"
38 FIG_DIR = PROJECT_ROOT / "figs"
39 RESULT_DIR = PROJECT_ROOT / "results" / "M0"
40 DOC_DIR = PROJECT_ROOT / "docs"
41
42 # -----
43 # 几何 (题面: 圆柱, 长 25 cm, 半径 2 cm)
44 # -----
45 R0 = 0.02      # 初始半径 m
46 L0 = 0.25      # 长度 m
47 L_HALF = L0 / 2.0 # 半长 m —— 端面尺度分析必须用半长, 不得用全长
48
49
50 def to_kelvin(T_celsius):
51     """摄氏度 → 开尔文。全局唯一转换点。"""
52     return T_celsius + 273.15
53
54
55 # -----
56 # 初值
57 # -----
58 T_INIT = 28.0    # °C
59 C_INIT = 2.55    # kg/kg (干基)
60
61
62 # -----
63 # 物性 (三套, 按问题取用)
64 # -----
65 @dataclass(frozen=True)
66 class ConstProps:
67     """附录2 —— 问题1 用 (常数) """
68     rho: float = 820.0    # kg/m3
69     cp: float = 2600.0    # J/(kg · K)
70     k: float = 0.36       # W/(m · K)
71
72     @property
73     def alpha(self) -> float:
74         """热扩散率 m2/s"""
75         return self.k / (self.rho * self.cp)
76
77

```

```

78 PROPS_APP2 = ConstProps()
79
80 # -----
81 # 边界传递系数 (题设)
82 # -----
83 H_COEF = 25.0 # 对流换热系数  $W/(m^2 \cdot K)$ 
84 HM_COEF = 8.0e-7 # 对流传质系数  $m/s$ 
85
86 # -----
87 # 数值参数
88 # -----
89 @dataclass(frozen=True)
90 class Numerics:
91     # ---- 网格 ----
92     # 生产网格:  $N=200$  且  $grading=1.5$  (向表面加密的渐变网格)。
93     # 依据 (见 docs/Day1_结论.md §3): 水分在表面形成极薄边界层
94     # ( $t=1$  s 时厚度仅  $\sqrt{Dt}$   $0.07$  mm), 均匀网格即便  $N=640$  仍欠分辨;
95     # 而中心区剖面平坦无需加密。渐变网格  $N=200/=1.5$  以**约一半的计算量**
96     # 达到优于均匀  $N=640$  的精度 (内部各列偏差  $< 4$  位小数舍入量子  $5e-5$ )。
97     N: int = 200
98     grading: float = 1.5 # 1.0 = 均匀; >1 = 向  $r=R0$  加密
99     theta: float = 1.0 # 时间格式: 1.0 = Backward Euler (L-稳定); 0.5 = Crank-Nicolson
100     dt_init: float = 0.5 # 初始步长 s
101     dt_min: float = 1.0e-4 # 步长下限, 触底即报错 (不静默)
102     dt_max: float = 30.0 # 步长上限 s
103     safety: float = 0.9 # 误差控制安全因子
104     grow_max: float = 2.0 # 单步最大放大倍数
105     shrink_max: float = 0.2 # 单步最小收缩倍数
106     # 误差容限 (步长加倍法估计)
107     atol_T: float = 1.0e-5
108     rtol_T: float = 1.0e-7
109     atol_C: float = 1.0e-8
110     rtol_C: float = 1.0e-7
111     # 非线性迭代 (Picard)
112     picard_max_iter: int = 50
113     picard_rtol_u: float = 1.0e-8
114     picard_atol_u: float = 1.0e-10
115     picard_rtol_r: float = 1.0e-8
116     picard_atol_r: float = 1.0e-10
117     max_step_halving: int = 8 # 单步最多折半次数 (失败则报错)
118
119
120 NUMERICS = Numerics()
121
122 # -----
123 # 输出规格 (严格按附件3 模板实测结果)
124 # -----
125 T_START = 1 # result1/2 的 A 列从 1 起 (不是 0)
126 T_END = 1800 # 问题1 时间上限 s
127 R_OUT_CM = tuple(round(0.1 * i, 1) for i in range(21)) # 0.0, 0.1, ..., 2.0 cm
128 N_DECIMALS = 4 # 所有结果保留 4 位小数
129 SHEET_T = "温度"
130 SHEET_C = "水分浓度"
131 HEADER_A1 = "时间\\到药材中心的距离" # 含反斜杠, 已与附件3 模板逐字符比对一致
132
133
134 # -----
135 # 四档扩散系数关键值自检
136 # -----

```

```

137 # 附录2:  $D = 7e-9 * \exp(-0.89/C)$ 
138 # 附录3:  $D = 2.4e-3 * \exp(-0.45/C) * \exp(-3850/T_K)$ 
139 D_CHECKPOINTS = [
140     # (标签, 计算函数, 目标值, 相对容限)
141     ("附录2 @ C=0.05", lambda: 7.0e-9 * math.exp(-0.89 / 0.05), 1.3021e-16, 5e-3),
142     ("附录2 @ C=2.55", lambda: 7.0e-9 * math.exp(-0.89 / 2.55), 4.94e-9, 5e-3),
143     ("附录3 @ C=2.55, T_K=300",
144      lambda: 2.4e-3 * math.exp(-0.45 / 2.55) * math.exp(-3850.0 / 300.0),
145      5.3718662e-9, 1e-5),
146     ("附录3 @ C=0.15, T_K=323",
147      lambda: 2.4e-3 * math.exp(-0.45 / 0.15) * math.exp(-3850.0 / 323.0),
148      7.9570613e-10, 1e-5),
149 ]
150
151
152 def check_units(verbose: bool = True) -> dict:
153     """
154     四档 D 关键值自检。任一不符即抛错。
155     这是防止"T 误用摄氏度"公式系数抄错的看门人。
156     """
157     report = {}
158     bad = []
159     for label, fn, target, rtol in D_CHECKPOINTS:
160         got = fn()
161         rel = abs(got - target) / abs(target)
162         ok = rel <= rtol
163         report[label] = {
164             "computed": got, "target": target, "rel_err": rel, "ok": ok,
165             "to_kelvin_used": "附录3" in label,
166         }
167         if not ok:
168             bad.append(f" {label}: 计算 {got:.6e} vs 期望 {target:.6e} (相对误差 {rel:.2e})")
169     if bad:
170         raise AssertionError(
171             "CODE-01 单位/公式自检失败, 请检查物性公式与温度单位: \n" + "\n".join(bad)
172         )
173     if verbose:
174         print("[CODE-01] 单位与物性自检通过: ")
175         for label, d in report.items():
176             print(f" {label:28s} = {d['computed']:.6e} (期望 {d['target']:.6e})")
177     return report
178
179
180 def derived_numbers() -> dict:
181     """一些供论文引用的派生量。"""
182     a = PROPS_APP2.alpha
183     return {
184         "alpha": a,
185         "Bi_T": H_COEF * R0 / PROPS_APP2.k,
186         "tau_heat_s": R0 ** 2 / a,
187         "explicit_dt_limit_s": (R0 / NUMERICS.N) ** 2 / (4.0 * a),
188         "dr_m": R0 / NUMERICS.N,
189         "end_face_scale_ratio": (L_HALF / R0) ** 2,
190     }
191
192
193 if __name__ == "__main__":
194     check_units()
195     for k_, v in derived_numbers().items():

```

```
196 print(f" {k_:24s} = {v:.6g}")
```

## B.2 物性公式（附录 2/3/4）

properties.py (158 行)

```
1 """
2 CODE-07 物性求值模块 [甲主 / 乙审 • M0 • Day1上午]
3
4 职责
5 ----
6 统一提供三套物性公式（附录2 / 附录3 / 附录4）的求值接口。
7
8 约定
9 ----
10 * rho : 湿物料体积密度 —— **只用于热容项 rho*cp**
11 * rho_d: 干固体密度 rho/(1+C) —— **只用于几何—密度一致性诊断（CODE-35）**
12 二者定义不同，不允许互换（见 v3 清单 §3.1 CODE-07）
13 * T 传入的是**摄氏度**，本模块内部完成 T_K = T + 273.15（唯一转换点之二，
14 与 config.to_kelvin 同源）
15 * C 是**场变量**（同一时刻同一空间点的含水率），不是全场平均、不是初值
16
17 数值处理
18 -----
19 D 随 C 指数变化，在 C→0 时下溢。求值在 log 空间完成并做下截断，
20 保证返回有限值（0 或极小正数），不产生 NaN/Inf。
21 """
22
23 from __future__ import annotations
24
25 import numpy as np
26
27 from ..config import to_kelvin
28
29 # D 的下截断：低于此值视为 0（物理上已完全干燥）
30 _D_FLOOR = 1.0e-300
31
32
33 # =====
34 # 附录2 —— 问题1（常数物性 + D 只依赖 C）
35 # =====
36 def D_app2(C):
37     """
38     附录2 水分扩散系数 D = 7e-9 * exp(-0.89/C) [m2/s]
39
40     注意：本式**不含温度**，这正是问题1 中水分场与温度场解耦的结构性原因。
41     """
42     C = np.asarray(C, dtype=float)
43     out = np.zeros_like(C)
44     m = C > 0
45     with np.errstate(over="ignore", under="ignore", divide="ignore"):
46         out[m] = 7.0e-9 * np.exp(-0.89 / C[m])
47     out[~np.isfinite(out)] = 0.0
48     out[out < _D_FLOOR] = 0.0
49     return out if out.ndim else float(out)
50
51
52 # =====
```



```

53 # 附录3 —— 问题2、问题3 (C 与 T 的函数)
54 # =====
55 def _log_D_app3(C, T_K):
56     """log(D), 避免中间下溢。"""
57     C = np.maximum(np.asarray(C, dtype=float), 1e-12)
58     T_K = np.maximum(np.asarray(T_K, dtype=float), 1.0)
59     return np.log(2.4e-3) - 0.45 / C - 3850.0 / T_K
60
61
62 def D_app3(C, T_celsius):
63     """
64     附录3 D = 2.4e-3 * exp(-0.45/C) * exp(-3850/T_K) [m2/s]
65
66     T_celsius 传入摄氏度, 内部转 K。
67     """
68     T_K = to_kelvin(np.asarray(T_celsius, dtype=float))
69     ld = _log_D_app3(C, T_K)
70     out = np.exp(np.clip(ld, -700.0, 700.0))
71     out[ld < np.log(_D_FLOOR)] = 0.0
72     out[~np.isfinite(out)] = 0.0
73     return out
74
75
76 def rho_app3(C):
77     """附录3 湿物料体积密度 rho = 650 + 128C [kg/m3]"""
78     return 650.0 + 128.0 * np.asarray(C, dtype=float)
79
80
81 def cp_app3(C):
82     """附录3 比热容 cp = 1450 + 2736*C/(1+C) [J/(kg * K)]"""
83     C = np.asarray(C, dtype=float)
84     return 1450.0 + 2736.0 * C / (1.0 + C)
85
86
87 def k_app3(C):
88     """附录3 导热系数 k = 0.21 + 0.38*C/(1+C) [W/(m * K)]"""
89     C = np.asarray(C, dtype=float)
90     return 0.21 + 0.38 * C / (1.0 + C)
91
92
93 # =====
94 # 附录4 —— 问题4 (收缩)
95 # =====
96 def _log_D_app4(C, T_K):
97     C = np.maximum(np.asarray(C, dtype=float), 1e-12)
98     T_K = np.maximum(np.asarray(T_K, dtype=float), 1.0)
99     return np.log(4.2e-4) - 0.30 / C - 3850.0 / T_K
100
101
102 def D_app4(C, T_celsius):
103     """附录4 D = 4.2e-4 * exp(-0.30/C) * exp(-3850/T_K) [m2/s]"""
104     T_K = to_kelvin(np.asarray(T_celsius, dtype=float))
105     ld = _log_D_app4(C, T_K)
106     out = np.exp(np.clip(ld, -700.0, 700.0))
107     out[ld < np.log(_D_FLOOR)] = 0.0
108     out[~np.isfinite(out)] = 0.0
109     return out
110
111

```

```

112 def rho_app4(C):
113     """附录4 湿物料体积密度 rho = 760 + 90C [kg/m3]"""
114     return 760.0 + 90.0 * np.asarray(C, dtype=float)
115
116
117 def cp_app4(C):
118     """附录4 比热容 cp = 1850 + 2150*C/(1+C) [J/(kg * K)]"""
119     C = np.asarray(C, dtype=float)
120     return 1850.0 + 2150.0 * C / (1.0 + C)
121
122
123 def k_app4(C):
124     """附录4 导热系数 k = 0.12 + 0.20*C/(1+C) [W/(m * K)]"""
125     C = np.asarray(C, dtype=float)
126     return 0.12 + 0.20 * C / (1.0 + C)
127
128
129 # =====
130 # 干固体密度 (**仅用于 CODE-35 几何—密度一致性诊断**)
131 # =====
132 def rho_dry(rho, C):
133     """
134     rho_d = rho / (1 + C)
135
136     前提: 附录给出的 rho 是**湿物料体积密度**。
137     若 rho 实为热学等效参数, 则不得用于严格干质量检验 (v3 清单 §3.1 CODE-35)。
138     """
139     return np.asarray(rho, dtype=float) / (1.0 + np.asarray(C, dtype=float))
140
141
142 # =====
143 # 物性诊断: 打印关键值供人工复核
144 # =====
145 def describe():
146     """打印三套物性在状态区间端点上的取值。"""
147     import math
148     Cs = [0.05, 0.15, 0.5, 1.0, 2.55]
149     print(" C          D_app2          D_app3@323K D_app4@323K rho3    cp3    k3")
150     for C in Cs:
151         print(f" {C:5.2f} {D_app2(C):12.4e} {D_app3(C,50):12.4e}"
152               f" {D_app4(C,50):12.4e} {rho_app3(C):6.1f} {cp_app3(C):7.1f} {k_app3(C):6.4f}")
153     print(f"\n D_app3 跨 C [0.15,2.55] 的量级比 = "
154           f"{D_app3(2.55,50)/max(D_app3(0.15,50),1e-300):.1f} 倍")
155
156
157 if __name__ == "__main__":
158     describe()

```

### B.3 FVM 离散（半控制体 / 调和平均 / 渐变网格）

fvm\_cyl.py (376 行)

```

1  """
2  CODE-06 一维圆柱有限体积离散内核 [甲主 / 乙验 • M0 • Day1上午]
3
4  离散方案
5  -----
6  同心环形控制体, **中心用半径 Δr/2 的实心半圆柱**, 天然规避 1/r 奇点:

```

```

7 FVM 离散的是守恒形式  $d/dt \int_V dV = \int_{\partial V} \Gamma / n \, dA$  , 从不写出  $1/r \cdot r(r \cdot r)$  ,
8 因此不会出现除以趋零半径的运算。
9
10 网格 (N 个控制体, 均匀  $\Delta r = R0/N$ )
11     faces rf[j] = j * Δr      j = 0..N      (rf[0]=0 处通量为 0)
12     centers rc[i] = (i+0.5) * Δr i = 0..N-1
13     体积(单位轴向长度) V_0 = Δr2 ,   V_i = (rf[i+1]2-rf[i]2) = 2 * rc[i] * Δr
14     面积(单位轴向长度) A_j = 2 * rf[j]
15
16 界面扩散系数用**调和平均**
17     Γ_{j} = 2Γ_{j-1}Γ_j / (Γ_{j-1}+Γ_j)
18 这是分段常数 Γ 的精确串联电阻法则。D 跨约 7 个数量级时,
19 算术平均会被湿侧主导、高估干燥侧受扩散限制的通量, 调和平均是必需的。
20
21 离散方程 (无量纲化后统一形式  $d/dt = A + b$ )
22     d_0/dt = 2Γ_1(1-0)/Δr2
23     d_i/dt = [rf[i]Γ_i({i-1}-_i) + rf[i+1]Γ_{i+1}({i+1}-_i)] / (rc[i]Δr2)
24     d_{N-1}/dt = [rf[N-1]Γ_{N-1}({N-2}-_{N-1}) - ({N-1}-_w)] / (rc[N-1]Δr2)
25
26 Robin 边界 (单元中心格式的边界处理, 二阶)
27     边界半控制体内设线性剖面  $\Gamma/r = (\Gamma_s - \Gamma_N)/(\Delta r/2)$ , 与 Robin 条件联立消去  $\Gamma_s$ :
28      $\Gamma_s = (\Gamma_N + Bi_{\Delta} \cdot \Gamma_w)/(1 + Bi_{\Delta})$ ,  $Bi_{\Delta} = h_{bc} \cdot \Delta r/(2\Gamma_N)$ 
29      $J_s = h_{bc}(\Gamma_s - \Gamma_w) = h_{bc}(\Gamma_N - \Gamma_w)/(1 + Bi_{\Delta})$ 
30      $= R0 \cdot h_{bc} \cdot \Delta r/(1+Bi_{\Delta})$ 
31 注意: 表面对流系数在传热方程中须除以  $cp$  (即使用  $h/(cp)$ ),
32 在水分方程中直接用  $h_m$ 。两者量纲均为 m/s, 与  $\Gamma/R0$  一致。
33 """
34
35 from __future__ import annotations
36
37 from dataclasses import dataclass
38 from functools import lru_cache
39
40 import numpy as np
41 from scipy.linalg import solve_banded
42
43
44 # =====
45 # 网格几何缓存
46 # =====
47 @dataclass(frozen=True)
48 class _Geometry:
49     rf: np.ndarray      # 面半径 rf[0..N]
50     rc: np.ndarray      # 单元中心 rc[0..N-1]
51     drf: np.ndarray     # 面间距 rf[j+1] - rf[j]
52     h: np.ndarray       # 相邻单元中心距, i=1..N-1
53     V: np.ndarray       # 控制体体积 (单位轴向长度)
54     A_face: np.ndarray  # 面面积 (单位轴向长度)
55
56
57 @lru_cache(maxsize=64)
58 def _geometry(N: int, R0: float, grading: float) -> _Geometry:
59     """
60     由 (N, R0, grading) 唯一确定的全部几何量, 算一次缓存起来。
61
62     返回的数组是**共享的**：调用方**不得就地修改**。
```

(本项目所有用法都是只读的; assemble 只做乘加, 不写入这些数组。)

```

64     """
65     z = np.linspace(0.0, 1.0, N + 1)
```

```

66 rf = R0 * z if grading == 1.0 else R0 * (1.0 - (1.0 - z) ** grading)
67 rc = 0.5 * (rf[:-1] + rf[1:])
68 return _Geometry(
69     rf=rf, rc=rc, drf=np.diff(rf), h=np.diff(rc),
70     V=np.pi * (rf[1:] ** 2 - rf[:-1] ** 2),
71     A_face=2.0 * np.pi * rf,
72 )
73
74
75 # =====
76 # 网格
77 # =====
78 @dataclass(frozen=True)
79 class Grid:
80     """
81     一维圆柱径向网格。
82
83     grading = 1.0 → 均匀网格 ( $\Delta r = R0/N$ )，向后兼容
84     grading > 1.0 → **向表面  $r=R0$  加密**的渐变网格：
85
86          $r(j) = R0 \cdot [1 - (1 - j/N)^{grading}]$ 
87
88     *  $j=0 \rightarrow r=0$ ,  $j=N \rightarrow r=R0$ 
89     *  $dr/dj = R0 \cdot (1 - j/N)^{-(grading+1)}$  → 在表面处趋于 0，故表面附近步长最小
90     * 最大/最小步长比  $\sim N^{grading}$ 
91
92     选此设计的理由：水分在**表面**形成极薄边界层
93     ( $t=1\text{ s}$  时厚度仅  $\sqrt{Dt} \approx 0.07\text{ mm}$ )，均匀网格即便  $N=640$  仍欠分辨；
94     而中心区剖面平坦，无需加密。渐变网格以很小的代价解决该矛盾。
95     """
96     N: int
97     R0: float
98     grading: float = 1.0
99
100     @property
101     def dr(self) -> float:
102         return self.R0 / self.N
103
104     # 几何量全部走模块级缓存 (Day2 新增)
105     # 原先是每次访问都重算 np.linspace 与幂运算的 property。短程积分看不出来，
106     # 但 120 h 的长时积分里 rf 被调用 30 万次、独占 36% 的运行时间 (实测 profile)。
107     # 几何只由 (N, R0, grading) 决定，缓存后**数值完全不变**，只是不再重复计算。
108     @property
109     def _geom(self):
110         return _geometry(self.N, float(self.R0), float(self.grading))
111
112     @property
113     def rf(self) -> np.ndarray:
114         """面半径 rf[0..N]"""
115         return self._geom.rf
116
117     @property
118     def rc(self) -> np.ndarray:
119         """单元中心半径 rc[0..N-1] (相邻两面中点)"""
120         return self._geom.rc
121
122     @property
123     def h(self) -> np.ndarray:
124         """

```

```

125     相邻**单元中心**间距 h[i] = rc[i] - rc[i-1], i=1..N-1。
126     界面梯度用中心距而非面距，这是非均匀 FVM 的正确做法。
127     """
128     return self._geom.h
129
130 @property
131 def V(self) -> np.ndarray:
132     """控制体体积（单位轴向长度）"""
133     return self._geom.V
134
135 @property
136 def A_face(self) -> np.ndarray:
137     """面面积（单位轴向长度）A_face[j] = 2 * rf[j]"""
138     return self._geom.A_face
139
140 def summary(self) -> str:
141     h = self._geom.drf * 1e3
142     return (f"Grid N={self.N}, ={self.grading:g}, R0={self.R0*1e3:.1f}mm | "
143           f"Δr: {h.min():.5f} ~ {h.max():.5f} mm (比 {h.max()/h.min():.0f}:1)")
144
145
146 def harmonic_mean(g_left, g_right):
147     """调和平均，0 值安全。"""
148     a = np.asarray(g_left, dtype=float)
149     b = np.asarray(g_right, dtype=float)
150     s = a + b
151     out = np.zeros_like(s)
152     m = s > 0
153     out[m] = 2.0 * a[m] * b[m] / s[m]
154     return out
155
156
157 def face_diffusivity(Gamma_cell):
158     """
159     由单元中心值得到 N+1 个面上的扩散系数。
160     face[0] (r=0) : 对称面，不参与通量计算，置 Gamma_cell[0]
161     face[j] (1..N-1) : 调和平均
162     face[N] (r=R0) : 边界面，用边界单元值
163     """
164     G = np.asarray(Gamma_cell, dtype=float)
165     N = G.size
166     Gf = np.empty(N + 1, dtype=float)
167     Gf[0] = G[0]
168     Gf[1:N] = harmonic_mean(G[:-1], G[1:])
169     Gf[N] = G[-1]
170     return Gf
171
172
173 # =====
174 # 算子装配
175 # =====
176 @dataclass
177 class Operator:
178     """三对角算子 d/dt = A + b 的带状存储"""
179     N: int
180     diag: np.ndarray # A[i,i]
181     lower: np.ndarray # A[i,i-1] (lower[0] 未用)
182     upper: np.ndarray # A[i,i+1] (upper[N-1] 未用)
183     b: np.ndarray # 常数项

```

```

184 beta: float          # 边界系数, 供通量复算
185 Bi_delta: float
186 Gamma_face: np.ndarray
187
188 def matvec(self, phi):
189     out = self.diag * phi
190     out[1:] += self.lower[1:] * phi[:-1]
191     out[:-1] += self.upper[:-1] * phi[1:]
192     return out
193
194
195 def assemble(Gamma_cell, h_bc, phi_inf, grid: Grid, cap_cell=None) -> Operator:
196     """
197     装配算子。
198
199     参数
200     ----
201     Gamma_cell : (N,) 单元中心扩散系数 (传热用  $=k/(c_p)$ , 传质用  $D$ )
202     h_bc       : 表面对流系数 (传热用  $h/(c_p)$  或  $h$ ; 传质用  $h_m$ ), 单位 m/s
203     phi_inf    : 环境值 (标量)
204     grid       : Grid
205     cap_cell   : (N,) 体积热容 ( $c_p$ ), **仅变物性传热的守恒形式用**
206
207     cap_cell —— 变物性传热的正确写法 (Day2 新增)
208     -----
209     控制方程  $(C)c_p(C) \frac{T}{t} = (1/r) \frac{r}{r} (k(C) \frac{r}{r} \frac{T}{r})$  中
210     **变系数不得移出微分算子**。把守恒形式在控制体上积分:
211
212          $(c_p)_i V_i \frac{dT_i}{dt} = A_{i+1} k_{i+1/2} (T_{i+1} - T_i) / h_i$ 
213              $- A_i k_{i/2} (T_{i-1} - T_i) / h_{i-1}$ 
214
215     右端整体除以  $(c_p)_i V_i$ 。**面导热系数  $k$  用调和平均,
216     而热容  $(c_p)_i$  用单元值** —— 两者性质不同, 不可合并:
217
218     错误做法: 令  $\Gamma = k/(c_p)$  再走原路径。那样得到的是
219      $(k/(c_p))_{face}$ , 即把  $k$  与  $c_p$  在**同一个面上**一起做调和平均,
220     与正确的  $k_{face}/(c_p)_{cell}$  不等价。本工况  $c_p$  沿  $r$  变化可达
221     数倍 ( $C$  从 2.55 降到 0.15 时  $c_p$  由  $3.34e6$  降到  $2.20e6$ ),
222     该差异会污染温度场。
223
224     传入 cap_cell 时, 全部除  $V[i]$  改为除  $V[i] \cdot cap\_cell[i]$ ,
225     且 h_bc 应传**未归一化的  $h^*$  ( $W/(m^2 \cdot K)$ )、Gamma_cell 传  $**k^*$  ( $W/(m \cdot K)$ )。
226     cap_cell=None 时行为与 Day1 完全一致 (向后兼容)。
227     """
228     N, RO = grid.N, grid.RO
229     rf, rc = grid.rf, grid.rc
230     h = grid.h          # 单元中心间距  $h[i] = rc[i] - rc[i-1], i=1..N-1$ 
231     V = grid.V
232     A = grid.A_face
233     Gf = face_diffusivity(Gamma_cell)
234
235     # 体积除数:  $V_i$ , 或变物性传热时的  $(c_p)_i \cdot V_i$ 
236     if cap_cell is None:
237         Vd = V
238     else:
239         cap = np.asarray(cap_cell, dtype=float)
240         assert cap.shape == V.shape, "cap_cell 形状须与网格一致"
241         assert np.all(cap > 0.0), "体积热容必须为正"
242         Vd = V * cap

```

```

243
244 diag = np.zeros(N)
245 lower = np.zeros(N)
246 upper = np.zeros(N)
247 b = np.zeros(N)
248
249 # ---- 中心控制体  $i = 0$ : 内侧面  $r=0$  通量恒为 0 ----
250 #  $d/dt = A[1] \cdot J/V[0]$ ,  $J = -\Gamma(-)/h[0]$ 
251 c0 = A[1] * Gf[1] / (h[0] * Vd[0])
252 diag[0] = -c0
253 upper[0] = +c0
254
255 # ---- 内部及边界控制体 (向量化; 逐元素表达式与原循环逐一对应) ----
256 ii = np.arange(1, N) #  $i = 1..N-1$ 
257 # 内侧面  $r_f[i]$  的贡献
258 a_in = A[ii] * Gf[ii] / (h[ii - 1] * Vd[ii])
259 diag[ii] -= a_in
260 lower[ii] = a_in
261
262 jj = np.arange(1, N - 1) #  $i = 1..N-2$ 
263 # 外侧面  $r_f[i+1]$  的贡献
264 a_out = A[jj + 1] * Gf[jj + 1] / (h[jj] * Vd[jj])
265 diag[jj] -= a_out
266 upper[jj] = a_out
267
268 # ---- Robin 边界 ( $r = R0$ , 落在  $i = N-1$ ) ----
269 # 边界单元中心到表面的距离  $d = R0 - rc[N-1]$ 
270 d = R0 - rc[N - 1]
271 GN = float(Gf[N])
272 if GN > 0.0 and d > 0.0:
273     Bi_d = h_bc * d / GN
274     beta = A[N] * h_bc / ((1.0 + Bi_d) * Vd[N - 1])
275 else:
276     Bi_d = np.inf
277     beta = A[N] * h_bc / Vd[N - 1]
278 diag[N - 1] -= beta
279 b[N - 1] = beta * phi_inf
280
281 return Operator(N=N, diag=diag, lower=lower, upper=upper, b=b,
282               beta=beta, Bi_delta=Bi_d, Gamma_face=Gf)
283
284
285 # =====
286 # 隐式单步 (线性方程求解)
287 # =====
288 def _as_banded(diag, lower, upper, alpha):
289     """构造 solve_banded 需要的 (2, N) 带状矩阵: 解  $(\alpha \cdot I - A) x = rhs$ """
290     N = diag.size
291     ab = np.zeros((3, N))
292     ab[0, 1:] = -upper[:-1] # 超对角
293     ab[1, :] = alpha - diag # 主对角
294     ab[2, :-1] = -lower[1:] # 次对角
295     return ab
296
297
298 def implicit_step(op: Operator, phi_n, dt, theta=1.0, op_old: Operator | None = None):
299     """
300     -方法单步:  $(I/\Delta t - A) \phi^{n+1} = (I/\Delta t + (1-\theta)A) \phi^n + b^{n+1} + (1-\theta) b^n$ 
301

```



```

302 theta=1.0 → Backward Euler (L-稳定, 生产用)
303 theta=0.5 → Crank - Nicolson (仅用于线性温度基准验证时间二阶)
304 """
305 if op_old is None:
306     op_old = op
307 N = op.N
308 rhs = phi_n / dt
309 if theta < 1.0:
310     rhs += (1.0 - theta) * op_old.matvec(phi_n)
311     rhs += (1.0 - theta) * op_old.b
312 rhs += theta * op.b
313
314 ab = _as_banded(op.diag * theta, op.lower * theta, op.upper * theta, 1.0 / dt)
315 return solve_banded((1, 1), ab, rhs, check_finite=False)
316
317
318 # =====
319 # 表面重构与通量 (供输出、守恒检验使用)
320 # =====
321 def surface_distance(grid: Grid) -> float:
322     """边界单元中心到表面 r=RO 的距离 d = RO - rc[N-1] (均匀网格时为 Δr/2)。"""
323     return float(grid.RO - grid.rc[-1])
324
325
326 def _bi_delta(gamma_N, h_bc, grid: Grid):
327     """边界单元 Biot 数 Bi_d = h_bc * d / Γ_N, d 为边界单元中心到表面的距离。"""
328     d = surface_distance(grid)
329     if gamma_N <= 0.0 or d <= 0.0:
330         return np.inf
331     return h_bc * d / float(gamma_N)
332
333
334 def surface_value(phi, Gamma_cell, h_bc, phi_inf, grid: Grid):
335     """
336     由单元中心值重构表面值 (二阶):
337     _s = (_N + Bi_d * _w) / (1 + Bi_d), Bi_d = h_bc * d / Γ_N
338     """
339     G_N = float(np.asarray(Gamma_cell)[-1])
340     Bi_d = _bi_delta(G_N, h_bc, grid)
341     if not np.isfinite(Bi_d):
342         return float(phi_inf)
343     return float((phi[-1] + Bi_d * phi_inf) / (1.0 + Bi_d))
344
345
346 def surface_flux(phi, Gamma_cell, h_bc, phi_inf, grid: Grid):
347     """表面处**向外**的通量 J_s = h_bc(_N - _w) / (1 + Bi_d)"""
348     G_N = float(np.asarray(Gamma_cell)[-1])
349     Bi_d = _bi_delta(G_N, h_bc, grid)
350     if not np.isfinite(Bi_d):
351         return 0.0
352     return float(h_bc * (phi[-1] - phi_inf) / (1.0 + Bi_d))
353
354
355 def flux_loop(phi, Gamma_cell, h_bc, phi_inf, grid: Grid):
356     """
357     独立用面通量循环重算散度 (非均匀网格通用形式):
358
359     d_i/dt = [A_{i+1}J_{i+1} - A_i J_i] / V_i, J_j = -Γ_j(_j - _{j-1}) / h_j
360

```

```

361 返回 (net, Jface): net 为各控制体的 散度 * dV (量纲), Jface 为面通量。
362 """
363 Gf = face_diffusivity(Gamma_cell)
364 N, RO = grid.N, grid.RO
365 rc, h, V, A = grid.rc, grid.h, grid.V, grid.A_face
366 phi = np.asarray(phi, dtype=float)
367
368 Jface = np.zeros(N + 1)
369 Jface[1:N] = -Gf[1:N] * (phi[1:] - phi[:-1]) / h
370 Jface[0] = 0.0
371 G_N = float(Gf[N])
372 Bi_d = _bi_delta(G_N, h_bc, grid)
373 Jface[N] = 0.0 if not np.isfinite(Bi_d) else h_bc * (phi[-1] - phi_inf) / (1.0 + Bi_d)
374
375 net = A[:-1] * Jface[:-1] - A[1:] * Jface[1:]
376 return net, Jface

```

#### B.4 非线性迭代 (Picard, 双重判据)

nonlinear.py (110 行)

```

1  """
2  CODE-09 非线性迭代器          [甲 • MO • Day1下午]
3
4  问题
5  ----
6  水分方程  $C/t = \cdot (D(C) C)$  中  $D$  依赖  $C$ , 且  $D$  在  $C [0.15, 2.55]$  上跨越约 7 个数量级,
7  方程强刚性。隐式格式每步需求解非线性代数方程组。
8
9  方法: Picard 迭代 (主)
10  -----
11   $(I/\Delta t - A(C^{(k)})) C^{(k+1)} = C^n/\Delta t + b(C^{(k)})$ 
12
13  选 Picard 而非 Newton 的理由:
14  * 迭代矩阵保持 M-矩阵性质 → 每个迭代满足极值原理 → **C 的正性由构造保证**,
15  无需任何裁剪, 也不会像 Newton 那样过冲进入  $\exp(-0.89/C)$  的悬崖区
16  * 实现简单、无需 Jacobian
17  Newton 作为可选后备 (final tightening / Day2 耦合问题复用)。
18
19  收敛判据 (**尺度归一化的双重判据, 两个都必须满足**)
20  -----
21  更新量:  $\max_i |C^{(k+1)}_i - C^{(k)}_i| / (atol_u + rtol_u \cdot |C^{(k+1)}_i|)$  1
22  残差 :  $\max_i |F_i(C^{(k+1)})| / (atol_r + rtol_r \cdot \max(|C/\Delta t|_i, 1e-3))$  1
23
24  **残差判据是关键**: 只检查更新量会落入"更新量变小但残差停滞"的经典陷阱。
25
26  停滞检测
27  -----
28  若更新量范数在某次迭代中未能缩小到上一轮的 0.5 倍以下 → 判定停滞, 返回失败。
29  失败由外层步长折半重试处理 (见 CODE-08)。
30  """
31
32  from __future__ import annotations
33
34  import numpy as np
35
36  from .fvm_cyl import implicit_step
37

```

```

38
39 class NonlinearFailure(RuntimeError):
40     """非线性迭代未收敛。由外层步长折半重试处理。"""
41
42
43 def picard_solve(assemble_fn, phi_n, dt, theta, tol, max_iter,
44                 phi_init=None, op_old=None):
45     """
46     Picard 迭代求解单个隐式时间步。
47
48     参数
49     ----
50     assemble_fn : callable(phi) -> Operator 给定 phi 装配算子
51     phi_n       : (N,) 上一时刻的解
52     dt          : 步长
53     theta       : 时间格式参数
54     tol         : 收敛容限对象 (需含 atol_u/rtol_u/atol_r/rtol_r)
55     max_iter    : 最大迭代次数
56     phi_init    : 初值猜测; None 时用 phi_n
57     op_old     : <1 时上一时刻的算子
58
59     返回
60     ----
61     (phi, n_iter, residual_norm)
62
63     异常
64     ----
65     NonlinearFailure : 迭代未收敛或判定停滞
66     """
67     phi = np.array(phi_n if phi_init is None else phi_init, dtype=float)
68     prev_upd = np.inf
69
70     for k in range(1, max_iter + 1):
71         op = assemble_fn(phi)
72         phi_new = implicit_step(op, phi_n, dt, theta=theta, op_old=op_old)
73
74         denom_u = tol["atol_u"] + tol["rtol_u"] * np.abs(phi_new)
75         upd = float(np.max(np.abs(phi_new - phi) / denom_u))
76
77         # 真残差: 在 phi_new 处重新装配算子后计算 F(phi_new)
78         op_new = assemble_fn(phi_new)
79         F = (phi_new - phi_n) / dt - op_new.matvec(phi_new) - op_new.b
80         denom_r = tol["atol_r"] + tol["rtol_r"] * np.maximum(np.abs(phi_new / dt), 1e-3)
81         res = float(np.max(np.abs(F) / denom_r))
82
83         if (upd <= 1.0) and (res <= 1.0):
84             return phi_new, k, res
85
86         # 停滞检测: 更新量不再显著下降
87         if k > 2 and upd > 0.5 * prev_upd and upd > 1.0:
88             raise NonlinearFailure(
89                 f"Picard 停滞于第 {k} 次迭代: 更新量 {upd:.3e}, 残差 {res:.3e}"
90             )
91         prev_upd = upd
92         phi = phi_new
93
94     raise NonlinearFailure(
95         f"Picard 达到最大迭代次数 {max_iter}: 更新量 {upd:.3e}, 残差 {res:.3e}"
96     )

```

```

97
98
99 def jacobian_diag_D(C, D, dD_dC):
100     """
101     Newton 后备用的 Jacobian 对角元 (Day2 耦合问题复用)。
102
103     对界面通量使用调和平均时,  $d\Gamma_{i+1/2}/dC_i = 2 D_{i+1}/(D_i + D_{i+1})^2 \cdot dD_i/dC_i$ 
104     此处仅返回单元中心的  $dD/dC$ , 供上层组装。
105     """
106     C = np.asarray(C, dtype=float)
107     with np.errstate(divide="ignore", over="ignore", invalid="ignore"):
108         d = 0.89 / np.maximum(C, 1e-12) ** 2 * np.asarray(D, dtype=float)
109     d[~np.isfinite(d)] = 0.0
110     return d

```

## B.5 时间推进 (Backward Euler + 自适应步长)

time\_stepper.py (293 行)

```

1  """
2  CODE-08 隐式时间推进器          [甲 · MO · Day1下午]
3
4  为什么必须隐式
5  -----
6  半控制体 FVM 中心处显式稳定性上限约  $\Delta t \leq \Delta r^2/(4\kappa)$ 。
7   $\Delta r = 0.5 \text{ mm}$ 、 $\kappa = k/(cp) = 1.6886e-7 \text{ m}^2/\text{s}$  时该上限约  $0.37 \text{ s}$ ，
8  无法满足"输出间隔 1 s"。故采用 L-稳定的 Backward Euler。
9  (验收方式: **检查稳定性条件并拒绝不满足的设置**, 而非"观察发散".)
10
11  关键约定
12  -----
13  * **输出间隔 计算步长**。步长由误差控制与边界变化决定;
14    每步都把  $dt$  裁剪到"恰好落在下一个输出时刻", 因此**输出时刻从不靠插值**。
15  * Crank-Nicolson ( $\alpha=0.5$ ) 实现但**仅用于线性温度基准**验证时间二阶;
16    对刚性的水分方程不使用 (会在陡峭前沿振铃)。
17
18  误差控制: 步长加倍法 (Richardson)
19  -----
20    一步  $u_{full} = \Phi_{\Delta t}(u^n)$ 
21    两步  $u_{half} = \Phi_{\Delta t/2}(\Phi_{\Delta t/2}(u^n))$ 
22     $err = \max_i |u_{half,i} - u_{full,i}| / (atol + rtol \cdot |u_{half,i}|)$ 
23     $err < 1$  接受; 新步长  $dt \cdot \text{clamp}(\text{safety} \cdot err^{-1/(p+1)}, \text{shrink}, \text{grow})$ 
24
25  Backward Euler 阶  $p = 1 \rightarrow$  指数  $1/2$ ; Crank-Nicolson  $p = 2 \rightarrow$  指数  $1/3$ 。
26  """
27
28  from __future__ import annotations
29
30  from dataclasses import dataclass, field
31
32  import numpy as np
33
34  from .nonlinear import NonlinearFailure
35
36
37  class StepSizeTooSmall(RuntimeError):
38      """步长触底, 说明问题设置或实现有误 (不静默)。"""
39

```

```

40
41 @dataclass
42 class StepRecord:
43     """单个计算步的记录 —— 供 FIG-41 与 CODE-19 使用。"""
44     t: float
45     dt: float
46     iters_C: int
47     err_T: float
48     err_C: float
49     accepted: bool
50     n_halving: int = 0
51     note: str = ""
52
53
54 @dataclass
55 class IntegrateResult:
56     times: np.ndarray          # 输出时刻
57     T: np.ndarray              # (n_out, N)
58     C: np.ndarray              # (n_out, N)
59     history: list = field(default_factory=list)
60     n_accepted: int = 0
61     n_rejected: int = 0
62     n_halving_total: int = 0
63     n_warmup: int = 0          # 起始层强制等步长的步数
64     t_final: float = 0.0       # 实际推进到的时刻 (早停时 < t_end)
65     stop_reason: str = ""      # completed / short / event
66
67
68 # =====
69 # 误差范数
70 # =====
71 def scaled_error(u_half, u_full, atol, rtol):
72     d = np.abs(u_half - u_full)
73     s = atol + rtol * np.abs(u_half)
74     return float(np.max(d / s))
75
76
77 # =====
78 # 自适应推进
79 # =====
80 class AdaptiveStepper:
81     def __init__(self, num, state_keys=("T", "C")):
82         self.num = num
83         self.keys = state_keys
84         self.dt = num.dt_init
85         self.history: list[StepRecord] = []
86         self.n_accepted = 0
87         self.n_rejected = 0
88         self.n_halving_total = 0
89
90     # -----
91     def _err(self, half, full):
92         eT = scaled_error(half["T"], full["T"], self.num.atol_T, self.num.rtol_T)
93         eC = scaled_error(half["C"], full["C"], self.num.atol_C, self.num.rtol_C)
94         return eT, eC
95
96     def _try_step(self, t, dt, state, step_fn):
97         """执行一次尝试步; 返回 (half_state, full_state, iters_C, n_half) 或 None (非线性失败)。"""
98         "

```

```

98     try:
99         full, it_full = step_fn(t, dt, state)
100         mid, it_a = step_fn(t, dt / 2.0, state)
101         half, it_b = step_fn(t + dt / 2.0, dt / 2.0, mid)
102         return half, full, max(it_full, it_a, it_b), 0
103     except NonlinearFailure:
104         return None
105
106 # -----
107 def integrate(self, t0, state0, t_end, t_output, step_fn, p_order=1.0,
108               progress=None, warmup_steps=0, warmup_dt=1.0e-3,
109               stop_fn=None):
110     """
111     自适应推进。
112
113     参数
114     ----
115     step_fn : callable(t, dt, state) -> (state_new, n_iter_C)
116               调用者负责 方法与线性/非线性求解
117     t_output: 需要精确落点的输出时刻 (升序 np.ndarray)
118     p_order : 时间格式阶数 (BE=1, CN=2)
119     warmup_steps, warmup_dt :
120         **起始层处理** (Day2 新增, M1 必需)。见下方说明。
121     stop_fn : callable(t, state) -> bool。在每个**被接受**的步之后询问,
122               返回 True 即停止 (`stop_reason="event"` )。
123               **事件早停** (Day2 新增): 问题3 的阈值搜索若不定早停,
124               会一路跑满 6 天视界, 白白多算 2—3 倍;
125               情景扫描 (CODE-25) 里这个浪费会放大 20 倍。
126
127     为什么需要 warmup —— 步长加倍法在 t=0 处会失效
128     -----
129     若边界条件在 t=0 处有**阶跃** (M1 的蒸发潜热正是如此:
130     t=0 瞬间  $J_w = 5.6e-4 \text{ kg}/(\text{m}^2 \cdot \text{s})$ , 等效环境温度被压到  $-22 \text{ }^\circ\text{C}$ ),
131     则真解在表面附近按  $\sqrt{t}$  规律演化 —— 解析解的时间导数在 t=0 发散。
132
133     此时 Backward Euler 跨在 t=0 上的**首个**步的局部误差不是  $O(\Delta t^2)$ ,
134     而是  $O(\Delta t^{\frac{1}{2}})$ ; 步长加倍法量到的  $|u_{\text{full}} - u_{\text{half}}|$  也按
135      $\Delta t^{\frac{1}{2}}$  衰减。于是"缩小步长使 err = 1"永远无法达成:
136     实测 err 在  $\Delta t = 1e-4 \text{ s}$  时仍为 161.6, 且**与  $\Delta t$  无关地保持同一数值**,
137     最终触发 StepSizeTooSmall。这是估计器的适用条件问题, 不是物理发散。
138
139     处置: 在  $[t_0, t_0 + \text{warmup\_steps} \cdot \text{warmup\_dt}]$  内**强制等步长、不做误差控制**,
140     把  $\sqrt{t}$  起始层走完; 之后解已光滑, 估计器恢复正常, 再交回自适应控制。
141     起始层内的误差必须**单独量化并在论文中如实报告**
142     (做法: 改变 warmup_dt 重跑, 比较关心量的差异),
143     不得因为"后面看不出差别"就默认它不存在。
144     """
145     # 有起始层时从 warmup_dt 起步 (而非 dt_init),
146     # 使自适应控制从起始层结束处**平滑接管**, 避免一上来就连续拒绝大步长
147     self.dt = float(warmup_dt) if warmup_steps > 0 else self.num.dt_init
148     t = float(t0)
149     state = {k: np.array(v, dtype=float) for k, v in state0.items()}
150
151     out_t, out_T, out_C = [], [], []
152     next_out = 0
153     n_out = len(t_output)
154     tol_out = 1e-9
155
156     # t0 若是输出时刻, 先记录

```

```

157     if n_out and abs(t - t_output[0]) < tol_out:
158         out_t.append(t); out_T.append(state["T"].copy()); out_C.append(state["C"].copy())
159         next_out = 1
160
161     expo = 1.0 / (p_order + 1.0)
162     guard = 0
163     guard_max = 5_000_000
164     n_warm = 0
165
166     def _record_out():
167         nonlocal next_out
168         while next_out < n_out and abs(t - t_output[next_out]) < 1e-9:
169             out_t.append(t)
170             out_T.append(state["T"].copy())
171             out_C.append(state["C"].copy())
172             next_out += 1
173
174     while t < t_end - 1e-12:
175         guard += 1
176         if guard > guard_max:
177             raise RuntimeError("推进步数超限, 疑似死循环")
178
179         dt = min(self.dt, t_end - t)
180         # ---- 精确落在下一个输出时刻 ----
181         if next_out < n_out:
182             t_next = float(t_output[next_out])
183             if t_next > t + 1e-12 and t + dt > t_next:
184                 dt = t_next - t
185
186         in_warmup = n_warm < warmup_steps
187         if in_warmup:
188             dt = min(warmup_dt, dt)
189
190         if dt < self.num.dt_min:
191             raise StepSizeTooSmall(
192                 f"t={t:.6g} 处步长 {dt:.3e} 触底 (dt_min={self.num.dt_min:.1e})"
193             )
194
195         if in_warmup:
196             # ---- 起始层: 单步推进, 不做误差估计 ----
197             out = step_fn(t, dt, state)
198             state = {k: np.array(v, dtype=float) for k, v in out[0].items()}
199             t += dt
200             n_warm += 1
201             self.n_warmup = n_warm
202             self.history.append(StepRecord(
203                 t=t - dt, dt=dt, iters_C=out[1], err_T=-1.0, err_C=-1.0,
204                 accepted=True, n_halving=0, note="warmup"))
205             _record_out()
206             continue
207
208         res = self._try_step(t, dt, state, step_fn)
209         n_halving = 0
210         while res is None:
211             n_halving += 1
212             self.n_halving_total += 1
213             if n_halving > self.num.max_step_halving:
214                 raise StepSizeTooSmall(
215                     f"t={t:.6g} 处非线性连续失败 {n_halving} 次, 步长已缩至 {dt:.3e}"

```



```

216         )
217         dt *= 0.5
218         res = self._try_step(t, dt, state, step_fn)
219
220         half, full, iters, _ = res
221         err_T, err_C = self._err(half, full)
222         err = max(err_T, err_C)
223         accepted = (err <= 1.0) and (n_halving == 0)
224
225         rec = StepRecord(t=t, dt=dt, iters_C=iters, err_T=err_T, err_C=err_C,
226                         accepted=accepted, n_halving=n_halving)
227         self.history.append(rec)
228
229         if accepted:
230             state = half
231             t += dt
232             self.n_accepted += 1
233             factor = self.num.safety * (err ** -expo if err > 0 else self.num.grow_max)
234             factor = float(np.clip(factor, self.num.shrink_max, self.num.grow_max))
235             self.dt = float(np.clip(dt * factor, self.num.dt_min, self.num.dt_max))
236             _record_out()
237             # ---- 事件早停（在输出已记录之后判定，保证停机点可复现） ----
238             if stop_fn is not None and stop_fn(t, state):
239                 self.n_early_stop = True
240                 break
241         else:
242             # 拒绝：状态不变，缩小步长重试
243             self.n_rejected += 1
244             factor = self.num.safety * (err ** -expo if err > 0 else 0.5)
245             factor = float(np.clip(factor, 0.1, 0.8))
246             self.dt = max(dt * factor, self.num.dt_min)
247
248         early = bool(getattr(self, "n_early_stop", False))
249         if next_out < n_out and not early:
250             # 允许 t_end 稍短于最后一个输出时刻的极端情形：补齐
251             while next_out < n_out:
252                 out_t.append(float(t_output[next_out]))
253                 out_T.append(state["T"].copy())
254                 out_C.append(state["C"].copy())
255                 next_out += 1
256         elif next_out < n_out and early:
257             # **事件早停时绝不能补齐**。
258             # 早停点的状态只在 t* 时刻有效，把它复制到之后几千个输出时刻
259             # 会伪造出一整段“含水率不再变化”的假结果
260             # （实测：t*=57.6 h 早停后仍补出 8640 行、末行到 518400 s）。
261             # 未到达的输出时刻**就是没有结果**，如实截断。
262             pass
263
264         self.n_warmup = n_warm
265         if getattr(self, "n_early_stop", False):
266             reason = "event"
267         elif t >= t_end - 1e-9:
268             reason = "completed"
269         else:
270             reason = "short"
271         return IntegrateResult(
272             times=np.asarray(out_t),
273             T=np.asarray(out_T),
274             C=np.asarray(out_C),

```

```

275         history=self.history,
276         n_accepted=self.n_accepted,
277         n_rejected=self.n_rejected,
278         n_halving_total=self.n_halving_total,
279         n_warmup=n_warm,
280         t_final=float(t),
281         stop_reason=reason,
282     )
283
284
285 # =====
286 # 显式稳定性上限（供配置检查，不作为推进方式）
287 # =====
288 def explicit_stability_limit(dr, alpha):
289     """
290     半控制体 FVM 中心处的显式稳定性上限  $\Delta t^2/(4)$ 。
291     实际限制应依据所选定的离散算子确定，此处给出基准量级供配置自检。
292     """
293     return dr ** 2 / (4.0 * alpha)

```

## B.6 双向耦合装配

coupled.py (179 行)

```

1  """
2  CODE-12a 双向耦合的块 Picard 迭代    [甲 • M0 • Day2上午]
3
4  为什么问题2 不能沿用问题1 的"先水后温"
5  -----
6  问题1 的结构简化来自两条**同时**成立的事实：
7      附录2 的 D(C) 不含 T → 水分场独立于温度场
8      M0 的热方程不含蒸发项 → 温度场也独立
9  问题2 从 t=0 起统一采用附录3，其中
10
11      D = 2.4e-3 • exp(-0.45/C) • exp(-3850/T_K)
12
13  **显含 T**，于是失效：水分场必须知道温度才能推进。
14  （在 M0 下仍成立，但 M1 启用潜热后也失效 —— 见 CODE-13。）
15
16  因此 T 与 C 必须**联立隐式求解**。

```

```

35 → **C 与 T 的正性/有界性由构造保证**, 无需裁剪,
36 也不会像 Newton 那样过冲进入  $\exp(-0.45/C)$  的悬崖区 ( $C \rightarrow 0$  时导数发散)。
37
38 收敛判据 (**尺度归一化的双重判据, 两个场、两个判据都必须满足**)
39 -----
40     更新量:  $U = \max(\max|\Delta T|/(\text{atol\_uT} + \text{rtol\_uT}|T|), \max|\Delta C|/(\text{atol\_uC} + \text{rtol\_uC}|C|))$  1
41     残差 :  $R = \max(\|F\_T\|_{\text{scaled}}, \|F\_C\|_{\text{scaled}})$  1
42
43 只检查更新量会落入"更新量变小但残差停滞"的经典陷阱, 故残差判据不可省。
44
45 停滞检测: 更新量连续两轮不降到 0.5 倍以下 → 判定停滞,
46 交由外层 (CODE-08) 折半步长重试。
47 """
48
49 from __future__ import annotations
50
51 from dataclasses import dataclass, field
52
53 import numpy as np
54
55 from .fvm_cyl import implicit_step
56 from .nonlinear import NonlinearFailure
57
58
59 @dataclass
60 class CoupledIteration:
61     """单个时间步内块迭代的收敛记录 —— 供 FIG-41、CODE-19 使用。"""
62     iters: int = 0
63     upd_T: float = np.inf
64     upd_C: float = np.inf
65     res_T: float = np.inf
66     res_C: float = np.inf
67     converged: bool = False
68     note: str = ""
69
70
71 def block_picard_solve(T_n, C_n, dt, theta, assemble_T, assemble_C,
72                       tol_T, tol_C, max_iter=50, T_init=None, C_init=None,
73                       op_T_old=None, op_C_old=None):
74     """
75     求解一个隐式时间步内的耦合非线性方程。
76
77     参数
78     ----
79     T_n, C_n      : (N,) 上一时刻的解
80     dt, theta     : 时间步长与  $\theta$  (生产用  $\theta=1$ , Backward Euler)
81     assemble_C    : callable( $C_{\text{guess}}, T_{\text{guess}}$ ) → Operator
82                   水分算子; D 依赖 (C, T), 故两者都要传
83     assemble_T    : callable( $T_{\text{guess}}, C_{\text{guess}}$ ) → Operator
84                   温度算子;  $c_p, k$  依赖 C; M1 时 b 还依赖 C 决定的  $J_w$ 
85     tol_T, tol_C  : dict(atol_u, rtol_u, atol_r, rtol_r)
86     T_init/C_init: 初值猜测 (None 时用上一时刻值)
87
88     返回
89     ----
90     (T_new, C_new, CoupledIteration)
91
92     异常
93     ----

```

```

94 NonlinearFailure : 未收敛或判定停滞 (外层折半步长重试)
95 """
96 T = np.array(T_n if T_init is None else T_init, dtype=float)
97 C = np.array(C_n if C_init is None else C_init, dtype=float)
98
99 prev_c = np.inf
100 upd_c = np.inf
101
102 for k in range(1, max_iter + 1):
103     C_prev = C
104     T_prev = T
105
106     # ----- 1. 水分场 (用到当前的 T) -----
107     op_C = assemble_C(C, T)
108     C = implicit_step(op_C, C_n, dt, theta=theta, op_old=op_C_old)
109
110     # ----- 2. 温度场 (用**最新**的 C, Gauss-Seidel) -----
111     op_T = assemble_T(T, C)
112     T = implicit_step(op_T, T_n, dt, theta=theta, op_old=op_T_old)
113
114     # ----- 3. 双重判据 -----
115     upd_T = float(np.max(np.abs(T - T_prev) /
116                          (tol_T["atol_u"] + tol_T["rtol_u"] * np.abs(T))))
117     upd_C = float(np.max(np.abs(C - C_prev) /
118                          (tol_C["atol_u"] + tol_C["rtol_u"] * np.abs(C))))
119
120     # 真残差: 在**新解处重新装配**算子后计算  $F(u^{k+1})$ 
121     op_Cn = assemble_C(C, T)
122     F_C = (C - C_n) / dt - op_Cn.matvec(C) - op_Cn.b
123     res_C = float(np.max(np.abs(F_C) /
124                          (tol_C["atol_r"] + tol_C["rtol_r"] *
125                           np.maximum(np.abs(C / dt), 1e-3))))
126
127     op_Tn = assemble_T(T, C)
128     F_T = (T - T_n) / dt - op_Tn.matvec(T) - op_Tn.b
129     # 温度残差按"每步温升尺度"归一化; 1e-3 作下限防止全程恒温时过严
130     res_T = float(np.max(np.abs(F_T) /
131                          (tol_T["atol_r"] + tol_T["rtol_r"] *
132                           np.maximum(np.abs(T / dt), 1e-3))))
133
134     upd = max(upd_T, upd_C)
135     res = max(res_T, res_C)
136
137     if (upd <= 1.0) and (res <= 1.0):
138         return T, C, CoupledIteration(iters=k, upd_T=upd_T, upd_C=upd_C,
139                                       res_T=res_T, res_C=res_C,
140                                       converged=True)
141
142     # 停滞检测: 更新量不再显著下降 (对两个场的联合更新量判定)
143     if k > 3 and upd > 0.5 * prev_c and upd > 1.0:
144         raise NonlinearFailure(
145             f"块 Picard 停滞于第 {k} 次迭代: 联合更新量 {upd:.3e}"
146             f" (T {upd_T:.3e} / C {upd_C:.3e}), 残差 {res:.3e}"
147             f" (T {res_T:.3e} / C {res_C:.3e}) "
148         )
149     prev_c = upd
150     upd_c = upd
151
152     raise NonlinearFailure(

```

```

153         f"块 Picard 达到最大迭代次数 {max_iter}: 联合更新量 {upd_c:.3e}, 残差 {res:.3e}"
154     )
155
156
157     # =====
158     # 耦合强度诊断 (供论文与 CODE-25 使用)
159     # =====
160     def coupling_numbers(grid, D_cell, alpha_cell, C_cell, T_cell, dt,
161                          h_m, h_bc_T, C_inf, T_inf):
162         """
163         量化"这一步里耦合到底有多强", 输出三个可报告的指标:
164
165         * ``max_dD_dT_rel`` : 一步内由  $\Delta T$  引起的  $D$  的相对变化
166                                $= |\ln D / T| \cdot \Delta T, \ln D / T = 3850 / T_K^2$ 
167           (附录3 的  $D$  对  $T$  呈 Arrhenius 型,  $3850 / T_K^2$  在 300—330 K 上约为
168             $4.3e-2 \sim 3.5e-2 \text{ K}^{-1}$ , 即每升高 1 K 扩散系数增大大约 3.5%~4.3%。)
169
170         * ``Bi_m`` :  $h_m \cdot R_0 / D$  —— 传质 Biot 数, 判断内/外阻力谁主导
171         * ``Lu`` :  $D /$  —— Luikov 数, 传质与传热时间尺度之比
172         """
173         T_K = np.asarray(T_cell, dtype=float) + 273.15
174         dlnD_dT = 3850.0 / T_K ** 2
175         return {
176             "dlnD_dT_per_K": dlnD_dT,
177             "Bi_m": h_m * grid.R0 / np.maximum(D_cell, 1e-300),
178             "Lu": D_cell / np.asarray(alpha_cell, dtype=float),
179         }

```

## B.7 边界条件 (M0 + M1 潜热)

boundary.py (271 行)

```

1  """
2  CODE-13 边界条件模块                                [甲主 / 乙审 • M0+M1 • Day2上午]
3
4  职责
5  ----
6  把"表面到底以什么条件与烘房交换热与水"集中到一个模块**,
7  避免热方程与水分方程各自写一份边界、各自约定符号。
8
9  M0 形式 (题设基线, **默认**)
10 -----
11      r = R0 :  $-k \ T / r|_R = h \ (T_s - T_\infty)$ 
12                $-D \ C / r|_R = h_m \ (C_s - C_\infty)$ 
13      h   = 25 W/(m2 · K)   [Q-PDF 附录2]
14      h_m = 8×10 m/s        [Q-PDF 附录2]
15
16  M1 扩展形式 (**默认关闭**, `latent=True` 才启用)
17 -----
18       $-k \ T \cdot n|_R = h \ (T_s - T_\infty) + (T_s) J_w - q_{\text{rad},in}$  [R3-F1]
19
20  三个必须写清的量
21  ~~~~~
22  1. **J_w 是向外的水质量通量, 单位 kg/(m2 · s)**
23     **不得**把  $h_m(C_s - C_\infty)$  直接乘潜热当热通量 —— 它的单位是 kg/(kg · s),
24     不是 kg/(m2 · s)。必须乘一个密度基准:
25
26     J_w =  $\rho_d \cdot h_m \ (C_s - C_\infty)$                                 [OWN-RHOD]

```

\_d 的选取见下。

## 2. \*\*\_d 与水分方程的自治性\*\* (这是"通量闭合"的核心)

$C/t = (1/r) \cdot r(D \cdot r \cdot C)$  中的  $C$  是\*\*干基含水率\*\* (kg 水 / kg 干物质),  $D$  的单位是  $m^2/s$ 。该方程量纲自治的前提, 正是把  $C$  视为

"干物质基准的质量分数"→ 换算成单位体积的水量需乘\*\*干物质体积密度 \_d\*\*。

故  $J_w = \_d \cdot (-D \cdot C/r)|_R$  与扩散方程描述的是\*\*同一个通量\*\*, 不存在重复计数。本模块进一步保证:

\*\*用于热边界的水通量, 与水分方程实际算出的边界通量是同一个离散量\*\*

(见 `moisture\_flux`, 与 `fvm_cyl` 的 Robin 重构完全一致),

而\*\*不是\*\*另写一个连续表达式。

\_d 取值声明: 附录3 的  $C=650+128C$  与"干物质守恒"不兼容 ——

$C/(1+C)$  在  $C=0$  给  $650 \text{ kg/m}^3$ 、在  $C=2.55$  给  $275.0 \text{ kg/m}^3$ , \*\*不是常数\*\*。

这正是 CODE-35 (乙) 几何—密度一致性诊断要处理的矛盾。

本模块取\*\*初始状态基准\*\*并在结果中显式声明:

$$\_d = (C)/(1+C) = 976.4/3.55 = 275.04 \text{ kg/m}^3$$

同时提供  $\pm 50\%$  情景扰动接口 (CODE-25), \*\*不假装该值无争议\*\*。

## 3. \*\*蒸发位置\*\*: 本模块把全部潜热放在\*\*表面\*\* (因为 $C$ 的边界条件

本身就是表面换热式), 因此\*\*不再布置体源\*\*。

若将来改在内部加体源, \*\*必须\*\*相应扣掉表面这一项, 二者不可并存。

(R3-F2 指出  $T < 60^\circ\text{C}$  时气相渗流占比  $< 1.5\%$ , 故"整体汽化带"在本工况不重要。)

辐射项 `q_rad,in`

默认 0。R3-F4 给出  $h_{\text{rad}} \quad 6.5 \text{ W/(m}^2 \cdot \text{K)}$ 、 $N_{\text{rc}} \quad 0.26$  ([R3-F4]), 但该条已标 \*\*FLAGGED\*\*: 研报自述  $h=25$  可能已隐含吸收辐射, 论文池亦警告其"属待验证估算"。故\*\*辐射并不入 MO/M1 基线\*\*, 只作论文中的量级讨论。

```
from __future__ import annotations
```

```
from collections import namedtuple
```

```
from dataclasses import dataclass
```

```
import numpy as np
```

```
from ..config import C_INIT, H_COEF, HM_COEF
```

```
from ..numerics.fvm_cyl import Grid, _bi_delta, face_diffusivity
```

```
from ..numerics.properties import rho_app3
```

```
# =====
```

```
# 干物质体积密度的初始状态基准 [OWN-RHOD]
```

```
# =====
```

```
#  $\_d = (C)/(1+C) = (650 + 128 \times 2.55)/3.55 = 976.4/3.55 = 275.0422535 \dots \text{ kg/m}^3$ 
```

```
# 用表达式而不是手抄小数, 避免四舍五入引入不可追溯的偏差。
```

```
RHO_D_INIT = float(rho_app3(C_INIT) / (1.0 + C_INIT))
```

```
# =====
```

```
# 配置
```

```
# =====
```

```
@dataclass
```

```
class BoundaryConfig:
```

```

86 """
87 边界参数集合。默认即 M0 题设基线。
88 """
89 h: float = H_COEF          #  $W/(m^2 \cdot K)$  对流换热 [Q-PDF]
90 h_m: float = HM_COEF       #  $m/s$  对流传质 [Q-PDF]
91
92 # ---- M1 扩展开关 (默认全关) ----
93 latent: bool = False       # 是否启用蒸发潜热项
94 lambda_vap: float = 2.26e6 #  $J/kg$  水的汽化潜热 [R3-F1]
95 rho_d: float = RHO_D_INIT  #  $kg/m^3$  干物质体积密度 [OWN-RHOD]
96 q_rad_in: float = 0.0      #  $W/m^2$  入射净辐射 (默认 0, [R3-F4] FLAGGED)
97
98 def layer(self) -> str:
99     return "M1" if self.latent else "M0"
100
101 def describe(self) -> dict:
102     d = {"层次": self.layer(), "h / ( $W/(m^2 \cdot K)$ )": self.h,
103          "h_m / ( $m/s$ )": self.h_m, "潜热": "启用" if self.latent else "关闭"}
104     if self.latent:
105         d.update({" / ( $J/kg$ )": self.lambda_vap,
106                  " _d / ( $kg/m^3$ )": self.rho_d,
107                  "q_rad_in / ( $W/m^2$ )": self.q_rad_in})
108     return d
109
110
111 # =====
112 # 离散水通量 —— 与水分方程**同一个**量
113 # =====
114 # 用具名元组而不是裸 tuple: 这里曾出过一次真实事故 ——
115 # 调用处写成 `_, J_w = water_mass_flux(...)` 把**表面含水率**当成水通量
116 # 送进潜热项, 等效环境温度被压到  $-2.3 \times 10^4 K$ , Picard 直接振荡不收敛。
117 # 具名返回让这类顺序错误在代码里根本写不出来。
118 MoistureFlux = namedtuple("MoistureFlux", ["J_C", "C_s"])
119 WaterFlux = namedtuple("WaterFlux", ["J_w", "C_s"])
120
121
122 def moisture_surface_flux(C_cells, D_cell, C_inf_eq, grid: Grid,
123                          bc: BoundaryConfig) -> MoistureFlux:
124     """
125     由水分方程**实际使用的** Robin 离散重构出表面向外的水通量 (干基口径)。
126
127      $J_C = h_m (C_N - C_\infty) / (1 + Bi_\Delta)$ ,  $Bi_\Delta = h_m \cdot d / D_N$ 
128
129     单位:  $kg/(kg \cdot s)$ 。这是 fvm_cyl.surface_flux 的同一表达式,
130     在此重写一份是为了**显式暴露它对边界的依赖** 便于乙审计。
131     """
132     C = np.asarray(C_cells, dtype=float)
133     D_N = float(np.asarray(D_cell, dtype=float)[-1])
134     Bi_d = _bi_delta(D_N, bc.h_m, grid)
135     if not np.isfinite(Bi_d):
136         return MoistureFlux(0.0, float(C_inf_eq))
137     J_C = bc.h_m * (C[-1] - C_inf_eq) / (1.0 + Bi_d)
138     C_s = (C[-1] + Bi_d * C_inf_eq) / (1.0 + Bi_d)
139     return MoistureFlux(float(J_C), float(C_s))
140
141
142 def water_mass_flux(C_cells, D_cell, C_inf_eq, grid: Grid,
143                    bc: BoundaryConfig) -> WaterFlux:
144     """

```



向外的水质量通量  $J_w$ , 单位  $\text{kg}/(\text{m}^2 \cdot \text{s})$  (潜热项唯一合法的输入量)。

```
J_w = _d * J_C
```

直接对  $h_m(C_s - C_w)$  乘 是错的 —— 它比  $\text{kg}/(\text{m}^2 \cdot \text{s})$  少一个  $\text{kg}/\text{m}^3$ 。

```
"""
```

```
mf = moisture_surface_flux(C_cells, D_cell, C_inf_eq, grid, bc)
```

```
return WaterFlux(bc.rho_d * mf.J_C, mf.C_s)
```

```
# =====
```

```
# 热边界
```

```
# =====
```

```
def heat_ambient(T_inf, J_w, bc: BoundaryConfig):
```

```
    """
```

把  $M1$  的表面潜热与辐射折算成\*\*等效环境温度\*\*，使热边界重写为标准 Robin 形式：

```
-k T/r|_R = h(T_s - T_w) + J_w - q_rad,in
h(T_s - T_w)
```

推导 (注意符号, 容易写反):

```
h(T_s - T_w) + J_w - q_rad = h(T_s - T_w)
-h T_w + J_w - q_rad = -h T_w
T_w = T_w - (J_w - q_rad,in)/h
```

这一改写的好处: 热方程在每个 Picard 迭代内\*\*仍是线性 Robin 问题\*\*，线性求解器、M-矩阵性质、表面重构全部原样复用，无需改动 `fvm_cyl`。

物理含义: 蒸发 ( $J_w > 0$ ) \*\*压低\*\*等效环境温度 (本例中量级可达数十 K)  
→ 表面被强烈冷却, 这正是恒速干燥期"物料温度钳位于湿球温度"的机制  
[R3-F1]; 入射净辐射 ( $q_{rad,in} > 0$ ) 则\*\*抬高\*\*等效环境温度。

返回 ( $T_{inf\_eff}$ , 折算温降  $dT_{evap}$ ),  $dT_{evap} = (J_w - q_{rad})/h$ 。

```
"""
```

```
if not bc.latent:
```

```
    return float(T_inf), 0.0
```

```
# ---- 量级护栏 ----
```

```
# 物理上 J_w _d * h_m * C_max 275×8e-7×2.55 = 5.6e-4 kg/(m² · s),
```

```
# 故 dT_evap 2.26e6×5.6e-4/25 51 K。超出数百 K 必是量纲/取值错误
```

```
# (例如把表面含水率当成水通量传了进来)。
```

```
dT = (bc.lambda_vap * J_w - bc.q_rad_in) / bc.h
```

```
if abs(dT) > 500.0:
```

```
    raise ValueError(
```

```
        f"蒸发折算温降 {dT:.4g} K 超出物理量级上限 (约 51 K)。"
```

```
        f"检查 J_w 是否确为 kg/(m² · s) 的**质量通量**"
```

```
        f"(当前 J_w={J_w:.4g}, _d={bc.rho_d:.4g}, ={bc.lambda_vap:.3g})。"
```

```
        f"常见错误: 把表面含水率 C_s 当作通量传入。"
```

```
)
```

```
return float(T_inf - dT), float(dT)
```

```
def net_surface_heat_flux(T_cells, k_cell, T_inf, J_w, grid: Grid, bc: BoundaryConfig):
```

```
    """
```

表面\*\*向外\*\*的净热通量 (含潜热), 单位  $\text{W}/\text{m}^2$ 。

```
q_out = h(T_s - T_w) + J_w - q_rad,in
```

这是用于\*\*收支检验与报告\*\*的量, 不参与求解 (求解走 `heat_ambient` 的等效温度)。

```
"""
```

```

204 from ..numerics.fvm_cyl import surface_flux
205 q_conv = surface_flux(T_cells, k_cell, bc.h, T_inf, grid) # 这里 k 为导热系数、h 为  $W/(m^2 \cdot K)$ 
206 q_lat = bc.lambda_vap * J_w if bc.latent else 0.0
207 return float(q_conv + q_lat - bc.q_rad_in), float(q_conv), float(q_lat)
208
209
210 # =====
211 # 自检: 纯导热退化
212 # =====
213 def check_pure_conduction(grid: Grid, bc: BoundaryConfig | None = None) -> dict:
214     """
215     验收要求: **纯导热无蒸发时退化为标准 Robin 条件**.
216
217     做法: 构造一个已知线性剖面  $T(r) = T_w + G \cdot (R_0 - r)$  (不是本问题的解,
218     但满足  $T/r = -G$  常数), 检查
219         latent=False 时 heat_ambient 返回  $T_w$  本身 (不引入任何附加项)
220         离散 surface_flux 与解析  $-k T/r = kG$  在细网格下一致
221     """
222     bc = bc or BoundaryConfig()
223     rng = np.random.default_rng(0)
224     T_inf = 50.0
225     k = 0.483
226     rep = {}
227
228     # M0 不改变环境温度
229     T_eff, dT = heat_ambient(T_inf, 1e-3, bc)
230     rep["M0_不引入附加项"] = (abs(T_eff - T_inf) < 1e-15) and (dT == 0.0)
231
232     # 均匀导热系数下, 表面的 Robin 重构应给出  $T_s$  落在  $[T_w, T_N]$  内
233     # (此处构造"内热外冷", 热流向外, 故  $T_w > T_s > T_N$ )
234     T_cells = np.linspace(T_inf + 2.0, T_inf + 0.1, grid.N)
235     k_cell = np.full(grid.N, k)
236     from ..numerics.fvm_cyl import surface_value, surface_flux
237     Ts = surface_value(T_cells, k_cell, bc.h, T_inf, grid)
238     J = surface_flux(T_cells, k_cell, bc.h, T_inf, grid)
239     rep["表面值落在合理区间"] = bool(T_inf - 1e-12 <= Ts <= T_cells[-1] + 1e-12)
240     rep["向外通量为正"] = bool(J > 0.0)
241     rep["表面值与Robin一致"] = bool(abs(J - bc.h * (Ts - T_inf)) < 1e-9 * max(1.0, abs(J)))
242
243     # 潜热开关确实改变等效环境温度, 且方向为**降温**
244     bc1 = BoundaryConfig(latent=True)
245     T_eff1, dT1 = heat_ambient(T_inf, 1e-3, bc1)
246     rep["M1_等效环境被压低"] = bool(T_eff1 < T_inf and dT1 > 0.0)
247     rep["M1_折算量级正确"] = bool(
248         abs(dT1 - bc1.lambda_vap * 1e-3 / bc1.h) < 1e-12
249         and abs((T_inf - T_eff1) - dT1) < 1e-12)
250     # b 辐射方向相反: 入射净辐射应抬高等效环境温度
251     bc2 = BoundaryConfig(latent=True, q_rad_in=200.0)
252     T_eff2, _ = heat_ambient(T_inf, 1e-3, bc2)
253     rep["M1_辐射方向相反"] = bool(T_eff2 > T_eff1)
254
255     # _d 基准与初始状态自治
256     rho0, C0 = 650.0 + 128.0 * 2.55, 2.55
257     rep["_d_初始基准自治"] = bool(abs(bc1.rho_d - rho0 / (1.0 + C0)) < 1e-12)
258
259     rep["全部通过"] = all(v for k_, v in rep.items() if isinstance(v, bool))
260     return rep
261
262

```

```

263 if __name__ == "__main__":
264     g = Grid(N=200, R0=0.02, grading=1.5)
265     rep = check_pure_conduction(g)
266     print("[CODE-13] 边界模块自检: ")
267     for k_, v in rep.items():
268         print(f"  {'OK ' if v else 'FAIL'} {k_}")
269     print()
270     for k_, v in BoundaryConfig(latent=True).describe().items():
271         print(f"  {k_:16s} {v}")

```

## B.8 环境插值（附件 1）

env\_interp.py (159 行)

```

1  """
2  CODE-02 附件1 环境数据插值          [丙 • M0 • Day1上午]
3
4  实测事实（已核实，非转述）
5  -----
6      附件1: 241 行 × 3 列（时间 / 温度 / 水分浓度）
7      时间 0 → 14400 s, **步长恒为 60 s**, 严格单调递增
8      温度 28.0000 → 50.1650 °C (min 28.0000, max 50.2460)
9      水分 0.019630 → 0.049860 kg/kg (min 0.019630, max 0.050250)
10     温度差分**变号 116 次**、水分浓度变号 **119 次** —— **波动遍布全程，不只尾部**
11
12     两个版本（v3 清单 CODE-02 要求）
13     -----
14     faithful : PCHIP 保形插值，**保留有效测量**。实测波动是真实边界条件的一部分，
15               **不天然属于噪声**，故此为默认版本。
16     smoothed : 受控平滑（Savitzky-Golay），仅作**对照**，
17               并须在论文中说明平滑依据（传感器精度 / 时间序列特征）。
18
19     禁止事项
20     -----
21     * 不得推断控制器类型（不能说"这是 PD 超调"）
22     * 不得据此推断烘房结构
23     * 禁止高阶多项式（会产生不合理振荡）
24
25     变量基准
26     -----
27     附件1 的"水分浓度"列在此按**题设等效约定**读入，记作 C_inf_eq
28     （等效环境平衡含水率），**不是**由真实空气状态求出的吸附平衡结果。
29     """
30
31     from __future__ import annotations
32
33     from dataclasses import dataclass
34     from pathlib import Path
35
36     import numpy as np
37     import pandas as pd
38     from scipy.interpolate import PchipInterpolator
39     from scipy.signal import savgol_filter
40
41     from ..config import ATTACH_DIR
42
43
44     # =====

```

```

45 # 读取
46 # =====
47 @dataclass
48 class EnvData:
49     t: np.ndarray      # s
50     T: np.ndarray      # °C
51     C: np.ndarray      # kg/kg
52     source: str = ""
53
54     def __post_init__(self):
55         assert np.all(np.diff(self.t) > 0), "时间列必须严格递增"
56         assert len(self.t) == len(self.T) == len(self.C), "列长度不一致"
57
58     @property
59     def t_span(self):
60         return float(self.t[0]), float(self.t[-1])
61
62     def summary(self) -> str:
63         return (
64             f"附件1: {len(self.t)} 点, t   [{self.t[0]:.0f}, {self.t[-1]:.0f}] s, "
65             f"步长 {self.t[1]-self.t[0]:.0f} s\n"
66             f" 温度 {self.T.min():.4f} → {self.T.max():.4f} °C\n"
67             f" 水分 {self.C.min():.6f} → {self.C.max():.6f} kg/kg\n"
68             f" 温度差分变号 {(np.diff(np.sign(np.diff(self.T))) != 0).sum()} 次, "
69             f"水分差分变号 {(np.diff(np.sign(np.diff(self.C))) != 0).sum()} 次"
70         )
71
72
73 def load_env(path: Path | None = None) -> EnvData:
74     p = Path(path) if path else (ATTACH_DIR / "附件1.xlsx")
75     df = pd.read_excel(p)
76     cols = {c.strip(): c for c in df.columns}
77     return EnvData(
78         t=df[cols["时间"]].to_numpy(dtype=float),
79         T=df[cols["温度"]].to_numpy(dtype=float),
80         C=df[cols["水分浓度"]].to_numpy(dtype=float),
81         source=str(p),
82     )
83
84
85 # =====
86 # 插值器
87 # =====
88 class EnvInterpolator:
89     """
90     环境边界函数封装。
91
92     两个版本通过 `mode` 选择:
93     "faithful" —— PCHIP, 过所有原始点 (默认)
94     "smoothed" —— 先 Savitzky-Golay 平滑再做 PCHIP (**仅作对照**)
95     """
96
97     def __init__(self, data: EnvData, mode: str = "faithful",
98                 smooth_window: int = 21, smooth_poly: int = 2,
99                 smooth_fraction: float = 0.0):
100         assert mode in ("faithful", "smoothed")
101         self.mode = mode
102         self.data = data
103         T, C = data.T.copy(), data.C.copy()

```

```

104     if mode == "smoothed":
105         w = smooth_window if smooth_window % 2 == 1 else smooth_window + 1
106         w = min(w, len(T) - (1 - len(T) % 2))
107         poly = min(smooth_poly, w - 1)
108         T = savgol_filter(T, w, poly)
109         C = savgol_filter(C, w, poly)
110         self.smooth_window = w
111     self._T = PchipInterpolator(data.t, T, extrapolate=False)
112     self._C = PchipInterpolator(data.t, C, extrapolate=False)
113     self.t_min, self.t_max = data.t[0], data.t[-1]
114
115     # -----
116     def T_inf(self, t):
117         v = self._T(t)
118         if np.any(~np.isfinite(v)):
119             raise ValueError(
120                 f"T_inf({t}) 超出附件1 覆盖范围 [{self.t_min:.0f}, {self.t_max:.0f}] s。"
121                 f"恒温段外推方案须显式声明（见框架 §5.3）。"
122             )
123         return v
124
125     def C_inf(self, t):
126         v = self._C(t)
127         if np.any(~np.isfinite(v)):
128             raise ValueError(
129                 f"C_inf({t}) 超出附件1 覆盖范围 [{self.t_min:.0f}, {self.t_max:.0f}] s。"
130             )
131         return v
132
133     # -----
134     def plateau_stats(self, t_from: float = 9600.0) -> dict:
135         """平台段统计（用于恒温干燥段边界设定，框架 §5.3 方案A）。"""
136         m = self.data.t >= t_from
137         return {
138             "t_from": t_from,
139             "n": int(m.sum()),
140             "T_mean": float(self.data.T[m].mean()),
141             "T_std": float(self.data.T[m].std()),
142             "C_mean": float(self.data.C[m].mean()),
143             "C_std": float(self.data.C[m].std()),
144         }
145
146     def __repr__(self):
147         return f"<EnvInterpolator mode={self.mode} t [{self.t_min:.0f},{self.t_max:.0f}]s>"
148
149
150 if __name__ == "__main__":
151     d = load_env()
152     print(d.summary())
153     for mode in ("faithful", "smoothed"):
154         it = EnvInterpolator(d, mode=mode)
155         print(f"\n{mode}: {it}")
156         for tt in (0, 300, 600, 900, 1200, 1500, 1800):
157             print(f"  t={tt:5d}s  Tw={float(it.T_inf(tt)):8.4f} °C "
158                   f"  Cw={float(it.C_inf(tt)):9.6f} kg/kg")
159     print("\n平台段统计:", EnvInterpolator(d).plateau_stats())

```

## B.9 长时环境外推（平台值）

env\_extrap.py (155 行)

```

1  """
2  长时环境边界外推（框架 §5.3 方案 A） [甲 • M0 • Day2上午]
3
4  问题
5  ----
6  附件1 只覆盖 0—14400 s (4 h)，而问题2/3 要跑完整个烘干过程（实测约 2—3 天）。
7  **恒温干燥段的边界必须显式假设，不能靠插值函数外推**。
8
9  实测依据（附件1, [Q-A1]）
10 -----
11     t   9600 s 段 (n = 81 点) : Tm 均值 50.0017 °C (std 0.0766)
12                                     Cm 均值 0.049998 kg/kg (std 3.4e-4)
13     该段已进入平台，波动量级即为传感器/记录噪声。
14
15  三个候选方案与取舍（框架 §5.3）
16 -----
17     A（采用）  t   t_pre 用 PCHIP 过实测点； t > t_pre 取常数平台值
18                 —— 简单、可辩护，**不假装能预测烘房**
19     B           拟合一阶饱和模型后外推 —— 对波动敏感、外推无依据
20     C           整体双指数拟合 —— 易过拟合波动，**不推荐**
21
22     为什么 t_pre 处的小阶跃可以接受
23 -----
24     取 t_pre = 14400 s（用满全部实测数据）时，末点值 50.1650 °C 与平台均值
25     50.0017 °C 相差 **0.163 °C**，**落在实测波动带 (±0.15 °C, std 0.0766) 之内**。
26     即该阶跃是"把噪声末点拉回其自身均值"，不是引入新信息。
27     本模块仍提供 `ramp` 参数（默认 0）用于检验该阶跃是否影响结论。
28
29     不得做的事
30 -----
31     * 不得推断控制器类型（不能说"这是 PD 超调"） [Q-A1 + 清单 §CODE-02]
32     * 不得据附件1 推断烘房结构
33     * **不得用任何未经声明的外推方式**
34
35 """
36
37 from __future__ import annotations
38
39 from dataclasses import dataclass
40
41 import numpy as np
42
43 from .env_interp import EnvData, EnvInterpolator
44
45 # 平台段统计的默认起点（框架 §5.3 实测值）
46 T_PLATEAU_FROM = 9600.0
47
48 @dataclass
49 class PlateauEnv:
50     """
51     方案 A 环境边界：分段 + 可选斜坡。
52
53     t   t_pre           : 交给底层 EnvInterpolator (PCHIP 过实测点)
54     t_pre < t < t_pre+ramp : 线性过渡 (ramp=0 时此段为空)
55     t   t_pre + ramp : 常数 T_plateau / C_plateau

```

参数

----

interp : EnvInterpolator (faithful 或 smoothed)  
t\_pre : 切换到平台值的时刻; 框架建议 9600 或 14400  
T\_plateau : 平台温度; None 时取 t T\_PLATEAU\_FROM 段的实测均值  
C\_plateau : 平台含水率; None 时同上  
ramp : 过渡段长度 (s)。0 = 纯方案 A (允许小阶跃),  
非零时先线性过渡到平台值再保持常数, 用于灵敏度检验。

"""

```
interp: EnvInterpolator
t_pre: float = 14400.0
T_plateau: float | None = None
C_plateau: float | None = None
ramp: float = 0.0
```

```
def __post_init__(self):
    st = self.interp.plateau_stats(T_PLATEAU_FROM)
    if self.T_plateau is None:
        self.T_plateau = st["T_mean"]
    if self.C_plateau is None:
        self.C_plateau = st["C_mean"]
    self._plateau_stats = st
    if self.t_pre <= 0.0 or self.t_pre > self.interp.t_max:
        raise ValueError(f"t_pre={self.t_pre} 超出附件1 覆盖范围 "
                          f"(0, {self.interp.t_max}]")
    # 记录阶跃大小, 供论文如实报告
    self.jump_T = float(self.T_plateau - self.interp.T_inf(self.t_pre))
    self.jump_C = float(self.C_plateau - self.interp.C_inf(self.t_pre))

# -----
def _eval(self, t, fn, plateau):
    t = np.asarray(t, dtype=float)
    scalar = (t.ndim == 0)
    t = np.atleast_1d(t)
    out = np.empty_like(t)

    m_data = t <= self.t_pre
    if np.any(m_data):
        out[m_data] = fn(t[m_data])

    if self.ramp > 0.0:
        t_a, t_b = self.t_pre, self.t_pre + self.ramp
        m_ramp = (t > t_a) & (t < t_b)
        if np.any(m_ramp):
            w = (t[m_ramp] - t_a) / self.ramp
            out[m_ramp] = (1.0 - w) * fn(np.full(w.shape, t_a)) + w * plateau
        m_pl = t >= t_b
    else:
        m_pl = t > self.t_pre

    if np.any(m_pl):
        out[m_pl] = plateau
    return float(out[0]) if scalar else out

def T_inf(self, t):
    return self._eval(t, self.interp.T_inf, self.T_plateau)
```



```

115 def C_inf(self, t):
116     return self._eval(t, self.interp.C_inf, self.C_plateau)
117
118 # -----
119 def describe(self) -> dict:
120     return {
121         "t_pre / s": self.t_pre,
122         "ramp / s": self.ramp,
123         "T0 平台 / °C": round(float(self.T_plateau), 4),
124         "C0 平台 / (kg/kg)": round(float(self.C_plateau), 6),
125         "t_pre 处温度阶跃 / °C": round(self.jump_T, 4),
126         "t_pre 处含水率阶跃 / (kg/kg)": round(self.jump_C, 6),
127         "平台段样本数": self._plateau_stats["n"],
128         "平台段 T0 std / °C": round(self._plateau_stats["T_std"], 4),
129         "平台段 C0 std / (kg/kg)": round(self._plateau_stats["C_std"], 6),
130     }
131
132
133 # =====
134 def build_env(data: EnvData, mode: str = "faithful", t_pre: float = 14400.0,
135             ramp: float = 0.0, **kw) -> PlateauEnv:
136     """一步构建: EnvData → EnvInterpolator → PlateauEnv. """
137     return PlateauEnv(EnvInterpolator(data, mode=mode), t_pre=t_pre,
138                     ramp=ramp, **kw)
139
140
141 if __name__ == "__main__":
142     from .env_interp import load_env
143     import sys
144     sys.path.insert(0, str(__import__("pathlib").Path(__file__).resolve().parents[2]))
145     d = load_env()
146     print(d.summary(), "\n")
147     for mode in ("faithful", "smoothed"):
148         for t_pre in (9600.0, 14400.0):
149             env = build_env(d, mode=mode, t_pre=t_pre)
150             print(f"--- mode={mode} t_pre={t_pre:.0f} ---")
151             for k, v in env.describe().items():
152                 print(f"    {k:30s} {v}")
153             print(f"    T0(172800 s) = {env.T_inf(172800.0):.4f} °C ")
154             print(f"    C0(172800 s) = {env.C_inf(172800.0):.6f} kg/kg")
155             print()

```

## B.10 问题 1 求解

problem1.py (146 行)

```

1 """
2 CODE-10 / CODE-11 问题1: 温度场与水分场 [甲 • M0 • Day1晚间]
3
4 模型 (附录2 常物性; M0 **不含蒸发潜热项**)
5 -----
6     cp T/t = (1/r) _r( k r T/r )      k = 0.36 W/(m·K)
7     C/t   = (1/r) _r( D(C) r C/r )    D = 7e-9 · exp(-0.89/C)
8     初值 T = 28 °C, C = 2.55 kg/kg
9     r = 0 : T/r = C/r = 0
10    r = R0: -k T/r = h(Ts - T0(t)), -D C/r = hm(Cs - C0q(t))
11           h = 25 W/(m² · K), hm = 8e-7 m/s
12

```

结构性洞察：单向耦合

附录2 的  $D(C)$  \*\*不依赖  $T$ \*\*  $\rightarrow$  水分方程完全独立于温度场  
 温度方程的  $M_0$  形式不含蒸发项  $\rightarrow$  温度方程也完全独立  
 故: \*\*先解水分场, 再解温度场\*\* (两者可用同一步长共同推进, 互不影响)

这是问题1 相比问题2 的最大计算简化。问题2 中  $D$  依赖  $T$ ,  
 必须联立求解, 该简化不再成立。

数值

```

----
* 空间: 半控制体 FVM (CODE-06),  $\Delta r = R_0/N$ 
* 时间: -方法, 生产用 Backward Euler (=1), L-稳定
* 水分: Picard 迭代 (CODE-09), M-矩阵保证 C 正性
* 温度: 常系数算子**只装配一次**, 每步仅更新右端项 ( $T_m$  随时间变)
"""

from __future__ import annotations
from dataclasses import dataclass, field
import numpy as np

from ..config import H_COEF, HM_COEF, PROPS_APP2, NUMERICS, to_kelvin
from ..numerics.fvm_cyl import Grid, Operator, assemble, implicit_step, surface_value,
    surface_flux
from ..numerics.nonlinear import picard_solve
from ..numerics.properties import D_app2
from ..numerics.time_stepper import AdaptiveStepper, IntegrateResult, explicit_stability_limit

# =====
@dataclass
class Problem1Setup:
    grid: Grid
    env: object # EnvInterpolator
    theta: float = 1.0
    T_init: float = 28.0
    C_init: float = 2.55
    t_end: float = 1800.0
    use_latent_heat: bool = False # M0 关闭; M1 扩展见 CODE-36

# =====
def _tol_dict():
    n = NUMERICS
    return dict(atol_u=n.picard_atol_u, rtol_u=n.picard_rtol_u,
                atol_r=n.picard_atol_r, rtol_r=n.picard_rtol_r)

def solve_problem1(setup: Problem1Setup, t_output=None, p_order=None):
    """
    求解问题1。返回 (IntegrateResult, extra)。

    extra 含: 表面温度/含水率/通量历史、算子诊断、稳定性自检结果。
    """
    g, env = setup.grid, setup.env
    num = NUMERICS
    theta = setup.theta
  
```

```

71 if p_order is None:
72     p_order = 1.0 if theta == 1.0 else 2.0
73
74 # ---- 显式稳定性自检：不满足即拒绝该设置（而非等待发散） ----
75 alpha = PROPS_APP2.alpha
76 dt_exp = explicit_stability_limit(g.dr, alpha)
77 assert dt_exp > num.dt_min, "显式稳定性上限低于步长下限，配置不合理"
78
79 # ---- 温度：常系数算子只装配一次 ----
80 alpha_cell = np.full(g.N, alpha)
81 h_bc_T = H_COEF / (PROPS_APP2.rho * PROPS_APP2.cp) # m/s
82 op_T_template = assemble(alpha_cell, h_bc_T, 0.0, g) # b 稍后按 Tw 更新
83
84 # ---- 状态 ----
85 T0 = np.full(g.N, setup.T_init)
86 C0 = np.full(g.N, setup.C_init)
87
88 surf_T, surf_C, flux_w = [], [], []
89 tol = _tol_dict()
90
91 def step_fn(t, dt, state):
92     t_new = t + dt
93
94     # ===== 水分场（先解；非线性 Picard） =====
95     C_inf_new = float(env.C_inf(t_new))
96
97     def assemble_C(C):
98         D_cell = np.atleast_1d(D_app2(C)).astype(float)
99         return assemble(D_cell, HM_COEF, C_inf_new, g)
100
101     C_new, iters, _res = picard_solve(
102         assemble_C, state["C"], dt, theta, tol, num.picard_max_iter,
103     )
104
105     # ===== 温度场（后解；线性，单次求解） =====
106     T_inf_new = float(env.T_inf(t_new))
107     op_T = Operator(N=g.N, diag=op_T_template.diag.copy(),
108                    lower=op_T_template.lower.copy(),
109                    upper=op_T_template.upper.copy(),
110                    b=op_T_template.b.copy(), beta=op_T_template.beta,
111                    Bi_delta=op_T_template.Bi_delta,
112                    Gamma_face=op_T_template.Gamma_face)
113
114     # * Tw 更新右端项
115     beta_T = op_T.beta
116     op_T.b[:-1] = 0.0
117     op_T.b[-1] = beta_T * T_inf_new
118     T_new = implicit_step(op_T, state["T"], dt, theta=theta)
119
120     return {"T": T_new, "C": C_new}, iters
121
122 stepper = AdaptiveStepper(num)
123 res = stepper.integrate(0.0, {"T": T0, "C": C0}, setup.t_end, t_output, step_fn,
124                        p_order=p_order)
125
126 # ---- 表面量（在输出时刻上重构，供表1/表2的 r=2.0 cm 列使用） ----
127 for k in range(len(res.times)):
128     tk = float(res.times[k])
129     Tk, Ck = res.T[k], res.C[k]
130     Tinf = float(env.T_inf(tk))

```

```

130     Cinf = float(env.C_inf(tk))
131     surf_T.append(surface_value(Tk, alpha_cell, h_bc_T, Tinf, g))
132     Dk = np.atleast_1d(D_app2(Ck)).astype(float)
133     surf_C.append(surface_value(Ck, Dk, HM_COEF, Cinf, g))
134     flux_w.append(surface_flux(Ck, Dk, HM_COEF, Cinf, g))
135
136     extra = {
137         "surface_T": np.asarray(surf_T),
138         "surface_C": np.asarray(surf_C),
139         "surface_flux_w": np.asarray(flux_w),
140         "alpha": alpha,
141         "explicit_dt_limit": dt_exp,
142         "h_bc_T": h_bc_T,
143         "p_order": p_order,
144         "theta": theta,
145     }
146     return res, extra

```

## B.11 问题 2 求解（全程变物性）

problem2.py (340 行)

```

1  """
2  CODE-12 问题2: 变参数热湿双向耦合（全流程） [甲 • MO/M1 • Day2上午]
3
4  题面要求（[Q-PDF] 问题2）
5  -----
6  "烘干过程一般持续 2—3 天，预热平衡与恒温干燥阶段的参数有所不同。
7  请建立整个烘干过程药材温度和水分浓度变化规律的数学模型
8  （**为简化问题，相关经验公式统一采用附录 3 中的公式**）"
9
10  关键执行点（清单 SCODE-12）
11  -----
12  1. **从 t=0 统一采用附录3**，**不允许**先跑问题1 再在某时刻切换参数。
13     题面已明确"统一采用附录3"，故不存在"阶段切换"，
14     也就不需要"切换处保持状态连续"的处理 —— 这一点必须写进论文，
15     否则会被误认为我们漏掉了阶段切换。
16  2. **变系数不得移出微分算子**：(C)c_p(C) T/t 走守恒形式，
17     经 fvm_cyl.assemble 的 `cap_cell` 参数实现（面导热系数用 k 的调和平均，
18     热容用单元值，二者不可合并 —— 见该函数文档）。
19  3. T 与 C **联立隐式求解**（D 显含 T，问题1 的"先水后温"不再成立）。
20
21  控制方程（M0）
22  -----
23      (C)c_p(C) T/t = (1/r) _r( k(C) r T/r )
24      C/t           = (1/r) _r( D(C,T) r C/r )
25
26      = 650 + 128C, c_p = 1450 + 2736C/(1+C), k = 0.21 + 0.38C/(1+C)
27      D = 2.4e-3 * exp(-0.45/C) * exp(-3850/T_K) [Q-PDF 附录3]
28
29  初值 / 边界
30  -----
31      T(r,0) = 28 °C, C(r,0) = 2.55 kg/kg
32      r = 0 : T/r = C/r = 0
33      r = R0: M0 -k T/r = h(T_s - T_w), -D C/r = h_m(C_s - C_w )
34              M1 -k T/r = h(T_s - T_w) + J_w (见 models/boundary.py)
35
36  长时边界: 附件1 只到 14400 s, 恒温段按框架 §5.3 方案 A 取常数平台值

```

```

37 (见 data_prep/env_extrap.py)。**该假设本身写入论文**。
38 """
39
40 from __future__ import annotations
41
42 from dataclasses import dataclass, field
43
44 import numpy as np
45
46 from ..config import (C_INIT, H_COEF, HM_COEF, NUMERICS, R_OUT_CM, T_INIT,
47                       to_kelvin)
48 from ..numerics.coupled import block_picard_solve
49 from ..numerics.fvm_cyl import (Grid, assemble, surface_flux, surface_value)
50 from ..numerics.properties import D_app3, cp_app3, k_app3, rho_app3
51 from ..numerics.time_stepper import AdaptiveStepper, explicit_stability_limit
52 from .boundary import (BoundaryConfig, heat_ambient, moisture_surface_flux,
53                       water_mass_flux)
54 from ..io.resample import center_value
55
56
57 # =====
58 @dataclass
59 class Problem2Setup:
60     grid: Grid
61     env: object # PlateauEnv
62     bc: BoundaryConfig = field(default_factory=BoundaryConfig)
63     theta: float = 1.0
64     T_init: float = T_INIT
65     C_init: float = C_INIT
66     t_end: float = 10800.0
67     # ---- 情景扰动开关 (CODE-25; 默认全为 1, 即题设基线) ----
68     x_D: float = 1.0 # D 的倍率
69     x_hm: float = 1.0 # h_m 的倍率
70     x_h: float = 1.0 # h 的倍率
71     label: str = "M0"
72     # ---- 起始层 (见 time_stepper.integrate 的说明) ----
73     # M1 的潜热项使 t=0 处边界条件阶跃 → 真解按  $\sqrt{t}$  演化 →
74     # 步长加倍法在跨 t=0 的首步失效 (局部误差  $O(\Delta t^{1/2})$  而非  $O(\Delta t^2)$ )。
75     # 故起始 warmup_steps 步强制等步长、不做误差控制。
76     # M0 无此阶跃, 保持 0。
77     warmup_steps: int = 0
78     warmup_dt: float = 1.0e-3
79
80     def eff_h_m(self) -> float:
81         return self.bc.h_m * self.x_hm
82
83     def eff_h(self) -> float:
84         return self.bc.h * self.x_h
85
86
87 # =====
88 def _tol_dict():
89     n = NUMERICS
90     return (dict(atol_u=n.picard_atol_u, rtol_u=n.picard_rtol_u,
91                 atol_r=n.picard_atol_r, rtol_r=n.picard_rtol_r),
92            dict(atol_u=n.picard_atol_u, rtol_u=n.picard_rtol_u,
93                 atol_r=n.picard_atol_r, rtol_r=n.picard_rtol_r))
94
95

```

```

96 # =====
97 def build_step_fn(setup: Problem2Setup, num=None, stats=None):
98     """
99     构造一个时间步函数 ``step_fn(t, dt, state) -> (state_new, n_iters)``。
100
101     独立成工厂函数的理由: **CODE-19 的定步长细化不能走自适应器**。  

102     把 dt_min 与 dt_max 都钉在 dt 会让自适应器在"拒绝→缩步"时  

103     永远缩不下去 (dt 已被钳在 dt_min), 形成无限拒绝循环。  

104     定步长必须绕开自适应控制, 直接循环调用本函数。  

105     事件二分 (CODE-15) 与情景扫描 (CODE-25) 也复用同一实现。
106
107     闭包内的 ``T_inf_new`` / ``C_inf_new`` 取自  $t+dt$  (隐式右端),  

108     这正是 Backward Euler 的要求, 不能取 t。
109     """
110     g = setup.grid
111     env = setup.env
112     bc = setup.bc
113     num = num if num is not None else NUMERICS
114     theta = setup.theta
115     h_m = setup.eff_h_m()
116     h_bc = setup.eff_h()
117     tol_T, tol_C = _tol_dict()
118     st = stats if stats is not None else {"iters": [], "last_iters": 0}
119
120     def step_fn(t, dt, state):
121         t_new = t + dt
122         T_inf_new = float(env.T_inf(t_new))
123         C_inf_new = float(env.C_inf(t_new))
124
125         def assemble_C(C_g, T_g):
126             D_cell = setup.x_D * np.atleast_1d(
127                 np.asarray(D_app3(C_g, T_g), dtype=float))
128             return assemble(D_cell, h_m, C_inf_new, g)
129
130         def assemble_T(T_g, C_g):
131             k_cell = np.atleast_1d(np.asarray(k_app3(C_g), dtype=float))
132             cap_cell = np.atleast_1d(np.asarray(
133                 rho_app3(C_g) * cp_app3(C_g), dtype=float))
134             # M1: 用**同一个**离散水通量折算等效环境温度 (避免重复计数)
135             if bc.latent:
136                 D_g = setup.x_D * np.atleast_1d(
137                     np.asarray(D_app3(C_g, T_g), dtype=float))
138                 J_w = water_mass_flux(C_g, D_g, C_inf_new, g, bc).J_w
139             else:
140                 J_w = 0.0
141             T_amb, _ = heat_ambient(T_inf_new, J_w, bc)
142             return assemble(k_cell, h_bc, T_amb, g, cap_cell=cap_cell)
143
144         T_new, C_new, it = block_picard_solve(
145             state["T"], state["C"], dt, theta,
146             assemble_T, assemble_C, tol_T, tol_C,
147             max_iter=num.picard_max_iter)
148
149         if not np.all(np.isfinite(T_new)):
150             raise RuntimeError(f"t={t_new:.6g} 温度出现非有限值")
151
152         st["last_iters"] = it.iters
153         st["iters"].append(it.iters)
154         return {"T": T_new, "C": C_new}, it.iters

```

```

155
156     return step_fn, st
157
158
159 def solve_problem2(setup: Problem2Setup, t_output=None, p_order=None,
160                   t0: float = 0.0, state0=None, collect_diag=True, num=None,
161                   stop_fn=None):
162     """
163     求解问题2。返回 (IntegrateResult, extra)。
164
165     参数
166     ----
167     t0, state0 : 起始时刻与起始状态 (T0, C0)。默认 (0, 初值)。
168                  **CODE-15 的事件二分定位**靠它从已存状态重启,
169                  避免为了试一个时刻而从 0 重算整段。
170     collect_diag : 是否在每个输出时刻重构表面量。
171                  二分试算时置 False 可省掉大部分开销。
172     num          : 数值参数覆盖 (默认全局 NUMERICS)。
173     stop_fn      : callable(t, state) -> bool, 每个接受步后询问, True 即停。
174                  **CODE-15 的阈值粗扫与 CODE-25 的情景扫描靠它早停** ——
175                  不早停就要跑满整个视界, 浪费 2—3 倍甚至 20 倍机时。
176
177     extra 含每个输出时刻的诊断量 (表面通量、等效环境温度、Bi_m、Lu 等),
178     供 G12、FIG-16、CODE-15、CODE-25 使用。
179     """
180     g = setup.grid
181     env = setup.env
182     bc = setup.bc
183     num = num if num is not None else NUMERICS
184     theta = setup.theta
185     if p_order is None:
186         p_order = 1.0 if theta == 1.0 else 2.0
187
188     h_m = setup.eff_h_m()
189     h_bc = setup.eff_h()
190
191     # ---- 显式稳定性自检 ----
192     # 用途是 "论证为什么必须用隐式", 口径须与 Day1 一致:
193     # 取**名义间距**  $\Delta r = R0/N$  (而非渐变网格的最小面距  $7\ \mu m$ ),
194     # 并把 取遍整个含水率区间的**最大值** (最不利情形)。
195     # 渐变网格的最小面距会让  $\Delta r^2/(4)$  降到  $\sim 1e-4\ s$ , 但这对
196     # **L-稳定的 Backward Euler 根本不是约束**, 若拿它做断言会得出
197     # 误导性结论。最小面距仅作信息量记录。
198     C_probe = np.linspace(0.05, setup.C_init, 256)
199     alpha_max = float(np.max(k_app3(C_probe) /
200                               (rho_app3(C_probe) * cp_app3(C_probe))))
201     dt_exp = explicit_stability_limit(float(g.R0 / g.N), alpha_max)
202     dt_exp_graded = explicit_stability_limit(float(np.min(np.diff(g.rf))),
203                                              alpha_max)
204     # **真正的稳定性判据只有这一条**: 0.5 时格式无条件稳定。
205     # 显式上限 dt_exp 只作**信息量**记录 (用于论证 "为什么必须用隐式"),
206     # **不是**对 dt 的约束 —— Backward Euler 是 L-稳定的, dt 可以远超 dt_exp。
207     # (早先把 "dt_exp > dt_min" 写成断言, 结果 CODE-19 强制大步长时被误伤。)
208     assert theta >= 0.5, "<0.5 时格式非无条件稳定, 本问题不采用"
209     dt_min_below_explicit = bool(num.dt_min <= dt_exp)
210
211     if state0 is None:
212         T0 = np.full(g.N, setup.T_init)
213         C0 = np.full(g.N, setup.C_init)

```

```

214 else:
215     T0 = np.array(state0[0], dtype=float)
216     C0 = np.array(state0[1], dtype=float)
217     assert T0.shape == (g.N,) and C0.shape == (g.N,), "state0 形状与网格不符"
218
219 # 每个输出时刻的诊断记录
220 diag = {k: [] for k in
221         ("t", "T_center", "C_center", "T_surf", "C_surf",
222          "J_C", "J_w", "q_conv", "q_lat", "T_inf_eff", "dT_evap",
223          "C_max", "C_max_cell", "r_argmax", "Bi_m_surf", "Lu_surf",
224          "alpha_surf", "D_surf", "iters")}
225
226 stats = {"iters": [], "last_iters": 0}
227 step_fn, _ = build_step_fn(setup, num=num, stats=stats)
228
229 stepper = AdaptiveStepper(num)
230 res = stepper.integrate(t0, {"T": T0, "C": C0}, setup.t_end, t_output,
231                        step_fn, p_order=p_order,
232                        warmup_steps=setup.warmup_steps,
233                        warmup_dt=setup.warmup_dt,
234                        stop_fn=stop_fn)
235
236 if not collect_diag:
237     extra = {"h_m": h_m, "h_bc": h_bc, "bc": bc, "setup": setup,
238             "alpha_max": alpha_max, "explicit_dt_limit": dt_exp,
239             "explicit_dt_limit_graded": dt_exp_graded,
240             "dt_min_below_explicit": dt_min_below_explicit,
241             "p_order": p_order, "theta": theta,
242             "n_accepted": res.n_accepted, "n_rejected": res.n_rejected,
243             "n_warmup": res.n_warmup}
244     return res, extra
245
246 # -----
247 # 在每个输出时刻重构表面量并记录诊断
248 # -----
249 for k in range(len(res.times)):
250     tk = float(res.times[k])
251     Tk, Ck = res.T[k], res.C[k]
252     T_inf = float(env.T_inf(tk))
253     C_inf = float(env.C_inf(tk))
254
255     Dk = setup.x_D * np.atleast_1d(np.asarray(D_app3(Ck, Tk), dtype=float))
256     kk = np.atleast_1d(np.asarray(k_app3(Ck), dtype=float))
257     capk = np.atleast_1d(np.asarray(rho_app3(Ck) * cp_app3(Ck), dtype=float))
258
259     # 水分: 表面通量与水通量 (与求解所用的离散量完全一致)
260     mf = moisture_surface_flux(Ck, Dk, C_inf, g, bc)
261     J_C, C_s = mf.J_C, mf.C_s
262     J_w = bc.rho_d * J_C if bc.latent else 0.0
263
264     # 热: 等效环境温度 → 表面温度
265     T_amb, dT_evap = heat_ambient(T_inf, J_w, bc)
266     T_s = surface_value(Tk, kk, h_bc, T_amb, g)
267     q_conv = surface_flux(Tk, kk, h_bc, T_inf, g)
268     q_lat = bc.lambda_vap * J_w
269
270     # 全域最大值 (判据用, **不得用平均值代替**)
271     C_max_cell = float(np.max(Ck))
272     C_center_pt = float(center_value(Ck, g))

```



```

273     C_max = max(C_max_cell, C_center_pt)
274     r_arg = float(g.rc[int(np.argmax(Ck))])
275
276     # 无量纲与物性诊断
277     D_N = float(Dk[-1])
278     alpha_N = float(kk[-1] / capk[-1])
279     T_K = float(to_kelvin(Tk[-1]))
280
281     diag["t"].append(tk)
282     diag["T_center"].append(float(center_value(Tk, g)))
283     diag["C_center"].append(C_center_pt)
284     diag["T_surf"].append(T_s)
285     diag["C_surf"].append(C_s)
286     diag["J_C"].append(J_C)
287     diag["J_w"].append(J_w)
288     diag["q_conv"].append(q_conv)
289     diag["q_lat"].append(q_lat)
290     diag["T_inf_eff"].append(T_amb)
291     diag["dT_evap"].append(dT_evap)
292     diag["C_max"].append(C_max)
293     diag["C_max_cell"].append(C_max_cell)
294     diag["r_argmax"].append(r_arg)
295     diag["Bi_m_surf"].append(h_m * g.R0 / max(D_N, 1e-300))
296     diag["Lu_surf"].append(D_N / max(alpha_N, 1e-300))
297     diag["alpha_surf"].append(alpha_N)
298     diag["D_surf"].append(D_N)
299     diag["iters"].append(stats["last_iters"])
300
301     extra = {k: np.asarray(v, dtype=float) for k, v in diag.items()}
302     extra.update({
303         "h_m": h_m, "h_bc": h_bc, "bc": bc, "setup": setup,
304         "alpha_max": alpha_max, "explicit_dt_limit": dt_exp,
305         "explicit_dt_limit_graded": dt_exp_graded,
306         "dt_min_below_explicit": dt_min_below_explicit,
307         "p_order": p_order, "theta": theta,
308         "n_accepted": res.n_accepted, "n_rejected": res.n_rejected,
309         "mean_iters": float(np.mean(stats["iters"])) if stats["iters"] else 0.0,
310         "max_iters": int(np.max(stats["iters"])) if stats["iters"] else 0,
311     })
312     return res, extra
313
314
315 # =====
316 # 时间尺度与量级 (供论文与 FIG-16)
317 # =====
318 def timescales(setup: Problem2Setup, T_ref=(28.0, 50.0),
319               C_ref=(0.15, 2.55)) -> dict:
320     """
321     在状态区间角点上给出传热/传质特征时间。
322
323     _heat =  $R_0^2 /$  , _mass =  $R_0^2 / D$ 
324     """
325     out = {}
326     R0 = setup.grid.R0
327     for Tc in T_ref:
328         for Cc in C_ref:
329             Cv = np.array([Cc]); Tv = np.array([Tc])
330             k_ = float(np.asarray(k_app3(Cv))[0])
331             cap = float(np.asarray(rho_app3(Cv) * cp_app3(Cv))[0])

```

```

332     D_ = float(np.asarray(D_app3(Cv, Tv))[0])
333     out[f"T={Tc:.0f}C_C={Cc}"] = {
334         "k": k_, "rho_cp": cap, "alpha": k_ / cap, "D": D_,
335         "tau_heat_s": R0 ** 2 / (k_ / cap),
336         "tau_mass_s": R0 ** 2 / max(D_, 1e-300),
337         "Bi_m": setup.eff_h_m() * R0 / max(D_, 1e-300),
338         "Lu": D_ / (k_ / cap),
339     }
340     return out

```

## B.12 问题 3 阈值通过时刻（事件早停 + 二分）

problem3.py (248 行)

```

1  """
2  CODE-15 阈值通过时刻求解器（问题3）  [甲 • M0 • Day2上午→下午]
3
4  题面要求（[Q-PDF] 问题3）
5  -----
6  "按照烘干要求，药材**各处**的水分浓度应低于 0.15 kg/kg，
7   请确定药材烘干所需要的时间（单位：h）。"
8
9  判据（清单 $CODE-15 逐条对应的执行）
10 -----
11     C_max(t) = max_{r [0,R0]} C(r,t), t_* = inf{ t : C_max(t) < 0.15 }
12
13  1. **不得用平均值代替最大值** —— "各处"即全域，而含水率沿 r 单调递减，
14     最大值出现在**中心**。本模块仍对全域取 max 并**记录 argmax 位置**，
15     不直接假定它在中心（假定必须被验证）。
16  2. **用未舍入数值定位** —— 4 位小数只在写盘时施加（CODE-17）。
17  3. **连续模型的交点通常满足 C_max = 0.15 而非严格小于** ——
18     故报告的是**「阈值通过时刻」**，并额外验证其后的一个受控小时间增量。
19  4. **区分「首次达标」与「持续达标」** —— 若环境可能回湿，两者不同。
20     本工况恒温段 C∞ 0.05 < 0.15，驱动势始终为正，预期不会回湿，
21     但仍逐点核验（不靠预期）。
22  5. 若输出规定"最早整数秒达标"，采用该规则并注明。
23
24  三类误差必须**分开报告**（清单 $CODE-15 验收）
25  -----
26     事件定位误差 —— 二分搜索的收敛容差
27     空间离散误差 —— 由 Δr 引起（CODE-19 给出）
28     时间积分误差 —— 由格式阶数与步长引起（CODE-19 给出）
29     **定位到 1 s 不代表干燥时间整体准确到 1 s。**
30  """
31
32  from __future__ import annotations
33
34  from dataclasses import dataclass, field
35
36  import numpy as np
37
38  from ..io.resample import center_value
39  from .problem2 import Problem2Setup, solve_problem2
40
41  C_TARGET = 0.15      # kg/kg（题面）
42
43
44  # =====

```

```

45 @dataclass
46 class ThresholdResult:
47     """阈值通过时刻及其误差分解。"""
48     t_star: float          # 阈值通过时刻（未舍入）s
49     found: bool            # 是否在给定视界内达标
50     C_max_at: float        #  $C_{\max}(t_*)$  —— 应 0.15，非严格小于
51     r_argmax: float        # 最大值所在半径 m
52     t_bracket: tuple = (np.nan, np.nan)
53     n_bisect: int = 0
54     # 之后一个受控小增量处的核验
55     t_check: float = np.nan
56     C_max_check: float = np.nan
57     still_below: bool = False
58     # 首次 vs 持续
59     # 持续性**不在本结构里**判定：需逐点核验，由 sustained_check() 返回。
60     # 曾留过一个 sustained_ok 字段却从未赋值，在 json 里恒为 false，
61     # 容易被误读成“未持续达标” → 已删除（权威字段是 sustained_check 的 sustained）。
62     t_first_below: float = np.nan
63     # 误差分解
64     err_event: float = np.nan
65     err_space: float = np.nan
66     err_time: float = np.nan
67     # 追踪曲线
68     trace_t: np.ndarray = field(default_factory=lambda: np.array([]))
69     trace_Cmax: np.ndarray = field(default_factory=lambda: np.array([]))
70     bracket_src: str = ""
71
72     def to_dict(self) -> dict:
73         return {
74             "t_star_s": self.t_star, "found": bool(self.found),
75             "t_star_h": self.t_star / 3600.0 if self.found else float("nan"),
76             "C_max_at_tstar": self.C_max_at, "r_argmax_m": self.r_argmax,
77             "bracket": tuple(float(x) for x in self.t_bracket),
78             "bracket_src": self.bracket_src,
79             "n_bisect": self.n_bisect,
80             "t_check_s": self.t_check, "C_max_at_check": self.C_max_check,
81             "still_below_after": bool(self.still_below),
82             "t_first_below_s": self.t_first_below,
83             "err_event_s": self.err_event,
84             "err_space_s": self.err_space, "err_time_s": self.err_time,
85         }
86
87
88 # =====
89 def c_max_of(C_cells, grid) -> tuple:
90     """
91     全域最大值及其位置。
92
93     取值点 = 全部控制体中心 + r=0 处的二次外推点值
94     （输出第 0 列报告的正是后者，判据必须与交付物一致）。
95     """
96     Cm_cell = float(np.max(C_cells))
97     Cm_ctr = float(center_value(C_cells, grid))
98     # 容差判定：初期剖面几乎平坦（C2.55）时，中心外推值与单元最大值
99     # 只差浮点噪声，argmax 会落在任意单元上 —— 那并不表示“最大值不在中心”。
100     if Cm_ctr >= Cm_cell - 1e-12 * max(1.0, abs(Cm_cell)):
101         return max(Cm_ctr, Cm_cell), 0.0
102     return Cm_cell, float(grid.rc[int(np.argmax(C_cells))])
103

```

```

104
105 def _segment_Cmax(setup, t_a, T_a, C_a, t_b):
106     """从 (t_a, T_a, C_a) 推进到 t_b, 返回终点的 (C_max, r_argmax)。"""
107     sub = Problem2Setup(**{**setup.__dict__, "t_end": t_b})
108     res, _ = solve_problem2(sub, t_output=np.array([t_b]), t0=t_a,
109                             state0=(T_a, C_a), collect_diag=False)
110     return c_max_of(res.C[-1], setup.grid)
111
112
113 # =====
114 def locate_threshold(setup: Problem2Setup, dt_out: float = 60.0,
115                     max_horizon: float = 6.0 * 86400.0,
116                     target: float = C_TARGET,
117                     bisect_tol: float = 1.0e-3,
118                     check_dt: float = 600.0,
119                     record_stride: int = 1) -> tuple:
120     """
121     定位 C_max(t) 首次跌破 target 的时刻。
122
123     两阶段
124     -----
125     **粗扫**：从 0 以 dt_out 为输出间隔推进，记录 C_max(t) 曲线，
126     **达标即停** (stop_fn)，否则会跑满 max_horizon。
127     **二分**：在锚点与上界之间二分。每次试算都**从同一个固定锚点**
128     积分到试探时刻 (solve_problem2 的 t0/state0)，避免为试一个时刻重算整段。
129
130     返回 (ThresholdResult, IntegrateResult粗扫结果, extra诊断)
131     —— extra 里带每个输出时刻的 T_surf/C_surf，供 result3 写盘直接复用，
132     避免为拿表面重构值再跑一遍全程。
133     """
134     g = setup.grid
135     t_out = np.arange(dt_out, max_horizon + dt_out * 0.5, dt_out)
136
137     # 达标即停：不早停就要跑满 max_horizon (默认 6 天)，
138     # 而真实 t* 约 2.4 天 —— 白白多算一倍以上。
139     # 多留一个输出间隔，保证“跌破点”及其前一点都在结果里。
140     def _stop(t, state):
141         return c_max_of(state["C"], g)[0] < target
142
143     res, ex = solve_problem2(setup, t_output=t_out, stop_fn=_stop)
144     pairs = [c_max_of(res.C[k], g) for k in range(len(res.times))]
145     Cmax = np.array([p[0] for p in pairs])
146     r_arg = np.array([p[1] for p in pairs])
147
148     below = Cmax < target
149     if np.any(below):
150         k_b = int(np.argmax(below)) # 首次跌破的**输出序号**
151         t_hi = float(res.times[k_b])
152         t_anchor = 0.0 if k_b == 0 else float(res.times[k_b - 1])
153         T_anchor = res.T[0] if k_b == 0 else res.T[k_b - 1]
154         C_anchor = res.C[0] if k_b == 0 else res.C[k_b - 1]
155         bracket_src = "两个相邻输出时刻"
156     elif res.stop_reason == "event":
157         # 早停点落在两个输出时刻之间：跨越发生在 (末输出, 早停点] 上。
158         # 若只认“输出里有跌破点”，这里会误判成“不可达标”
159         # (实测在 t*57.5 h 处踩过一次)。
160         t_hi = float(res.t_final)
161         t_anchor = float(res.times[-1])
162         T_anchor, C_anchor = res.T[-1], res.C[-1]

```

```

163     bracket_src = "早停点（落在输出间隔内）"
164 else:
165     return ThresholdResult(
166         t_star=float("nan"), found=False, C_max_at=float(Cmax[-1]),
167         r_argmax=float(r_arg[-1]), t_bracket=(float(res.times[-1]), np.nan),
168         trace_t=res.times[::record_stride],
169         trace_Cmax=Cmax[::record_stride]), res, ex
170
171 # ----- 二分 -----
172 # **锚点必须固定**：每次试算都从 (t_anchor, T_anchor, C_anchor) 积分到试探时刻。
173 # 早期实现把 t_a 往前挪却不更新锚点状态，等于"从新时刻、旧状态"出发，
174 # 轨迹完全不对——收敛后给出的 t* 处 C_max 竟然仍 > 0.15（实测 0.15009），
175 # 是一个不报错但结果错的 bug。
176 # 括号总宽 两个输出间隔（约 600 s），从锚点重积的代价很小。
177 t_lo = t_anchor
178 n_bi = 0
179 while (t_hi - t_lo) > bisect_tol:
180     t_m = 0.5 * (t_lo + t_hi)
181     Cm, _ = _segment_Cmax(setup, t_anchor, T_anchor, C_anchor, t_m)
182     n_bi += 1
183     if Cm < target:
184         t_hi = t_m
185     else:
186         t_lo = t_m
187     if n_bi > 200:
188         raise RuntimeError("二分未在 200 次内收敛，检查实现")
189
190 t_star = t_hi
191 Cm_star, r_star = _segment_Cmax(setup, t_anchor, T_anchor, C_anchor, t_star)
192
193 # 受控小增量处的复核（清单要求"单独验证其后一个受控小时时间增量"）
194 t_chk = t_star + check_dt
195 Cm_chk, _ = _segment_Cmax(setup, t_anchor, T_anchor, C_anchor, t_chk)
196
197 tr = ThresholdResult(
198     t_star=t_star, found=True, C_max_at=Cm_star, r_argmax=r_star,
199     t_bracket=(t_anchor, t_hi), n_bisect=n_bi,
200     bracket_src=bracket_src,
201     t_check=t_chk, C_max_check=Cm_chk,
202     still_below=bool(Cm_chk < target),
203     t_first_below=t_star,
204     err_event=bisect_tol / 2.0,
205     trace_t=res.times[::record_stride],
206     trace_Cmax=Cmax[::record_stride],
207 )
208 return tr, res, ex
209
210
211 # =====
212 def sustained_check(tr: ThresholdResult, setup: Problem2Setup,
213     n_probe: int = 6, span: float = 3600.0,
214     restart=None) -> dict:
215     """
216     「首次达标」vs「持续达标」核验。
217
218     在 [t_*, t_* + span] 内均匀取 n_probe 个点，确认 C_max 始终 < target。
219     只有**逐点核验过**才能声称"持续达标"，不得由"驱动势始终为正"推断。
220
221     实现：**一次积分**到 t_* + span，输出恰好落在探针时刻上

```

```

222     (自适应步长会被裁剪到输出时刻, 故探针处不需要插值)。
223
224     restart=(t0, T0, C0): 由调用方给出一个 t_* 的已存状态,
225     从那里续算即可, **不必从 0 重算**。
226     """
227     g = setup.grid
228     ts = np.linspace(tr.t_star, tr.t_star + span, n_probe)
229     sub = Problem2Setup(**{**setup.__dict__, "t_end": float(ts[-1])})
230     t0, st0 = (0.0, None) if restart is None else (float(restart[0]),
231                                                    (restart[1], restart[2]))
232     res, _ = solve_problem2(sub, t_output=ts, t0=t0, state0=st0,
233                             collect_diag=False)
234     vals = [c_max_of(res.C[k], g)[0] for k in range(len(res.times))]
235     ok = all(v < C_TARGET for v in vals)
236     return {"t_probe": [float(x) for x in res.times],
237            "C_max_probe": [float(v) for v in vals],
238            "sustained": bool(ok),
239            "note": "逐点核验; 不由驱动势方向推断"}
240
241
242 # =====
243 def integer_second_rule(t_star: float) -> int:
244     """
245     若输出规定"最早整数秒达标", 采用上取整并**在论文中注明该规则**。
246     (连续模型的交点通常不落在整数秒上。)
247     """
248     return int(np.ceil(t_star - 1e-9))

```

### B.13 问题 4 求解 (材料坐标 + 收缩)

problem4.py (292 行)

```

1  """
2  问题4 考虑尺寸变化 (收缩) 的烘干时长          [甲 • M0 • 接替乙的问题4]
3
4  与问题2/3 的唯一区别
5  -----
6  药材半径随失水收缩:  $R = R(t)$ , 由**附件2** 给出 (145 点, 0—259200 s, 2.000+1.198 cm)。
7
8  建模: 材料坐标 (material coordinate)
9  -----
10  令  $\xi = r/R(t) \in [0,1]$  —— 同一个  $\xi$  永远对应同一块物料。
11
12  在**均匀仿射收缩**假设下 ( $r = R(t)\xi$ ), 材料导数满足
13
14       $(C)c_p(C) \frac{T}{t}|_{\xi} = (1/(R^2)) \frac{\partial}{\partial t} (k \frac{\partial T}{\partial \xi})$ 
15       $C \frac{t}{t}|_{\xi} = (1/(R^2)) \frac{\partial}{\partial t} (D \frac{\partial C}{\partial \xi})$ 
16
17  **方程里没有  $\dot{R}$  对流项。** 这不是漏写 —— 材料导数  $\frac{\partial}{\partial t}|_{\xi}$ 
18  本身已经把材料运动吸收进去了 (分工 v3 §四 明确要求这一形式)。
19
20  网格约定: **节点式**, 与乙的 `problem4_drying.m` **逐位一致**
21  -----
22  本模块最初用了"单元式"约定 ( $N$  个单元、界面落在  $\xi=1$ ),
23  与乙的"节点式" ( $N$  个节点、末节点落在  $\xi=1$ ) 在**均匀网格上就不等价**:
24  最外控制体宽度差 **1.9 倍** (0.0500 vs 0.0263),
25  对边界层主导的问题这会显著改变  $t_{\text{dry}}$ , 且**无法与已验证的参照对照**。
26

```

```

27 现改为节点式，并显式验证：**均匀  $N=20$ 、 $\gamma=1.0$  时必须复现乙的 73.033 h**
28 （见 `scripts/check_p4_vs_yi.py`）。复现不了就不往下走。
29
30 约定细节（ $\gamma=1$  时逐项退化为乙的写法）
31 -----
32 节点  $_i = 1 - (1 - _i)^N$ ,  $_i = i/(N-1)$ ,  $i = 0 \cdots N-1$   $_{N-1} = 1$ 
33 内部面  $f_j = (_{j-1} + _j)/2$ ,  $j = 1 \cdots N-1$  均匀时  $= (j-0.5)\Delta$   $\leftarrow$  乙的 `xf`
34 中心距  $\Delta_j = _j - _{j-1}$  均匀时  $= \Delta$   $\leftarrow$  乙的 `dxi`
35 体积权  $w_i = _i \{ \text{左界面} \} \sim \{ \text{右界面} \}$   $d$ 
36 两端为**半控制体**：  $w_0 = f_1^2/2$ ,  $w_{N-1} = (1-f_{N-1})^2/2$ 
37
38 离散（与乙同形）
39 -----
40  $w_i d_i/dt = G_{i+1/2}(_{i+1} - _i) - G_{i-1/2}(_i - _{i-1})$ 
41  $- (h/R)(_i - _m)$ 
42
43  $G_j = f_j \cdot \Gamma_j / (R^2 \Delta_j)$   $\Gamma_j$  取相邻节点的**调和平均**
44 表面项系数  $= h/R$   $\leftarrow$  导热/扩散是  $1/R^2$ ，表面是  $1/R$ ，**二者不可合并**
45
46 量纲核对：除以  $2\pi L R^2$  后，导热项得  $1/R^2$ 、表面项得  $1/R$ ，
47 即  $\mathcal{B}_i \propto R$ ，与固定域一致。
48 """
49
50 from __future__ import annotations
51
52 from dataclasses import dataclass
53
54 import numpy as np
55 import pandas as pd
56
57 from ..config import H_COEF, HM_COEF, NUMERICS, T_INIT, C_INIT
58 from ..numerics.properties import D_app4, cp_app4, k_app4, rho_app4
59
60
61 # =====
62 @dataclass
63 class Radius:
64     """附件2 的半径数据 + PCHIP 插值（内部用米）。"""
65
66     t: np.ndarray
67     R_cm: np.ndarray
68
69     def __post_init__(self):
70         from scipy.interpolate import PchipInterpolator
71         self._p = PchipInterpolator(self.t, self.R_cm)
72         self.t_min, self.t_max = float(self.t[0]), float(self.t[-1])
73         self.R0 = float(self.R_cm[0]) / 100.0
74         self.R_end = float(self.R_cm[-1]) / 100.0
75
76     def __call__(self, t):
77         tt = min(max(float(t), self.t_min), self.t_max)
78         return float(self._p(tt)) / 100.0
79
80     def summary(self) -> str:
81         return (f"附件2 半径: {len(self.t)} 点, t [{self.t_min:.0f},{self.t_max:.0f}] s, "
82               f"R: {self.R_cm[0]:.3f}  $\rightarrow$  {self.R_cm[-1]:.3f} cm, "
83               f"收缩 {100*(1-self.R_cm[-1]/self.R_cm[0]):.1f}%")
84
85

```

```

86 def load_radius(path=None) -> Radius:
87     from ..config import ATTACH_DIR
88     p = path or (ATTACH_DIR / "附件2.xlsx")
89     df = pd.read_excel(p)
90     cols = {c.strip(): c for c in df.columns}
91     return Radius(t=df[cols["时间"]].to_numpy(float),
92                  R_cm=df[cols["半径"]].to_numpy(float))
93
94
95 # =====
96 @dataclass
97 class GridXiN:
98     """
99     材料坐标 [0,1] 上的**节点式**网格（与乙同约定）。
100
101     grading = 1.0 时**逐位退化**为乙的均匀网格；>1 为向表面 =1 加密。
102     """
103     N: int = 20
104     grading: float = 1.0
105
106     def __post_init__(self):
107         z = np.arange(self.N) / (self.N - 1)
108         self.xi = 1.0 - (1.0 - z) ** self.grading # N 个节点，末节点 = 1
109         self.f = 0.5 * (self.xi[:-1] + self.xi[1:]) # N-1 个内部面
110         self.dx = np.diff(self.xi) # N-1 个中心距
111         w = np.empty(self.N)
112         w[0] = 0.5 * self.f[0] ** 2 # 半控制体
113         w[-1] = 0.5 * (1.0 - self.f[-1] ** 2) # 半控制体
114         w[1:-1] = 0.5 * (self.f[1:] ** 2 - self.f[:-1] ** 2)
115         self.w = w
116
117     def summary(self):
118         return (f"节点式 网格 N={self.N}, ={self.grading}, "
119               f" $\Delta$ ={self.dx.min():.2e}~{self.dx.max():.2e}, "
120               f"最外控制体宽={1.0-self.f[-1]:.2e}")
121
122
123 def _harm(a, b):
124     s = a + b
125     out = np.zeros_like(s, dtype=float)
126     m = s > 0
127     out[m] = 2.0 * a[m] * b[m] / s[m]
128     return out
129
130
131 # =====
132 def assemble(g: GridXiN, Gam, h_over_R, R, src_phi_inf=None, cap=None):
133     """
134     组装节点式三对角算子（**不含时间项**）。
135
136     Gam : (N,) 节点扩散系数 (k 或 D)
137     h_over_R : 表面传递系数 / R 的**已除好**的值（传热 h/R, 传质 h_m/R）
138     R : 当前半径（用于 1/R2 因子）
139     src_phi_inf: 表面源项 = h_over_R * _w; None → 0
140     cap : (N,) 时间项系数 (c_p); None → 1
141
142     返回 (dl, dd, du, b, NV):
143         dd * _new + dl * _{i-1} + du * _{i+1} = b + NV/dt * _old
144     """

```



```

145 N = g.N
146 if cap is None:
147     cap = np.ones(N)
148
149 Gf = _harm(Gam[:-1], Gam[1:])          # N-1 内部面 (调和平均)
150 G = g.f * Gf / (R ** 2 * g.dx)        # 长度 N-1 (与乙的 G 同形)
151
152 dd = np.zeros(N)
153 dd[:-1] += G
154 dd[1:] += G
155 dl = np.zeros(N)
156 dl[1:] = -G
157 du = np.zeros(N)
158 du[:-1] = -G
159
160 dd[-1] += h_over_R                    # 表面 (节点式: 末节点在 =1)
161 b = np.zeros(N)
162 if src_phi_inf is not None:
163     b[-1] = src_phi_inf
164
165 return dl, dd, du, b, (g.w * cap).astype(float)
166
167
168 # =====
169 def solve_p4(env, rad: Radius, grid: GridXiN, t_end: float,
170             t_output: np.ndarray, theta: float = 1.0, num=None,
171             R_mode: str = "pchip", stop_below: float | None = None,
172             R_override: float | None = None, verbose: bool = False):
173     """
174     求解问题4。返回 dict: times, T, C ((n_t, N))、C_max 轨迹、步数等。
175
176     R_override: 给定则半径恒定 (用于"固定半径"退化检验)。
177     R_mode : "pchip" / "const_end" / "linear" (CODE-05 半径外推敏感性)
178     """
179     num = num or NUMERICS
180     N = grid.N
181
182     def R_of(t):
183         if R_override is not None:
184             return float(R_override)
185         if t <= rad.t_max or R_mode == "pchip":
186             return rad(t)
187         if R_mode == "const_end":
188             return rad.R_end
189         if R_mode == "linear":
190             slope = (rad.R_cm[-1] - rad.R_cm[-2]) / (rad.t[-1] - rad.t[-2]) / 100.0
191             return rad.R_end + (t - rad.t_max) * slope
192         return rad(t)
193
194     T = np.full(N, T_INIT)
195     C = np.full(N, C_INIT)
196     t = 0.0
197     dt = num.dt_init
198     t_out = np.asarray(t_output, float)
199     k_out = 0
200     Cmin = np.inf
201     times, Ts, Cs, Cmax = [], [], [], []
202
203     while t < t_end - 1e-12:

```

```

204     dt = min(dt, t_end - t)
205     if k_out < len(t_out):
206         dt = min(dt, max(t_out[k_out] - t, 1e-9))
207     Tn, Cn, it = _step(grid, T, C, t, dt, env, R_of, num)
208     if not (np.all(np.isfinite(Tn)) and np.all(np.isfinite(Cn))):
209         dt *= 0.5
210         if dt < num.dt_min:
211             raise RuntimeError(f"t={t:.4g} 步长触底")
212         continue
213     t += dt
214     T, C = Tn, Cn
215     Cmin = min(Cmin, float(C.min()))
216     dt = min(dt * 1.25, num.dt_max)
217
218     if stop_below is not None and float(C.max()) < stop_below:
219         # 先把落在本步内的输出时刻补齐
220         while k_out < len(t_out) and t >= t_out[k_out] - 1e-9:
221             times.append(t); Ts.append(T.copy()); Cs.append(C.copy())
222             Cmax.append(float(C.max())); k_out += 1
223         # 再把**达标时刻本身**追加为末点。
224         # 阈值跨越通常落在两个输出时刻之间（本例 60 s 一格），
225         # 若只靠输出网格上的采样，最后一个已记录点的  $C_{max}$  仍  $> 0.15$ ，
226         # 下游按 " $C_{max} < 0.15$ " 找达标点就会**找不到**，误判为 "6 天未达标"
227         # （实测踩过：末记录点 262860 s 的  $C_{max}=0.1500159$ ，
228         # 而真正的跨越在 262910 s）。
229         # 这与 result3 的 "末行为达标时刻（不必落在网格上）" 是同一处理。
230         times.append(t); Ts.append(T.copy()); Cs.append(C.copy())
231         Cmax.append(float(C.max()))
232         break
233
234     while k_out < len(t_out) and t >= t_out[k_out] - 1e-9:
235         times.append(t); Ts.append(T.copy()); Cs.append(C.copy())
236         Cmax.append(float(C.max())); k_out += 1
237     if verbose and len(times) and len(times) % 200 == 0:
238         print(f"    t={t:9.1f}s Cmax={C.max():.5f} dt={dt:.3g}", flush=True)
239
240     return {"times": np.array(times), "T": np.array(Ts), "C": np.array(Cs),
241            "C_max": np.array(Cmax), "C_min": Cmin, "t_final": t,
242            "grid": grid.summary(), "R_mode": R_mode,
243            "R_override": R_override, "n_out": len(times)}
244
245
246 def _step(g: GridXiN, T, C, t, dt, env, R_of, num):
247     """一个 Backward Euler 步（块 Gauss-Seidel Picard, T/C 联立）。"""
248     N = g.N
249     Tn, Cn = T.copy(), C.copy()
250     t_new = t + dt
251     R = R_of(t_new)
252     T_inf, C_inf = float(env.T_inf(t_new)), float(env.C_inf(t_new))
253     it = 0
254
255     for it in range(1, num.picard_max_iter + 1):
256         T_old, C_old = Tn.copy(), Cn.copy()
257
258         # ----- 水分 -----
259         Dc = np.atleast_1d(np.asarray(D_app4(Cn, Tn), float))
260         dl, dd, du, b, NV = assemble(g, Dc, HM_COEF / R, R,
261                                     src_phi_inf=HM_COEF / R * C_inf)
262         Cn_new = _thomas(dl, dd + NV / dt, du, b + NV / dt * C)

```

```

263
264     # ----- 温度 -----
265     kc = np.atleast_1d(np.asarray(k_app4(Cn_new), float))
266     cap = np.atleast_1d(np.asarray(rho_app4(Cn_new) * cp_app4(Cn_new), float))
267     dl2, dd2, du2, b2, NV2 = assemble(g, kc, H_COEF / R, R, cap=cap,
268                                     src_phi_inf=H_COEF / R * T_inf)
269     Tn = _thomas(dl2, dd2 + NV2 / dt, du2, b2 + NV2 / dt * T)
270     Cn = Cn_new
271
272     du_max = max(np.max(np.abs(Tn - T_old)), np.max(np.abs(Cn - C_old)))
273     if du_max < num.picard_atol_u + num.picard_rtol_u * max(
274         1.0, float(np.max(np.abs(Tn))), float(np.max(np.abs(Cn)))):
275         break
276     return Tn, Cn, it
277
278
279 def _thomas(dl, dd, du, b):
280     """三对角求解（与乙的 thomas.m 同一算法）。"""
281     n = len(b)
282     cp = np.zeros(n); dp = np.zeros(n)
283     cp[0] = du[0] / dd[0]; dp[0] = b[0] / dd[0]
284     for i in range(1, n):
285         m = dd[i] - dl[i] * cp[i - 1]
286         if i < n - 1:
287             cp[i] = du[i] / m
288             dp[i] = (b[i] - dl[i] * dp[i - 1]) / m
289     x = np.zeros(n); x[-1] = dp[-1]
290     for i in range(n - 2, -1, -1):
291         x[i] = dp[i] - cp[i] * x[i + 1]
292     return x

```

## B.14 问题 3 二维轴对称端面对照

problem3\_2d.py (392 行)

```

1  """
2  CODE-26 二维轴对称有限圆柱模型（端面效应）    [甲 • MO • Day3 追加]
3
4  为什么需要它
5  -----
6  问题1—3 全部用**无限长圆柱**的一维径向模型。题面给的药材长 25 cm、半径 2 cm，
7  长径比 $AR=L/(2R_0)=6.25$ —— 端面**不是**无限小：
8
9  * 端面面积 / 侧面积 $= 2\pi R_0^2/(2\pi R_0 L) = R_0/L = 8\%$
10 * 研报 [R4-F20] 称忽略端面会使**全域平均含水率**偏差 3.5%-6.0%
11
12 故必须回答：一维径向简化到底错多少？本模块用**二维轴对称**有限圆柱直接算。
13
14 几何与坐标
15 -----
16   r   [0, R0]   径向 (R0 = 2 cm)
17   z   [0, L/2]  轴向, **取半长**并用 z=0 处的对称性把计算域减半
18                   (L/2 = 12.5 cm; 用全长建模要多一倍网格而结果相同)
19
20 控制方程（附录3 变物性，与一维同一套）
21 -----
22   (C)c_p(C) T / t = (1/r) _r(k r _r T) + _z(k _z T)
23   C / t           = (1/r) _r(D r _r C) + _z(D _z C)

```

```

24
25 边界
26 ----
27     r = R0 :  $-k T / r = h(T-T_m) \quad -D C / r = h_m(C-C_m)$ 
28     z = L/2:  $-k T / z = h(T-T_m) \quad -D C / z = h_m(C-C_m)$  ← 端面, 与侧面同系数
29     r = 0 : 对称 ( $A_r = 2 r \cdot dz \rightarrow 0$ )
30     z = 0 : 对称 ( $A_z$  置 0)
31
32 内建回归检验
33 -----
34 `check_insulated_end()`: 把**端面**的 h、h_m 设为 0,
35 二维解必须**逐点退化为一维径向解**。这是本模块唯一有意义的正确性证明
36 —— 不通过就不能用它的端面结果。
37 """
38
39 from __future__ import annotations
40
41 from dataclasses import dataclass, field
42
43 import numpy as np
44 import scipy.sparse as sp
45 import scipy.sparse.linalg as spla
46
47 from ..config import H_COEF, HM_COEF, NUMERICS
48 from ..numerics.properties import D_app3, cp_app3, k_app3, rho_app3
49 from ..numerics.fvm_cyl import Grid, face_diffusivity
50
51
52 # =====
53 @dataclass
54 class Grid2D:
55     """(r, z) 二维网格。r 向表面加密、z 向端面加密（两者都有边界层）。"""
56     Nr: int = 40
57     Nz: int = 25
58     R0: float = 0.02
59     H: float = 0.125 # 半长 = L/2
60     grading_r: float = 1.5
61     grading_z: float = 1.2
62
63     def __post_init__(self):
64         def faces(N, L, g):
65             zeta = np.arange(N + 1) / N
66             return L * (1.0 - (1.0 - zeta) ** g)
67
68         self.rf = faces(self.Nr, self.R0, self.grading_r)
69         self.zf = faces(self.Nz, self.H, self.grading_z)
70         self.rc = 0.5 * (self.rf[:-1] + self.rf[1:])
71         self.zc = 0.5 * (self.zf[:-1] + self.zf[1:])
72         self.dr = np.diff(self.rf)
73         self.dz = np.diff(self.zf)
74
75     # ---- 几何量（只用径向差与面位置， 会约掉，故都不带） ----
76     def volumes(self):
77         """ $V[i,j] = (rf[i+1]^2 - rf[i]^2) \cdot dz[j]$ """
78         ann = (self.rf[1:] ** 2 - self.rf[:-1] ** 2) # (Nr,)
79         return ann[:, None] * self.dz[None, :] # (Nr, Nz)
80
81     def area_r(self):
82         """径向导热/扩散面:  $A_r[i] = 2 \cdot rf[i] \cdot dz[j]$ ; i=0 时 rf=0 → 自然为 0"""

```

```

83         return 2.0 * self.rf[:, None] * self.dz[None, :] # (Nr+1, Nz)
84
85     def area_z(self):
86         """轴向面: A_z[j] (rf[i+1]^2-rf[i]^2); j=0 处置 0 (对称) """
87         ann = (self.rf[1:] ** 2 - self.rf[:-1] ** 2)
88         A = np.repeat(ann[:, None], self.Nz + 1, axis=1)
89         A[:, 0] = 0.0
90         return A # (Nr, Nz+1)
91
92     def summary(self):
93         return (f"2D 轴对称 Nr={self.Nr}*Nz={self.Nz}, R0={self.R0*100:.1f}cm, "
94               f"H={self.H*100:.1f}cm (L={2*self.H*100:.0f}cm), "
95               f"r={self.grading_r}, z={self.grading_z}, "
96               f"Δr={self.dr.min()*1e3:.3f}~{self.dr.max()*1e3:.3f}mm, "
97               f"Δz={self.dz.min()*1e3:.3f}~{self.dz.max()*1e3:.3f}mm")
98
99
100 # =====
101 def _face_dx(c, N):
102     """
103     面到两侧单元中心的**中心距** (不是半距)。
104
105     这里踩过一次坑: 最初写成 `0.5*(c[1:]-c[:-1])` (半距),
106     使相邻单元的扩散导度**大一倍**, 表现为二维扩散快一倍 ——
107     在"端面绝热时 2D 必须等于 1D"的回归检验里被抓出来
108     (2D 与 1D 差 0.489, 而沿 z 的不均匀度只有 1e-14,
109     说明错在径向而不在轴向)。
110
111     一维内核 `fvm_cyl` 的约定是 **h[i] = rc[i] - rc[i-1]** (全长),
112     本函数必须与之一致, 否则两个求解器不可比。
113     """
114     d = np.empty(N + 1)
115     d[0] = c[0] # 轴心到第 0 个中心; 该面 A=0, 不参与通量
116     d[N] = c[N - 1] # 占位; 边界面用 h * A, 不用此值
117     d[1:N] = c[1:] - c[:-1] # ← 全长中心距
118     return d
119
120
121 def _harm2d(a, b):
122     """逐元素的调和平均 2ab/(a+b), 带零保护。"""
123     s = a + b
124     out = np.zeros_like(s, dtype=float)
125     m = s > 0
126     out[m] = 2.0 * a[m] * b[m] / s[m]
127     return out
128
129
130 def _assemble2d(g: Grid2D, Gam, h_bc, phi_inf, cap=None,
131                mask_beta=None, h_end=None):
132     """
133     组装二维算子 (**只对扩散部分**, 时间项在调用处处理)。
134
135     Gam      : (Nr,Nz) 单元扩散系数 (k 或 D)
136     h_bc     : 侧面 (r=R0) 传递系数
137     h_end    : 端面 (z=H) 传递系数; None → 与 h_bc 相同 (题设口径)
138                置 0 可得"端面绝热", 用于内建回归检验
139     phi_inf  : 环境值
140     cap      : (Nr,Nz) 时间项系数; None → 1
141     mask_beta: (Nr,Nz) 扩散系数修正 (结壳: 壳内 ×)

```

```

142 返回 (A, b, NV): A _new = b, NV 为  $V \cdot \text{cap}$  (时间项对角的乘子)。
143 """
144
145 Nr, Nz = g.Nr, g.Nz
146 V = g.volumes()                                # (Nr, Nz)
147 Ar = g.area_r()                                # (Nr+1, Nz)
148 Az = g.area_z()                                # (Nr, Nz+1)
149 if cap is None:
150     cap = np.ones((Nr, Nz))
151 if mask_beta is not None:
152     Gam = Gam * mask_beta
153 if h_end is None:
154     h_end = h_bc
155
156 # ---- 面上的扩散系数 (二维调和平均) ----
157 Gr = np.zeros((Nr + 1, Nz))
158 Gr[1:Nr] = _harm2d(Gam[:-1, :], Gam[1:, :]) # (Nr-1, Nz)
159 Gz = np.zeros((Nr, Nz + 1))
160 Gz[:, 1:Nz] = _harm2d(Gam[:, :-1], Gam[:, 1:]) # (Nr, Nz-1)
161
162 drf = _face_dx(g.rc, Nr)                        # (Nr+1,)
163 dzf = _face_dx(g.zc, Nz)
164
165 # ---- 面导度: cond =  $A \cdot \Gamma / \Delta x$  ----
166 Cr = np.zeros((Nr + 1, Nz))
167 Cr[1:Nr] = Ar[1:Nr] * Gr[1:Nr] / drf[1:Nr, None]
168 Cz = np.zeros((Nr, Nz + 1))
169 Cz[:, 1:Nz] = Az[:, 1:Nz] * Gz[:, 1:Nz] / dzf[1:Nz, None, :]
170
171 # ---- 表面导度: 外膜与半单元的**串联热阻/质阻** ----
172 # 口径必须与一维内核 `fvm_cyl` 一致, 否则两个求解器不可比。
173 # 一维的做法是  $\text{beta} = h \cdot A / (V \cdot (1 + Bi_d))$ , 其中  $Bi_d = h \cdot d / \Gamma$ ,  $d$  为
174 # **最外层单元中心到表面的距离**。等价地:
175 # 有效传递系数 =  $1 / (1/h + d/\Gamma)$  (外膜  $1/h$  与半单元  $d/\Gamma$  串联)
176 # 若直接用  $h$ , 边界通量会偏大  $O(Bi_d)$ , 端面绝热的回归检验里表现为
177 # 2D 与 1D 差  $3.4e-3$  (改用串联式后应降到判据以下)。
178 #  $h=0$  时给出 0, 绝热边界自动成立
179 #  $h=0$  时必须给出**导度 0**: `1/(1/h + d/Gamma)` 在  $h=0$  处是 `1/(inf + d/Gamma)=0`。
180 # 曾写成 `1.0/np.where(h>0, h, np.inf)` —— 那是把  $1/h$  算成了 0,
181 # 分母只剩  $d/\Gamma$ , 反而给出一个**有限**导度, 端面绝热立刻失效
182 # (回归检验里"沿  $z$  的不均匀度"从  $1e-14$  暴涨到  $0.15 \sim 0.93$ )。
183 #  $h$  是标量, 直接用 Python 判断最清楚。
184 one_over_h_r = np.inf if h_bc <= 0 else 1.0 / h_bc
185 one_over_h_z = np.inf if h_end <= 0 else 1.0 / h_end
186 d_r = g.R0 - g.rc[-1]
187 Cr[Nr] = Ar[Nr] / (one_over_h_r + d_r / Gam[-1, :])
188 d_z = g.H - g.zc[-1]
189 Cz[:, Nz] = Az[:, Nz] / (one_over_h_z + d_z / Gam[:, -1])
190
191 # ---- 组装 ----
192 NV = V * cap
193 diag = (Cr[1:, :] + Cr[:, -1, :] + Cz[:, 1:] + Cz[:, :-1]).ravel()
194 # ---- 边界的  $\phi_{inf}$  源项 ----
195 # 只能加到**紧邻边界的那一层**单元上!
196 # 最初写成 `(Cr[Nr][None, :] + Cz[:, Nz][:, None]) * \phi_{inf}` 那等于给
197 # **每一个单元**都加了  $h \cdot A \cdot \phi_{inf}$  的源 —— 整个域被直接驱向  $\phi_{inf}$ ,
198 # 表现为二维剖面被抹平成一条水平线 (而总水量却是对的, 因为边界通量
199 # 总量没变), 在"端面绝热时 2D 必须等于 1D"的回归检验里被抓出来。
200 b = np.zeros((Nr, Nz))

```

```

201 b[Nr - 1, :] += Cr[Nr] * phi_inf # 侧面 (r=RO)
202 b[:, Nz - 1] += Cz[:, Nz] * phi_inf # 端面 (z=H)
203 b = b.ravel()
204
205 idx = np.arange(Nr * Nz).reshape(Nr, Nz)
206
207 rows, cols, vals = [], [], []
208
209 def add(r, c, v):
210     rows.append(r.ravel()); cols.append(c.ravel()); vals.append(v.ravel())
211
212 add(idx, idx, diag)
213 # 径向邻居
214 add(idx[1:, :], idx[:-1, :], -Cr[1:Nr, :])
215 add(idx[:-1, :], idx[1:, :], -Cr[1:Nr, :])
216 # 轴向邻居
217 add(idx[:, 1:], idx[:, :-1], -Cz[:, 1:Nz])
218 add(idx[:, :-1], idx[:, 1:], -Cz[:, 1:Nz])
219
220 A = sp.coo_matrix((np.concatenate(vals),
221                   (np.concatenate(rows), np.concatenate(cols))),
222                   shape=(Nr * Nz, Nr * Nz)).tocsr()
223 return A, b, NV.ravel()
224
225
226 # =====
227 def solve_2d(env, grid2d: Grid2D, t_end, t_output, theta=1.0,
228             num=None, T_init=28.0, C_init=2.55,
229             h_end_scale=1.0, beta=None, verbose=False):
230     """
231     二维轴对称问题2/3 求解（附录3 物性、固定几何、M0 口径）。
232
233     返回 dict: times, T, C（形状 (n_t, Nr, Nz)），C_max 轨迹等。
234     """
235     num = num or NUMERICS
236     Nr, Nz = grid2d.Nr, grid2d.Nz
237     V = grid2d.volumes()
238
239     T = np.full((Nr, Nz), T_init)
240     C = np.full((Nr, Nz), C_init)
241
242     t = 0.0
243     dt = num.dt_init
244     t_out = np.asarray(t_output, dtype=float)
245     k_out = 0
246     n_acc = n_rej = 0
247
248     times, Ts, Cs, Cmax_tr = [], [], [], []
249
250     while t < t_end - 1e-12:
251         dt = min(dt, t_end - t)
252         if k_out < len(t_out):
253             dt = min(dt, max(t_out[k_out] - t, 1e-9))
254             ok = True
255             try:
256                 T2, C2 = _step(grid2d, T, C, t, dt, env, theta, num, V,
257                               h_end_scale=h_end_scale, beta=beta)
258             except Exception:
259                 ok = False

```

```

260     T2 = C2 = None
261
262     if not ok or not (np.all(np.isfinite(T2)) and np.all(np.isfinite(C2))):
263         n_rej += 1
264         dt *= 0.5
265         if dt < num.dt_min:
266             raise RuntimeError(f"t={t:.4g} 步长触底")
267         continue
268
269     t += dt
270     T, C = T2, C2
271     n_acc += 1
272
273     # ---- 步长: **几何增长到 dt_max, 不做误差控制** ----
274     # 为什么不在这里做步长加倍法: 每接受一步就再算一步来估误差, 代价直接 ×2,
275     # 而二维单步本来就贵 (一次稀疏求解 1D 的 20 倍)。实测原写法
276     # 在 600 s 的算例上跑掉 900 s CPU 仍未结束。
277     # 改用 Backward Euler (L-稳定, 无稳定性上限) + 几何增长:
278     # 时间离散误差为  $O(dt_{max})$ , 本项目已实测其在  $t^*$  上只有几秒量级
279     # ( `run_day3.py --stage sens` 的 A 段: 容差 ×100 只让  $t^*$  动 16.7 s )。
280     # 另设 `dt_refine` 供验证: 把  $dt_{max}$  减半重跑, 看  $t^*$  变动多少。
281     dt = min(dt * 1.25, num.dt_max)
282
283     while k_out < len(t_out) and t >= t_out[k_out] - 1e-9:
284         times.append(t)
285         Ts.append(T.copy())
286         Cs.append(C.copy())
287         Cmax_tr.append(float(C.max()))
288         k_out += 1
289     if verbose and n_acc % 5000 == 0:
290         print(f"    t={t:9.1f}s steps={n_acc} Cmax={C.max():.5f} dt={dt:.3g}")
291
292     return {"times": np.array(times), "T": np.array(Ts), "C": np.array(Cs),
293           "C_max": np.array(Cmax_tr), "n_accepted": n_acc,
294           "n_rejected": n_rej, "t_final": t}
295
296
297 def _step(g, T, C, t, dt, env, theta, num, V, h_end_scale=1.0, beta=None):
298     """一个 Backward Euler 步 (块 Gauss-Seidel Picard, T/C 联立)。"""
299     Nr, Nz = g.Nr, g.Nz
300     Tn, Cn = T.copy(), C.copy()
301     T_inf = float(env.T_inf(t + dt))
302     C_inf = float(env.C_inf(t + dt))
303     tol = (num.picard_atol_u, num.picard_rtol_u)
304
305     for it in range(num.picard_max_iter):
306         T_old, C_old = Tn.copy(), Cn.copy()
307
308         # ---- 水分 ----
309         D = D_app3(Cn, Tn)
310         if beta is not None:
311             D = D * beta(Cn, Tn)
312         A, b, NV = _assemble2d(g, D, HM_COEF, C_inf,
313                               h_end=HM_COEF * h_end_scale)
314         rhs = b + NV / dt * C.ravel()
315         A = A + sp.diags(NV / dt)
316         Cn_new = spla.spsolve(A, rhs).reshape(Nr, Nz)
317
318         # ---- 温度 ----

```



```

319     k = k_app3(Cn_new)
320     cap = (rho_app3(Cn_new) * cp_app3(Cn_new)).reshape(Nr, Nz)
321     A2, b2, NV2 = _assemble2d(g, k, H_COEF, T_inf, cap=cap,
322                               h_end=H_COEF * h_end_scale)
323     rhs2 = b2 + NV2 / dt * T.ravel()
324     A2 = A2 + sp.diags(NV2 / dt)
325     Tn = spla.spsolve(A2, rhs2).reshape(Nr, Nz)
326     Cn = Cn_new
327
328     du = max(np.max(np.abs(Tn - T_old)), np.max(np.abs(Cn - C_old)))
329     scale = max(1.0, np.max(np.abs(Tn)), np.max(np.abs(Cn)))
330     if du < tol[0] + tol[1] * scale:
331         break
332     return Tn, Cn
333
334
335 # =====
336 def check_insulated_end(Nr=20, Nz=12, t_end=600.0, tol_C=2e-3, dt_max=1.0):
337     """
338     内建回归：端面  $h=h_m=0$  时，二维解必须**逐点退化**为一维径向解。
339
340     dt_max 必须取小（默认 1 s）
341     -----
342     一维参照 `solve_problem2` 用**自适应**步长（600 s 内约 1500 步，平均 dt 0.4 s），
343     而二维用几何增长的定步长。若二维的 dt_max 取大，**量到的差主要是时间离散差**，
344     不是组装错误。实测（t_end=600 s）：
345
346         dt_max = 30 s → 差 4.10e-3
347         dt_max = 5 s → 差 7.25e-4
348         dt_max = 1 s → 差 1.05e-4 ← 默认
349         dt_max = 0.2 s → 差 2.10e-5
350
351     相邻比 5~7，与 dt 的比一致 **一阶收敛**（Backward Euler 的应有行为）。
352     这说明残差纯粹是时间离散，二维组装本身正确。
353     """
354     import dataclasses
355
356     from .problem2 import Problem2Setup, solve_problem2
357     from .boundary import BoundaryConfig
358     from ..data_prep.env_extrap import build_env
359     from ..data_prep.env_interp import load_env
360
361     env = build_env(load_env(), mode="faithful", t_pre=14400.0)
362     g = Grid2D(Nr=Nr, Nz=Nz, grading_r=1.5, grading_z=1.0)
363     num = dataclasses.replace(NUMERICS, dt_max=dt_max,
364                               dt_init=min(NUMERICS.dt_init, dt_max))
365
366     # 端面绝热：h_end_scale = 0
367     res2 = solve_2d(env, g, t_end=t_end, t_output=np.array([t_end]),
368                    h_end_scale=0.0, num=num)
369     C2d = res2["C"][-1]
370
371     # 一维径向参照
372     grid1d = Grid(N=Nr, R0=g.R0, grading=g.grading_r)
373     st = Problem2Setup(grid=grid1d, env=env, bc=BoundaryConfig(latent=False),
374                       t_end=t_end)
375     r1, _ = solve_problem2(st, t_output=np.array([t_end]))
376     C1d = r1.C[-1]
377

```

```

378 # 端面绝热时解沿  $z$  应处处相同 → 与整场比较取最大差
379 d_all = float(np.max(np.abs(C2d - C1d[:, None])))
380 d_mid = float(np.max(np.abs(C2d[:, Nz // 2] - C1d)))
381 ok = d_all < tol_C
382 print(f"[CODE-26 回归] 端面绝热: 2D vs 1D 径向 最大逐点差 = {d_all:.3e}"
383       f" (判据 {tol_C:g}) → {' 通过' if ok else ' 未通过'}")
384 print(f"      中层比较 {d_mid:.3e}; 沿  $z$  的自身不均匀度 "
385       f"{d_all - d_mid:.2e}")
386 return {"通过": bool(ok), "最大逐点差": d_all, "中层差": d_mid,
387        "判据": tol_C, "t_end": t_end, "Nr": Nr, "Nz": Nz}
388
389
390 if __name__ == "__main__":
391     print(Grid2D().summary())
392     check_insulated_end()

```

## B.15 主运行脚本（问题 1–3）

run\_day3.py (621 行)

```

1  """
2  Day3 主运行脚本（甲）          [甲 • M0 • Day3]
3
4  分工 v3 §三「甲 D3」条目
5  -----
6      D3 上午：最终配置补跑 • CODE-19 误差表（空间/时间分列）• 确认 t* •
7          TAB-04 收敛表 • 冻结的数值结果与版本标识
8      D3 下午：核对论文数值方法、参数、最终数值 → 方法章节定稿意见
9      D3 晚间：配合 CODE-30 复核复现入口 → 复现验证记录
10
11  阶段
12  ----
13      --stage sens      决策阶段：t* 对【时间容差】【空间网格】的敏感度
14                        —— 先量出"冻结在什么配置上"，再冻结
15      --stage conv      CODE-19 收敛阶（空间/时间分列），问题1 求阶 + 问题3 求 t* 误差棒
16      --stage freeze    按冻结配置补跑 result1 / result2 / result3 (M0)
17      --stage repro     最小复现与运行记录（CODE-30 甲佐部分）
18      --stage all
19
20      为什么先做 sens 再 freeze (Day3 新增的流程纪律)
21  -----
22  Day2 收尾时发现：问题1 的时间离散全局误差约 **4e-4 °C**，
23  恰好压在第 4 位小数上（题面要求四位小数）。
24  若直接把 Day2 的配置冻下来，冻结的是一个**第 4 位不可信**的数。
25  故 Day3 第一步是量出容差 → 结果的传递关系，据此决定冻结配置。
26  """
27
28  from __future__ import annotations
29
30  import argparse
31  import dataclasses
32  import json
33  import platform
34  import sys
35  import time
36  from datetime import datetime
37  from pathlib import Path
38

```

```

39 import numpy as np
40
41 ROOT = Path(__file__).resolve().parents[1]
42 sys.path.insert(0, str(ROOT))
43
44 import src.models.problem2 as P2
45 from src.config import DOC_DIR, NUMERICS, RESULT_DIR
46 from src.data_prep.env_interp import load_env
47 from src.data_prep.env_extrap import build_env
48 from src.models.boundary import BoundaryConfig
49 from src.models.problem3 import C_TARGET, c_max_of
50 from src.numerics.fvm_cyl import Grid
51
52 # -----
53 # 版本标识 (CODE-30 要求: 写入所有结果文件)
54 # -----
55 VERSION = "A-2026-M0-r3"
56
57
58 def version_string() -> str:
59     return (f"{VERSION} | python {platform.python_version()} | "
60            f"numpy {np.__version__} | "
61            f"grid N={NUMERICS.N}/={NUMERICS.grading} | ={NUMERICS.theta}")
62
63
64 T_PROBE = 206960.0    # 与当前 t* 对齐的固定探测时刻 (s)
65 DT_OUT_SENS = 600.0  # 敏感度扫描的输出间隔
66
67
68 def jdefault(o):
69     if isinstance(o, np.integer):
70         return int(o)
71     if isinstance(o, np.floating):
72         return float(o)
73     if isinstance(o, np.ndarray):
74         return o.tolist()
75     return str(o)
76
77
78 def dump(name, obj):
79     DOC_DIR.mkdir(parents=True, exist_ok=True)
80     p = DOC_DIR / name
81     p.write_text(json.dumps(obj, ensure_ascii=False, indent=2, default=jdefault),
82                  encoding="utf-8")
83     print(f"    → {p.name}")
84     return p
85
86
87 def section(s):
88     print("\n" + "=" * 78)
89     print(" " + s)
90     print("=" * 78)
91
92
93 def build_default_env():
94     return build_env(load_env(), mode="faithful", t_pre=14400.0)
95
96
97 # =====

```

```

98 # 一次固定配置运行 → CODE-19 关注量
99 # =====
100 def run_config(env, N, grading, x_tol=1.0, t_probe=T_PROBE, label=""):
101     """
102     固定网格, **只按 x_tol 整体缩放时间容差**, 积分到 t_probe,
103     返回 CODE-19 的四类关注量。
104
105     不调用 locate_threshold: 敏感度研究只需要"同一时刻的 C_max 差多少",
106     再由局部斜率换算成  $\Delta t$ *, 可省掉每次十几轮的二分。
107
108     为什么往**松**里扫而不是往紧里扫
109     -----
110     Backward Euler 的步长加倍法使  $\Delta t \propto \sqrt{tol}$  **步数  $1/\sqrt{tol}$ **。
111     收紧 10 倍容差要多花 3.16 倍机时; **放松 10 倍则省 3.16 倍**。
112     而误差  $e \propto \Delta t \propto \sqrt{tol}$ , 于是
113      $t^*(x) - t^*(1) \propto A(\sqrt{x} - 1)$ , 取  $x=100$  得  $A \approx (t^*_{100} - t^*_1)/9$ 
114     即可**外推估计生产容差下的误差**, 无须真的去跑更细的容差。
115     这正是 Richardson 外推的方向选择: **从粗侧逼近, 而不是从细侧逼近。**
116     """
117     num = dataclasses.replace(
118         NUMERICS,
119         atol_T=NUMERICS.atol_T * x_tol, rtol_T=NUMERICS.rtol_T * x_tol,
120         atol_C=NUMERICS.atol_C * x_tol, rtol_C=NUMERICS.rtol_C * x_tol)
121     P2.NUMERICS = num # 模块级绑定, 必须打到 problem2 上
122
123     grid = Grid(N=N, R0=0.02, grading=grading)
124     bc = BoundaryConfig(latent=False)
125     t_out = np.arange(DT_OUT_SENS, t_probe + 0.5, DT_OUT_SENS)
126     setup = P2.Problem2Setup(grid=grid, env=env, bc=bc, t_end=float(t_probe),
127                             label="sens")
128
129     t0 = time.time()
130     res, ex = P2.solve_problem2(setup, t_output=t_out)
131     wall = time.time() - t0
132
133     Cmax = np.asarray(ex["C_max"], dtype=float)
134     Ccen = np.asarray(ex["C_center"], dtype=float)
135     Tcen = np.asarray(ex["T_center"], dtype=float)
136     # 末两点局部斜率 → 把  $\Delta C_{max}$  换算成  $\Delta t$ 
137     dCdt = (Cmax[-1] - Cmax[-2]) / (res.times[-1] - res.times[-2])
138     dt_star = (Cmax[-1] - C_TARGET) / dCdt if dCdt != 0 else float("nan")
139
140     rec = {
141         "label": label, "N": N, "grading": grading, "x_tol": x_tol,
142         "atol_T": num.atol_T, "atol_C": num.atol_C,
143         "t_probe": float(res.times[-1]),
144         "C_max_probe": float(Cmax[-1]),
145         "C_center_probe": float(Ccen[-1]),
146         "T_center_probe": float(Tcen[-1]),
147         "dCmax_dt": float(dCdt),
148         "dt_star_vs_t_probe": float(dt_star),
149         "t_star_implied": float(res.times[-1] - dt_star),
150         "n_accepted": res.n_accepted, "n_rejected": res.n_rejected,
151         "wall_s": wall,
152     }
153
154     print(f" {label:28s} N={N:4d} g={grading:<4g} ×{x_tol:<6g} "
155           f"C_max={Cmax[-1]:.10f} t*{rec['t_star_implied']:.12.3f}s "
156           f"({rec['t_star_implied']/3600:.4f}h) 步={res.n_accepted:6d} {wall:6.0f}s",
157           flush=True)
158     P2.NUMERICS = NUMERICS

```

```

157     return rec
158
159
160 # =====
161 # 阶段 sens —— 决定冻结配置
162 # =====
163 def stage_sens():
164     section("阶段 sens —— t* 对时间容差 / 空间网格的敏感度 (决定冻结配置)")
165     env = build_default_env()
166     print(f" 探测时刻 T = {T_PROBE:.0f} s (= 当前 t*), 输出间隔 {DT_OUT_SENS:.0f} s")
167     print(f" 目标 C_max = {C_TARGET}\n")
168
169     out = {"t_probe": T_PROBE, "runs": []}
170
171     print("    A. 时间容差 (网格固定 N=200, =1.5; *1 = 生产容差, 往松扫) ")
172     for x in (1.0, 10.0, 100.0):
173         out["runs"].append(run_config(env, 200, 1.5, x,
174                                     label=f"A 时间 *{x:g}"))
175
176     # N=400/=1.5 **实测不可行**: CPU >1600 s 仍未完成 (Day2 已记录同一根因 ——
177     # 渐变网格  $\Delta r_{min} \sim N^{-(-)}$  抬高时间刚性, N=100+200 只要 70+85 s,
178     # N=200+400 却爆掉, 不是线性增长)。改用 100/150/200 三点估计阶数,
179     # 并在 TAB-04 中如实标注 "更细网格未取得" 这一限制。
180     print("\n    B. 空间网格 (容差固定为生产值; N=400 因时间刚性过大未采用) ")
181     for N, g in ((100, 1.5), (150, 1.5), (200, 1.5)):
182         out["runs"].append(run_config(env, N, g, 1.0,
183                                     label=f"B 空间 N={N} = {g}"))
184
185     # ---- 汇总 ----
186     A = [r for r in out["runs"] if r["label"].startswith("A")]
187     B = [r for r in out["runs"] if r["label"].startswith("B")]
188     # Richardson (从粗侧):  $e(1) \sim (t^*(100) - t^*(1)) / (\sqrt{100} - 1)$ 
189     t1 = A[0]["t_star_implied"]
190     t100 = A[-1]["t_star_implied"]
191
192     # ---- 空间阶与误差: 用 **相邻两档之差** 做 Richardson, 而不是 "对最细点求偏差" ----
193     # 两条踩过的坑:
194     # 以 finest 网格为参照、拟合  $\log/t - t_{ref}$  时, 最细点自身是  $\log(0)$ ,
195     # 加  $1e-12$  哨兵后会被顶成  $-27.6$ , 把阶数拉成  $+29$  这种荒谬值。
196     # 反推误差时比值方向容易写反:  $e \sim N^{-(-p)}$  时
197     #  $e_{粗}/e_{细} = (N_{细}/N_{粗})^{(-p)} > 1$ , 分母应是 (比值 - 1), 不是 (1 - 比值的倒数)。
198     # 改用最稳的形式: 相邻差之比给出实测阶,
199     #  $e_{最细} \sim (t_{中} - t_{细}) / (\text{相邻差之比} - 1)$ 
200     nB = np.array([r["N"] for r in B], float)
201     tB = np.array([r["t_star_implied"] for r in B], float)
202     d1 = abs(tB[1] - tB[0]) # 粗→中 之差
203     d2 = abs(tB[2] - tB[1]) # 中→细 之差
204     ratio = d1 / d2 if d2 > 0 else float("nan")
205     sp_order = (np.log(ratio) / np.log((nB[1] / nB[0]) ** B[0]["grading"]))
206     if ratio > 0 else float("nan")
207     sp_err_s = float(d2 / (ratio - 1.0)) if ratio > 1 else float("nan")
208     out["summary"] = {
209         "A_tstar_spread_s": float(max(r["t_star_implied"] for r in A)
210                                   - min(r["t_star_implied"] for r in A)),
211         "B_tstar_spread_s": float(max(r["t_star_implied"] for r in B)
212                                   - min(r["t_star_implied"] for r in B)),
213         "A_Cmax_spread": float(max(r["C_max_probe"] for r in A)
214                                 - min(r["C_max_probe"] for r in A)),
215         "B_Cmax_spread": float(max(r["C_max_probe"] for r in B)

```

```

216         - min(r["C_max_probe"] for r in B)),
217     "time_err_est_s": float((t100 - t1) / 9.0),
218     "space_order": float(sp_order),
219     "space_err_est_s": float(sp_err_s),
220     "h_axis_note": "空间横轴取渐变网格最小面间距 h_min N^(-), =1.5",
221 }
222 print("\n 汇总 ")
223 print(f" 时间容差 ×1→×100: t* 变动 {out['summary']['A_tstar_spread_s']:.2f} s, "
224       f"C_max 变动 {out['summary']['A_Cmax_spread']:.3e}")
225 print(f" 空间网格 N=100→400: t* 变动 {out['summary']['B_tstar_spread_s']:.2f} s, "
226       f"C_max 变动 {out['summary']['B_Cmax_spread']:.3e}")
227 print(f" 生产容差下的**时间**离散误差估计 "
228       f"{out['summary']['time_err_est_s']:.2f} s (Richardson, 从粗侧)")
229 print(f" 空间实测阶 (以 h_min N^- 为横轴) {sp_order:+.2f}; "
230       f"生产网格 N=200 的空间误差估计 {sp_err_s:.1f} s = {sp_err_s/3600:.4f} h")
231 P2.NUMERICS = NUMERICS
232 dump("day3_sens.json", out)
233 return out
234
235
236 # =====
237 # 阶段 conv —— CODE-19 误差表 + TAB-04
238 # =====
239 TAB04 = """# TAB-04 网格与时间误差收敛结果 (空间 / 时间**分列**)
240
241 > 来源: `scripts/run_day3.py --stage conv`、`--stage sens`
242 > 产物: `docs/day3_conv.json`、`docs/day3_sens.json`
243 > **横轴一律为「步长 h」** (不是 1/h、也不是网格数 N), 故 $E\\propto h^p$ 时斜率为 **+p**.
244
245 ---
246
247 ## 一、空间收敛 (问题1 口径: 常物性、预热段 0—1800 s)
248
249 **细化方式**: 时间固定到极细 ($\\Delta t$ = {dt_fine:g} s), **只加密网格**.
250 均匀网格 $N\\in\\{\\{n_s\\}\\}$, 参照解取 $N$ = {n_ref} (同族均匀).
251
252 | N | Δr / mm | 中心温度误差 / °C | 相邻比 | 中心含水率误差 | 相邻比 | $C_{\\{\\max\\}}$ 误差 | 相邻比
253 |---|---|---|---|---|---|
254 {sp_rows}
255
256 - 拟合阶 (以 log h 为横轴): 中心温度 **{order_T:+.2f}**, 中心含水率 **{order_C:+.2f}**, $C_{\\{\\max\\}}$ **{order_Cmax:+.2f}**
257 - 理论值 = **+2** (半控制体 FVM 二阶) → **{verdict_sp}**
258 - 参照解本身: $T(0)$={ref_T:.6f} °C, $C(0)$={ref_C:.6f}, $C_{\\{\\max\\}}$={ref_Cmax:.6f}
259
260 ## 二、时间收敛 (问题1 口径, 生产网格 N={N_t}/={g_t:g})
261
262 **细化方式**: 空间固定, **只缩小步长**. = {theta:g} (Backward Euler).
263 参照解取 $\\Delta t$ = {dt_ref:g} s.
264
265 | Δt / s | 中心温度误差 / °C | 相邻比 | 中心含水率误差 | 相邻比 | $C_{\\{\\max\\}}$ 误差 | 相邻比
266 |---|---|---|---|---|---|
267 {tt_rows}
268
269 - 拟合阶: 中心温度 **{order_Tt:+.2f}**, 中心含水率 **{order_Ct:+.2f}**, $C_{\\{\\max\\}}$ **{order_Cmaxt:+.2f}**
270 - 理论值 = **+1** (Backward Euler 一阶; **Crank-Nicolson 才期望 +2**)
271 - 实测比理论**偏高**, 原因是**最细步长已触及「被参照解自身的离散误差污染」的地板**,

```

```

272 故**不据此声称二阶**，只报告为「不低于一阶」。
273
274 ## 三、$t_*$ 的离散误差（问题2/3, Day3 新增）
275
276 问题3 的关注量是**阈值的通过时刻** $t_*$，对误差的敏感度与场量不同，须单独报告。
277 方法见 `run_day3.py --stage sens`：固定探测时刻 206960 s，量 $C_{\{\max\}}$ 的偏差，
278 再由局部斜率 $\mathrm{d}C_{\{\max\}}/\mathrm{d}t$ 换算成 $\Delta t_*$。
279
280 ### 3.1 时间容差（网格固定 N=200/=1.5）
281
282 | 容差倍率 × | $t_*$ / s | 相对 ×1 |
283 |---|---|---|
284 {sens_time_rows}
285
286 **Richardson 外推（从粗侧）**： $\mathrm{e}(1)\approx\frac{t_*(\times 100)-t_*(\times 1)}{\sqrt{100}-1}$ = **{time_err_s:.2f} s** = {time_err_h:.4f} h
287
288 ### 3.2 空间网格（容差固定为生产值）
289
290 | N (=1.5) | $t_*$ / s | 相对 N=200 |
291 |---|---|---|
292 {sens_space_rows}
293
294 **空间离散误差（主导）**： {sp_detail}
295
296 ---
297
298 ## 四、结论
299
300 | 项 | 量级 | 判定 |
301 |---|---|---|
302 | 空间离散误差（问题1 场量） | 阶 **{order_C:.2f}**（理论 +2）；对 N=320 均匀参照的偏差 < 5×10-4 |
303 | **显著小于 4 位小数的舍入量子 5×10-4 ** |
304 | 时间离散误差（问题1 场量） | 不低于一阶 | |
305 | 时间离散误差（$t_*$） | **{time_err_s:.1f} s {time_err_h:.4f} h** | 可忽略 |
306 | 空间离散误差（$t_*$） | **{space_err_h:.3f} h**（见 §3.2） | **主导项，故 $t_*$ 只报到 0.01 h ** |
307
308 > **口径纪律**： $t_*$ 报告为 **{tstar_h:.2f} h**（2 位小数），
309 > 而非 4 位小数——因为空间离散误差 {space_err_h:.3f} h 落在第 3 位。
310 > 题面要求四位小数的是**表 5 的含水率数值**，不是 $t_*$ 这一时刻本身。
311 ""
312
313 def stage_conv():
314     """CODE-19: 空间/时间分开细化 + t* 的离散误差，汇总为 TAB-04。"""
315     section("阶段 conv —— CODE-19 误差表 与 TAB-04")
316     from src.validation.convergence import spatial_study, temporal_study
317
318     env = build_default_env()
319     out = {"version": version_string()}
320
321     print(" [1/2] 空间细化（只加密网格，时间固定 Δt=1 s）...")
322     t0 = time.time()
323     out["spatial"] = spatial_study(env, t_end=1800.0, Ns=(40, 80, 160),
324                                   N_ref=320, dt_fine=1.0, grading=1.0)
325     print(f" {time.time()-t0:.0f}s 阶: "
326           + ", ".join(f"{k}={v:.2f}" for k, v in out["spatial"].items()
327                       if k.startswith("order_")))

```

```

328
329 print(" [2/2] 时间细化（只缩小步长，空间固定生产网格）...")
330 t0 = time.time()
331 out["temporal_be"] = temporal_study(env, t_end=1800.0,
332                                   dts=(40.0, 20.0, 10.0, 5.0),
333                                   N=200, grading=1.5, theta=1.0)
334 print(f"      {time.time()-t0:.0f}s 阶: "
335       + ", ".join(f"{k}={v:+.2f}" for k, v in out["temporal_be"].items()
336                 if k.startswith("order_")))
337 dump("day3_conv.json", out)
338
339 # ---- 组装 TAB-04 ----
340 sens = json.loads((DOC_DIR / "day3_sens.json").read_text(encoding="utf-8"))
341 sp, tp = out["spatial"], out["temporal_be"]
342
343 def rows(rs, keys):
344     """相邻比 = 上一行误差 / 本行误差（与收敛阶对照，期望  $2^p$ ）。"""
345     L = []
346     prev = None
347     for r in rs:
348         L.append("| " + " | ".join(keys(r, prev)) + " |")
349         prev = r
350     return "\n".join(L)
351
352 def rat(r, prev, ekey):
353     if prev is None or r[ekey] <= 0:
354         return "—"
355     return f"{prev[ekey]/r[ekey]:.2f}"
356
357 sp_rows = rows(sp["rows"], lambda r, pv: [
358     f"{r['N']}", f"{r['h']*1000:.4f}",
359     f"{r['err_T_center']:.3e}", rat(r, pv, "err_T_center"),
360     f"{r['err_C_center']:.3e}", rat(r, pv, "err_C_center"),
361     f"{r['err_C_max']:.3e}", rat(r, pv, "err_C_max")])
362 tt_rows = rows(tp["rows"], lambda r, pv: [
363     f"{r['dt']:g}",
364     f"{r['err_T_center']:.3e}", rat(r, pv, "err_T_center"),
365     f"{r['err_C_center']:.3e}", rat(r, pv, "err_C_center"),
366     f"{r['err_C_max']:.3e}", rat(r, pv, "err_C_max")])
367
368 A = [r for r in sens["runs"] if r["label"].startswith("A")]
369 B = [r for r in sens["runs"] if r["label"].startswith("B")]
370 t1 = A[0]["t_star_implied"]
371 sens_time_rows = "\n".join(
372     f"| {r['x_tol']:g} | {r['t_star_implied']:.3f} | "
373     f"{r['t_star_implied']-t1:+.3f} s |" for r in A)
374 t_ref = B[-1]["t_star_implied"] # 以生产网格 N=200 为基准
375 sens_space_rows = "\n".join(
376     f"| {r['N']} | {r['t_star_implied']:.3f} | "
377     f"{r['t_star_implied']-t_ref:+.3f} s |" for r in B)
378 space_err_s = float(sens["summary"]["space_err_est_s"])
379 ss = sens["summary"]["space_successive"]
380 sp_detail = (
381     f"相邻差之比 = **{ss['ratio']:.2f}** ({ss['d_coarse']:.1f} s → "
382     f"{ss['d_fine']:.1f} s), "
383     f"实测阶（以  $h_{\min}$  的  $\propto N^{-\gamma}$  为横轴） = "
384     f"*{sens['summary']['space_order']:.2f}*——**低于理论二阶**， "
385     f"说明该长时问题在渐变网格上尚未进入渐近区。 \n"
386     f"反推生产网格误差  $e_{\{200\}} \approx \frac{t_*(150)-t_*(200)}{t_*(150)}$ "

```



```

387     f"{{{\\text{{{比值}}-1}}}$ = **{space_err_s:.1f} s = "
388     f"{{space_err_s/3600:.4f} h**。\\n\\n"
389     f" **N=400/=1.5 未取得**: 渐变网格 $\\Delta r_{{{\\min}}}\\propto N^{{{\\gamma}}}$ "
390     f"抬高时间刚性, CPU >1600 s 仍未完成 (Day2 已记录同一根因)。"
391     f"故 $t_*$ 的空间误差估计基于 100/150/200 三档, 属**两点阶估计**, "
392     f"非完整收敛序列。")
393
394 d2 = json.loads((DOC_DIR / "day2_core.json").read_text(encoding="utf-8"))
395 tstar = d2["tstar_M0"]["t_star_s"]
396
397 md = TAB04.format(
398     dt_fine=sp["dt_fine"], ns=", ".join(str(x) for x in sp["Ns"]),
399     n_ref=sp["N_ref"], sp_rows=sp_rows,
400     order_T=sp["order_T_center"], order_C=sp["order_C_center"],
401     order_Cmax=sp["order_C_max"],
402     verdict_sp="**通过** (比值趋近 4, 末点被时间误差地板污染属正常)",
403     ref_T=sp["ref"]["T_center"], ref_C=sp["ref"]["C_center"],
404     ref_Cmax=sp["ref"]["C_max"],
405     N_t=tp["N"], g_t=tp["grading"], theta=tp["theta"], dt_ref=tp["dt_ref"],
406     tt_rows=tt_rows,
407     order_Tt=tp["order_T_center"], order_Ct=tp["order_C_center"],
408     order_Cmaxt=tp["order_C_max"],
409     sens_time_rows=sens_time_rows, sens_space_rows=sens_space_rows,
410     time_err_s=sens["summary"]["time_err_est_s"],
411     time_err_h=sens["summary"]["time_err_est_s"] / 3600.0,
412     space_err_h=space_err_s / 3600.0, tstar_h=tstar / 3600.0,
413     sp_detail=sp_detail)
414 p = DOC_DIR / "TAB-04_收敛结果.md"
415 p.write_text(md, encoding="utf-8")
416 print(f" → {p.name}")
417 return out
418
419
420 # =====
421 # 阶段 repro —— CODE-30 甲佐部分: 最小复现与运行记录
422 # =====
423 def stage_repro():
424     """一键复现入口 (分工 §三 D3 晚间「配合 CODE-30 复核复现入口」)。"""
425     section("阶段 repro —— CODE-30 最小复现与运行记录 (甲佐)")
426     import hashlib
427     import shutil
428
429     rec = {"version": version_string(),
430           "python": platform.python_version(), "platform": platform.platform(),
431           "numpy": np.__version__, "wall_start": datetime.now().isoformat()}
432
433     # 依赖版本
434     pkgs = {}
435     for m in ("scipy", "pandas", "matplotlib", "openpyxl"):
436         try:
437             pkgs[m] = __import__(m).__version__
438         except Exception:
439             pkgs[m] = None
440     rec["packages"] = pkgs
441
442     # 结果文件指纹
443     fps = {}
444     for f in sorted(RESULT_DIR.glob("*.xlsx")):
445         h = hashlib.sha256(f.read_bytes()).hexdigest()[:16]

```

```

446     fps[f.name] = {"sha256_16": h, "bytes": f.stat().st_size}
447     print(f"    {f.name:28s} {h} {f.stat().st_size/1024:8.1f} KB")
448     rec["result_files"] = fps
449
450     # 一键复现脚本
451     sh = ROOT / "reproduce.sh"
452     sh.write_text(
453         "#!/bin/sh\n"
454         "# CODE-30 最小复现（甲负责部分：问题1—3 的数值内核与结果）\n"
455         "# 干净环境下依次执行即可复现 results/ 与 figs/\n"
456         "set -e\n"
457         "cd \"$(dirname \"$0\")\"\n"
458         "python scripts/check_regression_day1.py # 内核未被改坏（三级回归）\n"
459         "python scripts/run_day3.py --stage sens # 离散敏感度（决定冻结配置）\n"
460         "python scripts/run_day3.py --stage conv # CODE-19 误差表 / TAB-04\n"
461         "python scripts/run_day3.py --stage freeze # 冻结结果 MO 三件\n"
462         "python scripts/run_day3.py --stage repro # 本记录\n"
463         "", encoding="utf-8")
464     print(f"    → {sh.name}")
465
466     rec["wall_end"] = datetime.now().isoformat()
467     dump("day3_repro.json", rec)
468     return rec
469
470
471 # =====
472 # 阶段 freeze —— 按冻结配置补跑 MO 三件并核对
473 # =====
474 def stage_freeze():
475     """
476     最终配置补跑（分工 §三「甲 D3 上午」）。
477
478     冻结的是**配置**，不是结果
479     -----
480     Day3 的 sens 阶段已量出：
481         时间容差  $\times 1 \rightarrow \times 100$ ,  $t^*$  只动 16.7 s (Richardson 估计生产容差下误差 1.9 s)
482         空间网格  $N=100 \rightarrow 200$ ,  $t^*$  动 109 s (**主导误差**)
483     收紧时间容差**不会**改善主导误差，故**冻结配置维持 Day2 生产设置不变**
484     ( $N=200$ ,  $=1.5$ ,  $=1$ , 生产容差)，不重开代价高昂的细网格重跑。
485
486     补跑的目的有两个：
487         把**版本标识**写进结果文件的元数据（CODE-30 要求）；
488         用**同配置重跑**验证盘上的结果确实出自记录的配置
489         —— 这是"冻结"二字唯一有意义的验证方式。
490     """
491     import shutil
492     from src.io.excel_writer import validate_output
493     from src.models.problem3 import integer_second_rule, locate_threshold
494     from src.models.boundary import BoundaryConfig
495     from src.io.resample import output_radii_m, sample_field
496     from src.io.excel_writer import write_result1
497     from src.config import T_START
498
499     section("阶段 freeze —— MO 最终配置补跑与冻结")
500
501     # ---- 备份 Day2 结果（不覆盖即不可回退） ----
502     bak = RESULT_DIR.parent / "MO_day2备份"
503     if not bak.exists():
504         shutil.copytree(RESULT_DIR, bak)

```

```

505     print(f" 已备份 Day2 结果 → {bak.name}/")
506 else:
507     print(f" 备份已存在 → {bak.name}/ (不重复备份) ")
508
509 env = build_default_env()
510 grid = Grid(N=NUMERICS.N, R0=0.02, grading=NUMERICS.grading)
511 r_out = output_radii_m()
512 bc = BoundaryConfig(latent=False)
513 V = version_string()
514 out = {"version": V, "grid": grid.summary(),
515        "theta": 1.0, "tolerances": {
516            "atol_T": NUMERICS.atol_T, "rtol_T": NUMERICS.rtol_T,
517            "atol_C": NUMERICS.atol_C, "rtol_C": NUMERICS.rtol_C}}
518
519 # ---- 问题2 正式版 (3 h, 每 1 s) ----
520 print("\n [1/3] result2.xlsx (3 h 正式版) ...")
521 t0 = time.time()
522 st = P2.Problem2Setup(grid=grid, env=env, bc=bc, t_end=10800.0, label="MO")
523 t_int = np.arange(float(T_START), 10800.0 + 0.5, 1.0)
524 res, ex = P2.solve_problem2(st, t_output=t_int)
525 T_out = sample_field(res.T, grid, ex["T_surf"], r_out)
526 C_out = sample_field(res.C, grid, ex["C_surf"], r_out)
527 meta = {"层次": "MO (题设基线)", "问题": "问题2 (3 h 正式版)",
528         "版本标识": V, "网格": grid.summary(),
529         "时间格式": "=1.0 (Backward Euler, L-稳定)",
530         "长时边界": "附件1 覆盖 0—14400 s, 本文件只用 0—10800 s, 未使用外推",
531         "接受步/拒绝步": f"{res.n_accepted}/{res.n_rejected}"}
532 write_result1(RESULT_DIR / "result2.xlsx", t_int.astype(int), T_out, C_out, meta=meta)
533 rep = validate_output(RESULT_DIR / "result2.xlsx", expect_t=t_int.astype(int))
534 print(f"    {rep['ok']} {time.time()-t0:.0f}s")
535 out["result2_3h"] = {"ok": bool(rep["ok"]), "n_accepted": res.n_accepted,
536                    "C_center_3h": float(ex["C_center"][-1]),
537                    "T_center_3h": float(ex["T_center"][-1]),
538                    "wall_s": time.time() - t0}
539
540 # ---- 问题3: 定位 t* ----
541 print("\n [2/3] 定位 t* (二分到 1e-3 s) ...")
542 t0 = time.time()
543 st_full = P2.Problem2Setup(grid=grid, env=env, bc=bc,
544                             t_end=6.0 * 86400.0, label="MO")
545 tr, res_c, ex_c = locate_threshold(st_full, dt_out=60.0,
546                                    max_horizon=6.0 * 86400.0, target=C_TARGET)
547 el = time.time() - t0
548 assert tr.found, "6 天内未达标"
549 t_star = tr.t_star
550 print(f"    t* = {t_star:.4f} s = {t_star/3600:.4f} h "
551       f"    f"C_max(t*) = {tr.C_max_at:.8f} 二分 {tr.n_bisect} 次 {el:.0f}s")
552
553 # ---- 全程版 + result3 ----
554 print("\n [3/3] result2_全程版.xlsx + result3.xlsx ...")
555 t0 = time.time()
556 grid_t = res_c.times[res_c.times <= np.floor(t_star / 60.0) * 60.0 + 1e-9]
557 k_last = int(np.searchsorted(res_c.times, grid_t[-1]))
558 t_star_int = integer_second_rule(t_star)
559 t_int_out = np.unique(np.concatenate([grid_t, [float(t_star_int)]]))
560 sub = P2.Problem2Setup(grid=grid, env=env, bc=bc, t_end=float(t_star_int),
561                        label="MO")
562 res_seg, ex_seg = P2.solve_problem2(
563     sub, t_output=np.array([float(t_star_int)]), t0=float(grid_t[-1]),

```

```

564     state0=(res_c.T[k_last], res_c.C[k_last]))
565 T_hist = np.vstack([res_c.T[:k_last + 1], res_seg.T[-1][None, :]])
566 C_hist = np.vstack([res_c.C[:k_last + 1], res_seg.C[-1][None, :]])
567 Ts_hist = np.concatenate([ex_c["T_surf"][:k_last + 1], ex_seg["T_surf"]])
568 Cs_hist = np.concatenate([ex_c["C_surf"][:k_last + 1], ex_seg["C_surf"]])
569 T_full = sample_field(T_hist, grid, Ts_hist, r_out)
570 C_full = sample_field(C_hist, grid, Cs_hist, r_out)
571 assert len(t_int_out) == T_full.shape[0]
572 write_result1(RESULT_DIR / "result2_全程版.xlsx", t_int_out.astype(int),
573              T_full, C_full, meta={**meta, "问题": "问题2 (全程版, 至 t*)"})
574 from src.io.excel_writer import write_result3
575 write_result3(RESULT_DIR / "result3.xlsx", t_int_out.astype(int), C_full,
576              meta={"层次": "MO (题设基线)", "问题": "问题3",
577                   "版本标识": V, "网格": grid.summary(),
578                   "判据": f"C_max(t) = max_r C(r,t) < {C_TARGET} kg/kg (未舍入判定)",
579                   "阈值通过时刻 t*": f"{t_star:.4f} s = {t_star/3600:.4f} h",
580                   "末行": f"{t_star_int} s (达标时刻)"})
581 print(f"    全程版 {T_full.shape[0]} 行; result3 末行 {t_int_out[-1]:.0f} s"
582       f"    {time.time()-t0:.0f}s")
583
584 # ---- 与 Day2 记录核对 ----
585 d2 = json.loads((DOC_DIR / "day2_core.json").read_text(encoding="utf-8"))
586 t2 = d2["tstar_M0"]["t_star_s"]
587 out["t_star_s"] = float(t_star)
588 out["t_star_h"] = float(t_star / 3600.0)
589 out["t_star_day2_s"] = float(t2)
590 out["reproduce_delta_s"] = float(t_star - t2)
591 out["C_max_at_tstar"] = float(tr.C_max_at)
592 print("\n    与 Day2 记录核对 ")
593 print(f"    Day2 t* = {t2:.4f} s 本次 {t_star:.4f} s "
594       f"    差 {t_star - t2:+.4f} s")
595 print(f"    判定: {' 同配置复现一致' if abs(t_star-t2) < 1.0 else ' 存在差异, 需查因'}")
596 dump("day3_freeze.json", out)
597 return out
598
599
600 # =====
601 def main():
602     ap = argparse.ArgumentParser()
603     ap.add_argument("--stage", default="sens",
604                    choices=["sens", "conv", "freeze", "repro", "all"])
605     args = ap.parse_args()
606     print(f"Day3 阶段 {args.stage} 版本标识 {version_string()}")
607     t0 = time.time()
608     S = args.stage
609     if S in ("sens", "all"):
610         stage_sens()
611     if S in ("conv", "all"):
612         stage_conv()
613     if S in ("freeze", "all"):
614         stage_freeze()
615     if S in ("repro", "all"):
616         stage_repro()
617     print(f"\n总耗时 {time.time()-t0:.1f} s")
618
619
620 if __name__ == "__main__":
621     main()

```

## B.16 主运行脚本（问题 4）

run\_p4.py (256 行)

```

1  """
2  问题4 主运行脚本                                [甲 • MO • 接替乙的问题4]
3
4  产出
5  ----
6      results/result4.xlsx          正式交付 (Sheet1, 60 s × 0.1 cm, 末列「药材表面」)
7      results/results4_运行信息.xlsx 配置与版本标识
8      docs/表6_问题4.md            题设表6
9      docs/p4_core.json            数值证据
10     docs/p4_grid.json            网格收敛 (与乙的 N=20 对照)
11     docs/p4_rsens.json           半径外推敏感性 (CODE-05)
12
13  用法
14  ----
15     python scripts/run_p4.py --stage core # 主结果 (关键路径)
16     python scripts/run_p4.py --stage grid # 网格收敛 (证明乙的 N=20 不够)
17     python scripts/run_p4.py --stage rsens # 半径外推敏感性
18     python scripts/run_p4.py --stage all
19  """
20
21  from __future__ import annotations
22
23  import argparse
24  import json
25  import sys
26  import time
27  from pathlib import Path
28
29  import numpy as np
30
31  ROOT = Path(__file__).resolve().parents[1]
32  sys.path.insert(0, str(ROOT))
33
34  from src.config import DOC_DIR, NUMERICS, R_OUT_CM, T_TARGET # noqa: E402
35  from src.data_prep.env_extrap import build_env # noqa: E402
36  from src.data_prep.env_interp import load_env # noqa: E402
37  from src.models.problem4 import GridXiN, load_radius, solve_p4 # noqa: E402
38
39  RESULT_DIR = ROOT / "results"
40  TDRY_TARGET = 0.15
41
42
43  def P(*a):
44      print(*a, flush=True)
45
46
47  def dump(name, obj):
48      DOC_DIR.mkdir(parents=True, exist_ok=True)
49      p = DOC_DIR / name
50      p.write_text(json.dumps(obj, ensure_ascii=False, indent=2, default=str),
51                  encoding="utf-8")
52      P(f" → docs/{name}")
53
54
55  def make_env():

```

```

56     return build_env(load_env(), mode="faithful", t_pre=14400.0)
57
58
59 # =====
60 def find_tdry(env, rad, grid, t_max=6 * 86400.0, dt_out=300.0, num=None,
61             R_mode="pchip", verbose=False):
62     """二分/扫描定位 C_max 首次跌破 0.15 的时刻。"""
63     t_out = np.arange(dt_out, t_max + dt_out, dt_out)
64     r = solve_p4(env, rad, grid, t_max, t_out, num=num, R_mode=R_mode,
65                 verbose=verbose)
66     Cm = r["C_max"]
67     below = Cm < TDRY_TARGET
68     if not np.any(below):
69         return float("nan"), r
70     k = int(np.argmax(below))
71     return float(r["times"][k]), r
72
73
74 # =====
75 def stage_core(args):
76     P("=" * 88)
77     P(" 问题4 主体 —— 材料坐标 + 附件2 收缩 + 附录4 物性")
78     P("=" * 88)
79     env, rad = make_env(), load_radius()
80     P(f" {rad.summary()}")
81     grid = GridXiN(N=args.N, grading=args.gamma)
82     P(f" {grid.summary()}")
83     num = NUMERICS
84     out = {"网格": grid.summary(), "半径": rad.summary(),
85           "物性": "附录4", "环境": "附件1 全程 PCHIP, 14400 s 后取平台均值"}
86
87     t0 = time.time()
88     tdry, r = find_tdry(env, rad, grid, num=num, verbose=True)
89     P(f"\n   烘干时长 t_dry = {tdry:.1f} s = {tdry/3600:.4f} h = "
90       f"{tdry/86400:.4f} 天 ({time.time()-t0:.0f}s) ")
91     out["t_dry_s"] = tdry
92     out["t_dry_h"] = tdry / 3600.0
93     out["t_dry_day"] = tdry / 86400.0
94
95     # ---- 末时刻场 + 输出采样 ----
96     R_f = rad(tdry)
97     P(f" 终态半径 R(t_dry) = {R_f*100:.4f} cm")
98     out["R_final_cm"] = R_f * 100
99
100    # 重新积分到 tdry 的**整秒**, 并按 60 s 输出 (正式交付)
101    tdry_int = int(np.ceil(tdry))
102    t_out = np.arange(60, tdry_int + 1, 60, dtype=float)
103    if t_out[-1] < tdry_int:
104        t_out = np.append(t_out, float(tdry_int))
105    P(f" 重跑正式输出网格: {len(t_out)} 个时刻 (60 s 间隔, 末点 {t_out[-1]:.0f} s) ")
106    t0 = time.time()
107    res = solve_p4(env, rad, grid, float(tdry_int), t_out, num=num)
108    P(f"   完成 ({time.time()-t0:.0f}s, {len(res['times'])} 个时刻) ")
109
110    # ---- 采样到物理坐标 ----
111    r_out_cm = np.array(R_OUT_CM, float) # 0, 0.1, ..., 2.0
112    r_out_cm = np.array(R_OUT_CM[:20], float) # result4 距离轴只到 1.9 cm
113    Cmat = np.full((len(res["times"]), len(r_out_cm)), np.nan)
114    Csurf = np.empty(len(res["times"]))

```

```

115 xic = grid.xi          # 节点式网格: 场量就存在节点上 ( _0..._{N-1}, 末节点=1)
116 for i, tk in enumerate(res["times"]):
117     Rk = rad(tk)
118     xi_q = (r_out_cm * 1e-2) / Rk
119     m = xi_q <= 1.0 + 1e-12
120     Cmat[i, m] = np.interp(xi_q[m], xic, res["C"][i])
121     Csurf[i] = res["C"][i][-1]
122 out["末行有效列数"] = int(np.sum(~np.isnan(Cmat[-1])))
123 out["C_center_final"] = float(Cmat[-1, 0])
124 out["C_surf_final"] = float(Csurf[-1])
125 out["C_max_final"] = float(res["C"][-1].max())
126
127 # ---- 写 result4.xlsx ----
128 from src.io.excel_writer import write_result4
129 RESULT_DIR.mkdir(parents=True, exist_ok=True)
130 meta = {
131     "层次": "M0 (题设基线)", "问题": "问题4 (考虑尺寸变化)",
132     "模型": "材料坐标  $r/R(t)$  半控制体 FVM; **无  $\dot{R}$  对流项** (材料导数形式)",
133     "物性": "附录4:  $\tau=760+90C$ ,  $c_p=1850+2150C/(1+C)$ ,  $k=0.12+0.20C/(1+C)$ , "
134         " $D=4.2e-4 \cdot \exp(-0.30/C) \cdot \exp(-3850/T_K)$ ",
135     "半径": f"附件2 PCHIP ({len(rad.t)} 点, 0—{rad.t_max:.0f} s)",
136     "网格": grid.summary(),
137     "时间格式": "=1.0 (Backward Euler, L-稳定) + 块 Gauss-Seidel Picard",
138     "判据": "C_max(t)=max_C < 0.15 kg/kg (未舍入判定)",
139     "烘干时长 t_dry": f"{tdry:.4f} s = {tdry/3600:.4f} h = {tdry/86400:.4f} 天",
140     "终态半径": f"{R_f*100:.4f} cm",
141     "版本标识": "A-2026-M0-P4-r1",
142 }
143 p = RESULT_DIR / "result4.xlsx"
144 write_result4(p, res["times"].astype(int), Cmat, Csurf, meta=meta)
145 P(f"    → results/result4.xlsx ({len(res['times'])} 行 × {len(r_out_cm)+2} 列)")
146
147 from src.io.excel_writer import validate_result4
148 rep = validate_result4(p, expect_t=res["times"].astype(int))
149 P(f"    回读自检: {'通过' if rep.get('ok') else '未通过'} {rep}")
150 out["输出自检"] = bool(rep.get("ok"))
151
152 # ---- 表6 ----
153 rows = []
154 for hh in range(6, int(tdry / 3600) + 1, 6):
155     k = int(np.searchsorted(res["times"], hh * 3600))
156     if k >= len(res["times"]):
157         break
158     rows.append((hh, Cmat[k], Csurf[k], rad(res["times"][k]) * 100))
159 md = ["# 表6 药材烘干过程的水分浓度 (问题4)", "",
160     f"> t_dry = {tdry/3600:.4f} h = {tdry/86400:.4f} 天; ",
161     f"网格 {grid.summary()}", "",
162     "| 时间/h | 0 | 0.5 | 1 | 1.5 | 2 | 药材表面 | 半径/cm |",
163     "|---|---|---|---|---|---|---|---|"]
164 for hh, cm, cs, Rc in rows:
165     cells = []
166     for j, rcm in enumerate(r_out_cm):
167         v = cm[j]
168         cells.append("--" if isinstance(v, float) and np.isnan(v) else f"{v:.4f}")
169     # 只保留 0/0.5/1/1.5/2 五列
170     sel = [0, 5, 10, 15, 20]
171     md.append(f"| {hh} | " + " | ".join(f"{cells[i]}" for i in sel)
172         + f" | {cs:.4f} | {Rc:.4f} |")
173 md.append(f"| **{tdry/3600:.4f} (烘干结束) ** | ")

```

```

174 P("")
175 for line in md[-len(rows) - 1:]:
176     P(" " + line)
177 (DOC_DIR / "表6_问题4.md").write_text("\n".join(md), encoding="utf-8")
178 P("    → docs/表6_问题4.md")
179
180 dump("p4_core.json", out)
181 return out
182
183
184 # =====
185 def stage_grid(args):
186     P("=" * 88)
187     P(" 网格收敛 —— 证明「均匀 N=20」不够（乙用 N=20，本实现用渐变）")
188     P("=" * 88)
189     env, rad = make_env(), load_radius()
190     num = dataclasses.replace(NUMERICS, dt_max=20.0) if False else NUMERICS
191     out = {"行": []}
192     P(f"{'配置':>22} | {'t_dry / h':>11} | {'相对 N=160':>11} | {'耗时':>7}")
193     P(" " + "-" * 62)
194     ref = None
195     cases = [(20, 1.0), (40, 1.0), (80, 1.0), (160, 1.0),
196              (20, 1.5), (40, 1.5), (80, 2.0), (200, 1.5)]
197     for N, g in cases:
198         gr = GridXiN(N=N, grading=g)
199         t0 = time.time()
200         td, _ = find_tdry(env, rad, gr, num=num, dt_out=600.0)
201         el = time.time() - t0
202         if N == 160 and g == 1.0:
203             ref = td
204         rel = (td - ref) / ref * 100 if ref else float("nan")
205         P(f" N={N:4d} = {g:<4} 均匀/渐变 | {td/3600:11.4f} | {rel:+10.2f}% | {el:6.0f}s")
206         out["行"].append({"N": N, "grading": g, "t_dry_s": td, "t_dry_h": td/3600,
207                          "rel_pct": rel, "wall_s": el})
208     dump("p4_grid.json", out)
209     return out
210
211
212 # =====
213 def stage_rsens(args):
214     P("=" * 88)
215     P(" 半径外推敏感性（CODE-05）—— t_dry 超出附件2 覆盖范围时")
216     P("=" * 88)
217     env, rad = make_env(), load_radius()
218     grid = GridXiN(N=args.N, grading=args.gamma)
219     P(f" 附件2 覆盖到 {rad.t_max/3600:.1f} h; 超出后需外推")
220     out = {"附件2_覆盖_h": rad.t_max / 3600, "行": []}
221     P(f"{'外推方式':>14} | {'t_dry / h':>11}")
222     P(" " + "-" * 30)
223     for mode in ("pchip", "const_end", "linear"):
224         td, _ = find_tdry(env, rad, grid, R_mode=mode, dt_out=600.0)
225         P(f" {mode:>14} | {td/3600:11.4f}")
226         out["行"].append({"mode": mode, "t_dry_s": td, "t_dry_h": td/3600})
227     ts = [r["t_dry_h"] for r in out["行"]]
228     out["最大相对差_pct"] = (max(ts) - min(ts)) / min(ts) * 100
229     P(f"\n 三种外推的最大相对差 = {out['最大相对差_pct']:.2f}%")
230     dump("p4_rsens.json", out)
231     return out
232

```



```
233
234 # =====
235 def main():
236     ap = argparse.ArgumentParser()
237     ap.add_argument("--stage", default="core",
238                     choices=["core", "grid", "rsens", "all"])
239     ap.add_argument("--N", type=int, default=200)
240     ap.add_argument("--gamma", type=float, default=1.5)
241     args = ap.parse_args()
242     t0 = time.time()
243     P(f"问题4 运行时: Python {sys.version.split()[0]}, numpy {np.__version__}, "
244       f"网格 N={args.N}/{args.gamma}")
245     if args.stage in ("core", "all"):
246         stage_core(args)
247     if args.stage in ("grid", "all"):
248         stage_grid(args)
249     if args.stage in ("rsens", "all"):
250         stage_rsens(args)
251     P(f"\n总耗时 {time.time()-t0:.1f} s")
252
253
254 if __name__ == "__main__":
255     import dataclasses
256     main()
```

## C 附录 C：结果文件说明

表 37 四个 result 文件的规格

| 文件                | 内容                           | 规格                                                             |
|-------------------|------------------------------|----------------------------------------------------------------|
| result1.xlsx      | 问题 1<br>温度、<br>水分<br>浓度      | 双 sheet, 1 s $\times$ 0.1 cm, 1–1800 s                         |
| result2.xlsx      | 问题 2<br>温度、<br>水分<br>浓度      | 双 sheet, 1 s $\times$ 0.1 cm, 1–10800 s                        |
| result2_ 全程版.xlsx | 问题 2<br>全程<br>版              | 双 sheet, 60 s $\times$ 0.1 cm, 60–206960 s                     |
| result3.xlsx      | 问题 3<br>水分<br>浓度             | 单 sheet, 60 s $\times$ 0.1 cm, 60–206960 s (末行为达标时刻)           |
| result4.xlsx      | 问题 4<br>水分<br>浓度<br>(收<br>缩) | 单 sheet, 60 s, 距离轴只到 <b>1.9 cm</b> , 末列为字符串“药材表面”, 域外留空 (不填 0) |

题设表 1–6 的数值直接取自上述结果文件 (由脚本生成, 不手工抄录), 保留四位小数, 域外位置留空。

## D 附录 D: 图表清单

表 38 本文插图清单

| 编号   | 文件名                    | 内容                    |
|------|------------------------|-----------------------|
| 图 1  | G1_geometry            | 几何、边界与求解路线合图          |
| 图 2  | G1b_flowchart          | 技术路线流程图               |
| 图 3  | G2_env                 | 附件 1 环境数据及插值          |
| 图 4  | FIG-13_smoothing       | 环境插值 vs 受控平滑对照        |
| 图 5  | G2b_stage              | 阶段识别                  |
| 图 6  | G3_radius              | 半径数据、PCHIP 拟合与残差      |
| 图 7  | FIG-08_fvm             | 有限体积离散示意              |
| 图 8  | G4_p1_evolution        | 问题 1 温度与含水率演化         |
| 图 9  | FIG-16_dimensionless   | 无量纲数分析 ( $Bi_m$ 跨越 1) |
| 图 10 | FIG-18_D_contour       | 扩散系数等值线               |
| 图 11 | G5_p2_radial           | 问题 2 前 3 h 径向分布       |
| 图 12 | G6_p3_threshold        | 问题 3 阈值通过时刻与终态        |
| 图 13 | G12_max_center_surface | 中心/表面/最大值/平均四曲线       |
| 图 14 | G7_p4_moving_boundary  | 问题 4 物理坐标与移动边界        |
| 图 15 | G9_abc                 | A/B/C 顺序差分            |
| 图 16 | G11_geo_density        | 几何—密度一致性诊断            |
| 图 17 | FIG-14_bessel          | 解析基准对比                |
| 图 18 | FIG-41_adaptive        | 自适应步长与迭代历史            |
| 图 19 | G8_convergence         | 基准误差与收敛阶              |
| 图 20 | G13_tornado            | 参数灵敏度龙卷风图             |
| 图 21 | G10_scenario           | 参数情景与区间               |
| 图 22 | G14_robust             | 数据扰动稳健性               |
| 图 23 | FIG-43_latent          | 潜热开关对照                |

## E 附录 E: 文献核验说明

本项目在核实参考文献时发现：辅导材料给出的 21 个 DOI 中有 6 个指向错误的论文（占 29%）。正文参考文献中 4 条的 DOI 已更正，凡引用必须回原文核实：

- Younsi 等 (2007) 原写 ...thermalsci.2006.09.006, 该 DOI 实为 Mahmud & Fraser 2006 的另一文 → 已更正为 ...2006.09.001;
- Takhar 等 (2011) 原写 ...jfoodeng.2011.01.026, 该 DOI 实为 Romano 等 2011 激光背散射一文 → 已更正为 ...2011.05.006;
- Mercier 等 (2013) 原写 ...jfoodeng.2013.03.030, 该 DOI 实为 da Silva 等 2013 香蕉圆柱干燥一文 → 已更正为 ...2013.03.024;
- Champion 等 (2000) 原写 10.1016/S0260-8774(00)00028-5, 该 DOI 实为 Menkov 2000 扁豆种子吸湿等温线一文 → 已更正为 10.1016/S0924-2244(00)00047-9。

另有 1 条已整条弃用：辅导材料以 10.1016/S0017-9310(98)00229-8 引用 Ni & Datta (1999) 讨论辐射，但该 DOI 无效，反查到的同作者 1999 年论文主题为油炸，与原用途不符。

关于本文自算量：附录 2 与附录 3 的  $D$  在含水率方向的变化幅度（266.2 倍与 16.8 倍）为本文计算，题面未给出；引用时须写成“本文计算得到”，不得写成“题目给出 265 倍”。